

ACM-XCPC

slmhh

代码模版库



湖北工业大学  
HUBEI UNIVERSITY OF TECHNOLOGY

# Template For XCPC

slmhh @ HBUT

August 22, 2026

## Contents

|                                |           |  |  |
|--------------------------------|-----------|--|--|
| <b>1 数据结构</b>                  | <b>5</b>  |  |  |
| 1.1 单调队列                       | 5         |  |  |
| 1.2 单调栈                        | 5         |  |  |
| 1.3 并查集                        | 5         |  |  |
| 1.4 ST 表                       | 5         |  |  |
| 1.5 线段树                        | 5         |  |  |
| 1.5.1 区间求和                     | 5         |  |  |
| 1.5.2 区间加乘                     | 6         |  |  |
| 1.5.3 Info + Tag               | 7         |  |  |
| 1.6 树状数组                       | 8         |  |  |
| 1.7 李超线段树                      | 8         |  |  |
| 1.8 动态开点李超线段树                  | 9         |  |  |
| 1.9 珂朵莉树                       | 10        |  |  |
| 1.10 主席树 (可持久化线段树)             | 10        |  |  |
| 1.11 笛卡尔树                      | 11        |  |  |
| 1.12 pbds 扩展库实现平衡二叉树           | 11        |  |  |
| 1.13 vector 模拟平衡树              | 11        |  |  |
| <b>2 数论</b>                    | <b>12</b> |  |  |
| 2.1 欧拉筛                        | 12        |  |  |
| 2.1.1 线性筛质数                    | 12        |  |  |
| 2.1.2 线性筛因数个数                  | 12        |  |  |
| 2.1.3 线性筛因数和                   | 12        |  |  |
| 2.2 逆元                         | 12        |  |  |
| 2.2.1 费马小定理                    | 12        |  |  |
| 2.2.2 扩展欧几里得算法                 | 12        |  |  |
| 2.2.3 线性求乘法逆元                  | 12        |  |  |
| 2.3 扩展大步小步算法/exBSGS            | 13        |  |  |
| 2.4 欧拉函数                       | 13        |  |  |
| 2.4.1 单个数的欧拉函数                 | 13        |  |  |
| 2.4.2 线性筛欧拉函数                  | 13        |  |  |
| 2.4.3 欧拉函数常见性质                 | 13        |  |  |
| 2.5 莫比乌斯函数                     | 14        |  |  |
| 2.5.1 单个数的莫比乌斯函数               | 14        |  |  |
| 2.5.2 线性筛莫比乌斯函数                | 14        |  |  |
| 2.5.3 莫比乌斯常见性质                 | 14        |  |  |
| 2.5.4 莫比乌斯反演                   | 15        |  |  |
| 2.6 数论分块                       | 15        |  |  |
| 2.7 矩阵快速幂                      | 15        |  |  |
| 2.8 异或空间线性基 (贪心)               | 15        |  |  |
| 2.8.1 常见操作                     | 15        |  |  |
| 2.9 组合数                        | 16        |  |  |
| 2.9.1 通项公式                     | 16        |  |  |
| 2.9.2 卢卡斯定理                    | 16        |  |  |
| 2.10 中国剩余定理                    | 16        |  |  |
| 2.11 快速傅里叶变换 (FFT)             | 16        |  |  |
| 2.12 快速数论变换 (NTT)              | 17        |  |  |
| 2.13 高斯消元                      | 17        |  |  |
| 2.14 rho 算法与 MR 素性测试           | 18        |  |  |
| 2.15 求解连续按位异或                  | 18        |  |  |
| 2.16 其他结论                      | 18        |  |  |
| <b>3 组合数学</b>                  | <b>20</b> |  |  |
| 3.1 全排列数 (Full Permutation)    | 20        |  |  |
| 3.2 组合数 (Combinatorial Number) | 20        |  |  |
| 3.3 错排列 (derangement)          | 20        |  |  |
| 3.4 第一类斯特林数 (Stirling Number)  | 20        |  |  |
| 3.5 第二类斯特林数 (Stirling Number)  | 21        |  |  |
| 3.6 卡特兰数 (Catalan)             | 21        |  |  |
| 3.7 伯努利数 (Bernoulli)           | 21        |  |  |

|      |                                 |    |        |                 |    |
|------|---------------------------------|----|--------|-----------------|----|
| 3.8  | 斐波那契数列 (Fibonacci)              | 22 | 6.4    | Z 函数 (扩展 KMP)   | 33 |
| 3.9  | 球盒模型                            | 23 | 6.5    | Manacher        | 33 |
| 3.10 | 组合数结论                           | 23 | 6.6    | 字典树             | 33 |
| 3.11 | 多项式与生成函数结论                      | 24 | 6.7    | AC 自动机          | 34 |
| 4    | 博弈论                             | 25 | 6.8    | 后缀数组            | 34 |
| 4.1  | 巴什博弈                            | 25 | 6.9    | 回文自动机 PAM(回文树)  | 35 |
| 4.2  | 扩展巴什博弈                          | 25 | 6.10   | 后缀自动机 (SAM)     | 35 |
| 4.3  | Nim 博弈                          | 25 | 6.10.1 | 后缀自动机应用         | 36 |
| 4.4  | Moore' s Nim 游戏 (Nim-K 游戏)      | 25 | 7      | 图论              | 38 |
| 4.5  | Anti-Nim 游戏 (反 Nim 游戏)          | 25 | 7.1    | Dijkstra 算法     | 38 |
| 4.6  | 阶梯 - Nim 博弈                     | 25 | 7.2    | Floyd 算法        | 38 |
| 4.7  | SG 游戏 (有向图游戏)                   | 26 | 7.3    | 二分图最大匹配 (匈牙利算法) | 38 |
| 4.8  | Anti-SG 游戏 (反 SG 游戏)            | 26 | 7.4    | 差分约束            | 38 |
| 4.9  | Lasker' s-Nim 游戏 ( Multi-SG 游戏) | 26 | 7.5    | 2-SAT           | 39 |
| 4.10 | Every-SG 游戏                     | 26 | 7.6    | 割边              | 40 |
| 4.11 | 威佐夫博弈                           | 26 | 7.7    | 割点              | 40 |
| 4.12 | 斐波那契博弈                          | 27 | 7.8    | 缩点              | 41 |
| 4.13 | 树上删边游戏                          | 27 | 7.9    | 边双连通分量          | 41 |
| 4.14 | 无向图删边游戏 (Fusion Principle 定理)   | 27 | 7.10   | 欧拉路径            | 42 |
| 5    | 计算几何                            | 28 | 7.10.1 | 有向图的欧拉路径        | 42 |
| 5.1  | 极角排序                            | 28 | 7.10.2 | 无向图的欧拉路径        | 42 |
| 5.2  | 二维静态凸包 (Andrew 算法)              | 28 | 8      | 树上问题            | 43 |
| 5.3  | 闵可夫斯基和                          | 28 | 8.1    | 树的直径            | 43 |
| 5.4  | 平面最近点对                          | 29 | 8.2    | 树的重心            | 43 |
| 5.5  | 扫描线                             | 29 | 8.3    | 最近公共祖先 (LCA)    | 43 |
| 5.6  | 平面最远点对 (旋转卡壳)                   | 30 | 8.3.1  | 倍增算法            | 43 |
| 5.7  | 最小矩形覆盖 (旋转卡壳)                   | 30 | 8.3.2  | 重链剖分            | 44 |
| 5.8  | 最小圆覆盖                           | 30 | 8.4    | 树上差分            | 44 |
| 5.9  | 半平面交                            | 31 | 8.5    | 树上点分治           | 44 |
| 6    | 字符串                             | 32 | 8.6    | 树上启发式合并         | 45 |
| 6.1  | 字符串哈希                           | 32 | 8.7    | Prüfer 序列       | 46 |
| 6.2  | 最小表示法                           | 32 |        |                 |    |
| 6.3  | KMP                             | 32 |        |                 |    |

|                    |           |                          |           |
|--------------------|-----------|--------------------------|-----------|
| <b>9 网络流</b>       | <b>47</b> | <b>11.10bitset</b>       | <b>55</b> |
| 9.1 dinic 算法       | 47        | 11.11使用构造函数              | 55        |
| 9.2 最小费用最大流        | 47        | 11.12其他                  | 55        |
| 9.3 网络流建图技巧        | 48        | 11.13编译器设置               | 55        |
| 9.3.1 二分图最大匹配      | 48        | <b>12 杂项</b>             | <b>56</b> |
| 9.3.2 二分图最大权匹配     | 49        | 12.1 动态大小 bitset         | 56        |
| 9.3.3 二者取其一式       | 49        | 12.2 Gosper's Hack       | 56        |
| 9.3.4 最小点覆盖        | 49        | 12.3 绝对众数                | 56        |
| 9.3.5 最大权闭合子图      | 49        | 12.3.1 摩尔投票              | 56        |
| 9.3.6 最大点权独立集问题    | 49        | 12.3.2 拓展摩尔投票            | 56        |
| 9.3.7 最小路径覆盖问题     | 49        | 12.3.3 区间绝对众数 (线段树上摩尔投票) | 56        |
| 9.3.8 公平分配问题       | 50        | 12.4 模拟退火                | 56        |
| 9.3.9 餐饮问题         | 50        | 12.5 莫队                  | 57        |
| 9.3.10 上下界可行流问题    | 50        | 12.6 带修莫队                | 58        |
| 9.3.11 最大密度子图问题    | 50        | 12.7 幻方                  | 58        |
| <b>10 动态规划</b>     | <b>51</b> | 12.8 约瑟夫问题               | 58        |
| 10.1 多重背包 (单调队列优化) | 51        | 12.9 舞蹈链                 | 59        |
| 10.2 分组背包          | 51        | 12.10对拍                  | 60        |
| 10.3 有依赖的背包        | 51        | 12.10.1 Windows          | 60        |
| 10.4 数位 DP         | 51        | 12.10.2 Linux            | 60        |
| 10.5 SOSDP         | 52        | 12.11python 高精度          | 60        |
| 10.6 悬线法           | 52        | 12.12乱七八糟                | 60        |
| 10.7 编辑距离          | 52        |                          |           |
| 10.8 最小斯坦纳树        | 53        |                          |           |
| <b>11 C++ 语法</b>   | <b>54</b> |                          |           |
| 11.1 int128        | 54        |                          |           |
| 11.2 随机数           | 54        |                          |           |
| 11.3 手工哈希          | 54        |                          |           |
| 11.4 速度测试          | 54        |                          |           |
| 11.5 读取一行数字, 个数未知  | 54        |                          |           |
| 11.6 输出流控制         | 54        |                          |           |
| 11.7 查找后继          | 54        |                          |           |
| 11.8 bit 库与位运算函数   | 54        |                          |           |
| 11.9 优先队列          | 55        |                          |           |

# 1 数据结构

## 1.1 单调队列

$O(n)$  滑动窗口最值问题

```
1 //求最大构建递减的单调队列
2 for(int i = 0; i < n; i++){
3     while(!Q.empty() && a[i] >= a[Q.back()]) Q.pop_back();
4     Q.push_back(i);
5     if(i < k - 1) continue;
6     while(!Q.empty() && Q.front() <= i - k) Q.pop_front();
7 }
```

## 1.2 单调栈

$O(n)$  对于每个位置找到其左边第一个比它大的位置

```
1 std::vector<int> stk;
2 for(int i = 1; i <= n; i++){
3     while(!stk.empty() && a[stk.back()][0] <= a[i][0]) stk.pop_back();
4     if(!stk.empty()) ans[stk.back()] += a[i][1];
5     stk.push_back(i);
6 }
```

## 1.3 并查集

```
1 struct DSU{
2     vector<ll> p;
3     DSU ();
4     DSU(int n){init(n);};
5     void init(int n){
6         p.resize(n + 2);
7         for(int i = 1; i <= n; i++)
8             p[i] = i;
9     }
10    ll find(ll x){
11        if(p[x] != x) p[x] = find(p[x]);
12        return p[x];
13    }
14    void merge(ll x, ll y){
15        x = find(x);
16        y = find(y);
17        p[y] = x;
18    }
19 };
```

## 1.4 ST 表

ST 表是用于解决可重复贡献问题的数据结构, 如  $\max(x, y)$ ,  $\gcd(a, b)$  等。

```
1 #include<bits/stdc++.h>
2 #define all(x) (x).begin(),(x).end()
3 using namespace std;
4 typedef long long ll;
5 const int N = 1e5 + 10;
6
7 //P3865 【模板】ST 表
8 //静态区间最值
9 struct ST{
10     vector<vector<ll>> st;
11     ll N, op;
12     ST(vector<ll>& s, int op){
13         N = s.size();
14         st.resize(N + 1, vector<ll>(20, -1e18));
15         for(int i = 1; i <= N; i++){
16             st[i][0] = s[i];
17         }
18         this->op = op;
19         build_st(op);
20     }
21
22     ll opt1(const ll a, const ll b){return max(a, b);}
23     ll opt2(const ll a, const ll b){return min(a, b);}
24
25     ll query(ll l, ll r){
26         ll k = log2(r - l + 1);
27         if(op == 1) return opt1(st[l][k], st[r - (1 << k) + 1][k]);
28         return opt2(st[l][k], st[r - (1 << k) + 1][k]);
29     }
30     void build_st(int op){
31         for(int j = 1; j <= 20; j++){
32             for(int i = 1; i + (1 << j) - 1 <= N; i++){
33                 if(op == 1) st[i][j] = opt1(st[i][j - 1], st[i + (1 << (j - 1))][j - 1]);
34                 else st[i][j] = opt2(st[i][j - 1], st[i + (1 << (j - 1))][j - 1]);
35             }
36         }
37     }
38 };
```

## 1.5 线段树

### 1.5.1 区间求和

线段树可以在  $O(\log N)$  的时间复杂度内实现单点修改、区间修改、区间查询（区间求和, 求区间最大值, 求区间最小值）等操作。

```
1 struct SegmentTree{
2     ll n;
3     struct node{
4         ll date, tag;
5     };
6     vector<node> tr;
```

```

7   SegmentTree(vector<ll>& nums){
8       n = nums.size() - 1;
9       tr.resize(n * 4 + 10);
10      build(1,n,1,nums);
11  }
12  void push_up(ll p){
13      tr[p].date = tr[p << 1].date + tr[(p << 1) | 1].date;
14  }
15  void f(ll p,ll l,ll r,ll x){
16      tr[p].tag += x;
17      tr[p].date += x * (r - l + 1);
18  }
19  void push_down(ll p,ll l,ll r){
20      ll mid = (l + r) >> 1LL;
21      f(p << 1,l,mid,tr[p].tag);
22      f(p << 1 | 1,mid + 1,r,tr[p].tag);
23      tr[p].tag = 0;
24  }
25  void build(ll l,ll r,ll p,vector<ll>& nums){
26      if(l == r){
27          tr[p].date = nums[l];
28          return;
29      }
30      ll m = l + ((r - l) >> 1);
31      build(l,m,p << 1,nums),build(m + 1,r,(p << 1) | 1,nums);
32      push_up(p);
33  }
34  void update(ll l,ll r,ll s,ll t,ll p,ll c){
35      if(l <= s && t <= r){
36          tr[p].date += (t - s + 1) * c;
37          tr[p].tag += c;
38          return;
39      }
40      push_down(p,s,t);
41      ll m = s + ((t - s) >> 1);
42      if(l <= m) update(l,r,s,m,p << 1,c);
43      if(r > m) update(l,r,m + 1,t,(p << 1) | 1,c);
44      push_up(p);
45  }
46  ll query(ll l,ll r,ll s,ll t,ll p){
47      if(l <= s && t <= r) return tr[p].date;
48      push_down(p,s,t);
49      ll m = s + ((t - s) >> 1);
50      ll sum = 0;
51      if(l <= m) sum = query(l,r,s,m,p << 1);
52      if(r > m) sum += query(l,r,m + 1,t,(p << 1) | 1);
53      return sum;
54  }
55  void update(ll l,ll r,ll x){
56      update(l,r,1,n,1,x);
57  }
58  ll query(ll l,ll r){
59      return query(l,r,1,n,1);

```

```

60     }
61 };

```

### 1.5.2 区间加乘

```

1   struct SegmentTree{
2       ll n,mod = 998244353;
3       struct node{
4           ll date = 0,add = 0,mu = 1;
5       };
6       vector<node> tr;
7       SegmentTree(vector<ll>& nums){
8           n = nums.size() - 1;
9           tr.resize(n * 4 + 10);
10          build(1,n,1,nums);
11      }
12      void push_up(ll p){
13          tr[p].date = (tr[p << 1].date + tr[(p << 1) | 1].date) % mod;
14      }
15      void f(ll p,ll l,ll r,ll x,ll k){
16          tr[p].date = (tr[p].date * k + x * (r - l + 1)) % mod;
17          tr[p].mu = (tr[p].mu * k) % mod;
18          tr[p].add = (tr[p].add * k % mod + x) % mod;
19      }
20      void push_down(ll p,ll l,ll r){
21          ll mid = (l + r) >> 1LL;
22          f(p << 1,l,mid,tr[p].add,tr[p].mu);
23          f(p << 1 | 1,mid + 1,r,tr[p].add,tr[p].mu);
24          tr[p].add = 0,tr[p].mu = 1;
25      }
26      void build(ll l,ll r,ll p,vector<ll>& nums){
27          if(l == r){
28              tr[p].date = nums[l];
29              return;
30          }
31          ll m = l + ((r - l) >> 1);
32          build(l,m,p << 1,nums),build(m + 1,r,(p << 1) | 1,nums);
33          push_up(p);
34      }
35      //x is add,k is mul
36      void update(ll l,ll r,ll s,ll t,ll p,ll x,ll k){
37          if(l <= s && t <= r){
38              tr[p].date = (tr[p].date * k % mod + (t - s + 1) * x) % mod;
39              tr[p].mu = (tr[p].mu * k) % mod;
40              tr[p].add = (tr[p].add * k % mod + x) % mod;
41              return;
42          }
43          push_down(p,s,t);
44          ll m = s + ((t - s) >> 1);
45          if(l <= m) update(l,r,s,m,p << 1,x,k);
46          if(r > m) update(l,r,m + 1,t,(p << 1) | 1,x,k);
47          push_up(p);
48      }

```

```

49 ll query(ll l,ll r,ll s,ll t,ll p){
50     if(l <= s && t <= r) return tr[p].date;
51     push_down(p,s,t);
52     ll m = s + ((t - s) >> 1);
53     ll sum = 0;
54     if(l <= m) sum = query(l,r,s,m,p << 1);
55     if(r > m) sum = (sum + query(l,r,m + 1,t,(p << 1) | 1)) % mod;
56     return sum;
57 }
58 void RangeMul(ll l,ll r,ll x){
59     update(l,r,1,n,1,0,x);
60 }
61 void RangeAdd(ll l,ll r,ll x){
62     update(l,r,1,n,1,x,1);
63 }
64 ll query(ll l,ll r){
65     return query(l,r,1,n,1);
66 }
67 };

```

### 1.5.3 Info + Tag

```

1  template<class Info,class Tag>
2  struct SegmentTree{
3      i64 N;
4      std::vector<Info> info;
5      std::vector<Tag> tag;
6      SegmentTree():N(0){};
7      SegmentTree(int _n,Info _v = Info()){init(_n,_v);}
8      template<class T>
9      SegmentTree(std::vector<T> _init){init(_init);}
10     void init(int _n,Info _v = Info()){init(std::vector<Info>(_n,_v));}
11
12     template<class T>
13     void init(std::vector<T> _init){
14         N = _init.size();
15         info.assign(4 << std::lg(N),Info());
16         tag.assign(4 << std::lg(N),Tag());
17         auto build = [&](auto&& self,int p,int l,int r) -> void{
18             if(l == r){
19                 info[p] = _init[l];
20                 return;
21             }
22             int m = (l + r) >> 1;
23             self(self,p << 1,l,m);
24             self(self,(p << 1) | 1,m + 1,r);
25             push_up(p);
26         };
27         build(build,1,1,N);
28     }
29     void push_up(int p){
30         info[p] = info[p << 1] + info[(p << 1) | 1];
31     }

```

```

32 void apply(int p,const Tag &v){
33     info[p].apply(v);
34     tag[p].apply(v);
35 }
36 void push_down(int p){
37     apply(p << 1,tag[p]);
38     apply((p << 1) | 1,tag[p]);
39     tag[p] = Tag();
40 }
41 void modify(int p,int l,int r,int x,int y,const Tag &v){
42     if(l > y || r < x) return;
43     if(x <= l && r <= y){
44         apply(p,v);
45         return;
46     }
47     push_down(p);
48     i64 m = (l + r) >> 1;
49     modify(p << 1,l,m,x,y,v);
50     modify((p << 1) | 1,m + 1,r,x,y,v);
51     push_up(p);
52 }
53 void modify(int l, int r, const Tag &v) {
54     return modify(1,1,N,l,r,v);
55 }
56 Info query(int p,int l,int r,int x,int y){
57     if(l > y || r < x) return Info();
58     if(x <= l && r <= y) return info[p];
59     push_down(p);
60     i64 m = (l + r) >> 1;
61     return query(p << 1,l,m,x,y) + query((p << 1) | 1,m + 1,r,x,y);
62 }
63 Info query(int l,int r){
64     return query(1,1,N,l,r);
65 }
66 };
67 struct Tag{
68     i64 x = 0;
69     void apply(const Tag &t) & {
70         x += t.x;
71     }
72 };
73 struct Info{
74     i64 x = 0,len = 1;
75     void apply(const Tag &t) & {
76         x += t.x * len;
77     }
78
79     Info operator+(const Info &other) const{
80         return Info{x + other.x,len + other.len};
81     }
82 };

```

## 1.6 树状数组

树状数组是一种支持单点修改和区间查询的，代码量小的数据结构。普通树状数组维护的信息及运算要满足结合律且可差分，如加法（和）、乘法（积）、异或等。

```

1 struct BIT{
2     vector<i64> tr;
3     BIT():tr(N){};
4     void update(i64 p,i64 x){
5         for(int i = p;i <= 1e5;i += i & -i)
6             tr[i] += x;
7     }
8     i64 query(i64 p){
9         i64 res = 0;
10        for(int i = p;i >= 1;i -= i & -i)
11            res += tr[i];
12        return res;
13    }
14 };

```

前驱后继扩展（常规 + 第  $k$  小值查询 + 元素排名查询 + 元素前驱后继查询）

注意，被查询的值都应该小于等于  $N$ ，否则会越界；如果离散化不可使用，则需要使用平衡树替代。

```

1 struct BIT {
2     int n;
3     vector<int> w;
4     BIT(int n) : n(n), w(n + 1) {}
5     void add(int x, int v) {
6         for (; x <= n; x += x & -x)
7             w[x] += v;
8     }
9     int kth(int x) { // 查找第 k 小的值
10        int ans = 0;
11        for (int i = __lg(n); i >= 0; i--) {
12            int val = ans + (1 << i);
13            if (val < n && w[val] < x) {
14                x -= w[val];
15                ans = val;
16            }
17        }
18        return ans + 1;
19    }
20    int get(int x) { // 查找 x 的排名
21        int ans = 1;
22        for(x--; x; x -= x & -x)
23            ans += w[x];
24        return ans;
25    }
26    int pre(int x) { return kth(get(x) - 1); } // 求小于 x 的最大值（前驱）
27    int suf(int x) { return kth(get(x + 1)); } // 求大于 x 的最小值（后继）
28 };

```

## 1.7 李超线段树

加入一个一次函数，定义域为  $[l, r]$ ，给定  $k$ ，求定义域包含  $k$  的所有一次函数中，在  $x = k$  处取值最大的那个，如果有多个函数取值相同，选编号最小的。

```

1 struct Lichao{
2     #define ls (p << 1)
3     #define rs ((p << 1) | 1)
4     static constexpr double eps = 1e-8;
5     struct line{
6         double k,b;
7         int id; // 部分题需要记录线段编号
8         line(double k = 0,double b = 0,int id = 0):k(k),b(b),id(id){};
9         double calc(int x){ return k * x + b;} // 计算线段在pos位置的纵坐标
10        double cross(const line& T){ return (T.b - b) / (k - T.k);} // 求两条线段的交点横坐标
11    };
12    struct node{
13        int l,r; // 区间左右端点
14        line L; // 最优线段
15        bool flag; // 记录区间内是否有最优线段
16        node(){};
17        node(int l,int r,line L,bool flag):l(l),r(r),L(L),flag(flag){};
18    };
19    std::vector<node> tr;
20    Lichao(){}
21    Lichao(int N):tr(N << 2){
22        build(1,1,N);
23    };
24    void build(int p,int l,int r){
25        tr[p] = {l,r,line(),false};
26        if(l == r) return;
27        int mid = (l + r) >> 1;
28        build(ls,l,mid),build(rs,mid + 1,r);
29    }
30    //O(log^2 n)
31    void update(int p,int L,int R,line ln){
32        int l = tr[p].l,r = tr[p].r;
33        if(L <= l && r <= R){ //如果插入线段的定义域覆盖整个区间
34            double lp = tr[p].L.calc(l),rp = tr[p].L.calc(r);
35            double lq = ln.calc(l),rq = ln.calc(r);
36
37            if(!tr[p].flag) tr[p].L = ln,tr[p].flag = true;
38            else if(lq - lp > eps && rq - rp > eps) tr[p].L = ln;
39            else if(lq - lp > eps || rq - rp > eps){
40                int mid = (l + r) >> 1;
41                if(ln.calc(mid) - tr[p].L.calc(mid) > eps) std::swap(tr[p].L,ln);
42                if(ln.calc(l) - tr[p].L.calc(l) > eps) update(ls,L,R,ln);
43                else update(rs,L,R,ln);
44            }
45        }
46        else{
47            int mid = (l + r) >> 1;
48            if(L <= mid) update(ls,L,R,ln);

```



```

49     if(R > mid) update(rs,L,R,ln);
50 }
51 }
52 //O(log n)
53 std::pair<double,int> query(int p,int pos){ //返回最大值和线段编号
54     int l = tr[p].l,r = tr[p].r;
55     double ans = tr[p].L.calc(pos);
56     int id = tr[p].L.id;
57     if(l == r) return {ans,id};
58     int mid = (l + r) >> 1;
59     std::pair<double,int> tq;
60     if(pos <= mid) tq = query(ls,pos);
61     else tq = query(rs,pos);
62
63     if(tq.first > ans || (fabs(tq.first - ans) < eps && tq.second < id))
64         ans = tq.first,id = tq.second;
65     return {ans,id};
66 }
67 std::pair<double,int> query(int x){
68     return query(1,x);
69 }
70 void update(i64 x0,i64 y0,i64 x1,i64 y1,i64 id){
71     line ln;
72     if(x0 == x1) ln = {0,std::max(y1,y0),id};
73     else{
74         double tk = (double)(y1 - y0) / (x1 - x0);
75         double tb = (double)(y0 - tk * x0);
76         ln = {tk,tb,id};
77     }
78     update(1,x0,x1,ln);
79 }
80 };

```

## 1.8 动态开点李超线段树

```

1 struct Lichao{
2     static constexpr double eps = 1e-8;
3     struct Line{
4         double k,b;
5         int id;
6         Line(double k = 0,double b = 0,int id = 0):k(k),b(b),id(id){};
7         double calc(int x){return k * x + b;}
8         double cross(const Line& T){return (T.b - b) / (k - T.k);}
9     };
10    struct Node{
11        int l,r,ls,rs;
12        Line L;
13        bool flag;
14        Node(){};
15        Node(int l,int r):l(l),r(r),ls(0),rs(0),flag(false){};
16    };
17    std::vector<Node> tr;

```

```

18 int cnt;
19 int L_range,R_range;
20 Lichao(int L, int R):L_range(L),R_range(R),cnt(1){
21     tr.push_back(Node());
22     tr.push_back(Node(L,R));
23 }
24 void new_node(int& p,int l,int r){
25     p = ++cnt;
26     tr.push_back(Node(l,r));
27 }
28 void update(int p,int L,int R,Line ln){
29     int l = tr[p].l, r = tr[p].r;
30     int mid = (l + r) >> 1;
31     if(L <= l && r <= R){
32         if(!tr[p].flag){
33             tr[p].L = ln;
34             tr[p].flag = true;
35             return;
36         }
37         double lp = tr[p].L.calc(l),rp = tr[p].L.calc(r);
38         double lq = ln.calc(l),rq = ln.calc(r);
39         if(!tr[p].flag) tr[p].L = ln,tr[p].flag = true;
40         else if(lq - lp > eps && rq - rp > eps) tr[p].L = ln;
41         else if(lq - lp > eps || rq - rp > eps){
42             int mid = (l + r) >> 1;
43             if(ln.calc(mid) - tr[p].L.calc(mid) > eps) std::swap(tr[p].L,ln);
44             if(ln.calc(l) - tr[p].L.calc(l) > eps){
45                 if(!tr[p].ls) new_node(tr[p].ls,l,mid);
46                 update(tr[p].ls,L,R,ln);
47             }
48             else{
49                 if(!tr[p].rs) new_node(tr[p].rs,mid + 1,r);
50                 update(tr[p].rs,L,R,ln);
51             }
52         }
53     }
54     else{
55         if(L <= mid){
56             if(!tr[p].ls) new_node(tr[p].ls,l,mid);
57             update(tr[p].ls,L,R,ln);
58         }
59         if(R > mid){
60             if(!tr[p].rs) new_node(tr[p].rs,mid + 1,r);
61             update(tr[p].rs,L,R,ln);
62         }
63     }
64 }
65 std::pair<double,int> query(int p,int pos){
66     int l = tr[p].l,r = tr[p].r;
67     double ans = tr[p].L.calc(pos);
68     int id = tr[p].L.id;
69     if(l == r) return {ans,id};
70     int mid = (l + r) >> 1;

```

```

71
72     std::pair<double,int> tq;
73     if(pos <= mid && tr[p].ls) tq = query(tr[p].ls, pos);
74     else if(pos > mid && tr[p].rs) tq = query(tr[p].rs, pos);
75     else tq = {-1e18,0};
76
77     if(tq.first > ans || (fabs(tq.first - ans) < eps && tq.second < id))
78         ans = tq.first, id = tq.second;
79     return {ans,id};
80 }
81 std::pair<double,int> query(int x){
82     return query(1,x);
83 }
84 void update(i64 x0,i64 y0,i64 x1,i64 y1,i64 id){
85     Line ln;
86     if(x0 == x1) ln = {0,std::max(y0,y1),id};
87     else{
88         double tk = (double)(y1 - y0) / (x1 - x0);
89         double tb = y0 - tk * x0;
90         ln = {tk, tb,id};
91     }
92     update(1,x0,x1,ln);
93 }
94 };

```

## 1.9 珂朵莉树

珂朵莉树的适用范围是有区间赋值操作且数据随机的题目。在随机数据下，珂朵莉树可以达到  $O(n \log \log n)$  的复杂度。

```

1 struct node{
2     ll l, r;
3     mutable ll v;
4     node(ll l, ll r, ll v) : l(l), r(r), v(v) {}
5     bool operator<(const node &o) const { return l < o.l; }
6 };
7 struct ODT{
8     set<node> tr;
9     ODT(){};
10    auto split(ll p){
11        auto t = tr.lower_bound({p,0,0});
12        if(t != tr.end() && t->l == p) return t;
13        t--;
14        auto [l,r,v] = *t;
15        tr.erase(t);
16        tr.insert({l,p-1,v});
17        return tr.insert({p,r,v}).first;
18    }
19    void assign(ll l,ll r,ll v){
20        auto end = split(r+1), begin = split(l);
21        tr.erase(begin, end);
22        tr.insert(node(l,r,v));

```

```

23    }
24    void add(ll l,ll r,ll v){
25        auto end = split(r+1);
26        for (auto it = split(l); it != end; it++)
27            it->v += v;
28    }
29    ll kth(ll l,ll r,ll k){
30        auto end = split(r+1);
31        vector<PII> v;
32        for (auto it = split(l); it != end; it++)
33            v.push_back({it->v, it->r - it->l + 1});
34        sort(v.begin(), v.end());
35        for (int i = 0; i < v.size(); i++){
36            k -= v[i].second;
37            if (k <= 0)
38                return v[i].first;
39        }
40    }
41    ll sum_of_pow(ll l, ll r, ll x, ll y){
42        ll tot = 0;
43        auto end = split(r+1);
44        for (auto it = split(l); it != end; it++)
45            tot = (tot + qpow(it->v, x, y) * (it->r - it->l + 1)) % y;
46        return tot;
47    }
48 };

```

## 1.10 主席树 (可持久化线段树)

以  $O(N \log N)$  的时间复杂度建树、查询、修改。

```

1 struct PresidentTree {
2     static constexpr int N = 2e5 + 10;
3     int cntNodes, root[N];
4     struct node {
5         int l, r;
6         int cnt;
7     } tr[4 * N + 17 * N];
8     void modify(int &u, int v, int l, int r, int x) {
9         u = ++cntNodes;
10        tr[u] = tr[v];
11        tr[u].cnt++;
12        if (l == r) return;
13        int mid = (l + r) / 2;
14        if (x <= mid)
15            modify(tr[u].l, tr[v].l, l, mid, x);
16        else
17            modify(tr[u].r, tr[v].r, mid + 1, r, x);
18    }
19    int kth(int u, int v, int l, int r, int k) {
20        if (l == r) return l;
21        int res = tr[tr[v].l].cnt - tr[tr[u].l].cnt;
22        int mid = (l + r) / 2;

```

```

23     if (k <= res)
24         return kth(tr[u].l, tr[v].l, 1, mid, k);
25     else
26         return kth(tr[u].r, tr[v].r, mid + 1, r, k - res);
27 }
28 };

```

## 1.11 笛卡尔树

```

1 template<typename T>
2 struct CarTree{
3     struct node{
4         int l,r;
5         node():l(0),r(0){}
6     };
7     vector<node> tr;
8     CarTree(){};
9     CarTree(vector<T>& a){
10         stack<T> stk;
11         tr.resize(a.size());
12         for(int i = 1;i < a.size();i++){
13             int last = 0;
14             while(!stk.empty() && a[stk.top()] > a[i]){
15                 last = stk.top();
16                 stk.pop();
17             }
18             if(last != 0) tr[i].l = last;
19             if(!stk.empty()) tr[stk.top()].r = i;
20             stk.push(i);
21         }
22     }
23 };

```

## 1.12 pbds 扩展库实现平衡二叉树

记得加上相应的头文件，同时需要注意定义时的参数，一般只需要修改第三个参数：即定义的是大根堆还是小根堆。

```

1 empty() / size()
2 insert(x) // 插入元素x
3 erase(x) // 删除元素/迭代器x
4 order_of_key(x) // 返回元素x的排名
5 find_by_order(x) // 返回排名为x的元素迭代器
6 lower_bound(x) / upper_bound(x) // 返回迭代器
7 join(Tree) // 将Tree树的全部元素并入当前的树
8 split(x, Tree) // 将大于x的元素放入Tree树

```

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 using namespace __gnu_pbds;
3 using V = pair<int, int>;
4 tree<V, null_type, less<V>, rb_tree_tag, tree_order_statistics_node_update> ver;

```

```

5 map<int, int> dic;
6
7 int n; cin >> n;
8 for (int i = 1, op, x; i <= n; i++) {
9     cin >> op >> x;
10    if (op == 1) { // 插入一个元素x, 允许重复
11        ver.insert({x, ++dic[x]});
12    } else if (op == 2) { // 删除元素x, 若有重复, 则任意删除一个
13        ver.erase({x, dic[x]--});
14    } else if (op == 3) { // 查询元素x的排名 (排名定义为比当前数小的数的个数+1)
15        cout << ver.order_of_key({x, 1}) + 1 << endl;
16    } else if (op == 4) { // 查询排名为x的元素
17        cout << ver.find_by_order(--x)->first << endl;
18    } else if (op == 5) { // 查询元素x的前驱
19        int idx = ver.order_of_key({x, 1}) - 1; // 无论x存不存在, idx都代表x的位置, 需要-1
20        cout << ver.find_by_order(idx)->first << endl;
21    } else if (op == 6) { // 查询元素x的后继
22        int idx = ver.order_of_key({x, dic[x]}); // 如果x不存在, 那么idx就是x的后继
23        if (ver.find({x, 1}) != ver.end()) idx++; // 如果x存在, 那么idx是x的位置, 需要+1
24        cout << ver.find_by_order(idx)->first << endl;
25    }
26 }

```

## 1.13 vector 模拟平衡树

```

1 int main(){
2     std::ios::sync_with_stdio(0), std::cin.tie(0), std::cout.tie(0);
3     i64 n;
4     std::cin >> n;
5     std::vector<i64> a;
6     while(n--){
7         i64 opt,x;
8         std::cin >> opt >> x;
9         //插入O(log n)
10        if(opt == 1) a.insert(lower_bound(a.begin(),a.end(),x),x);
11        //删除O(log n)
12        else if(opt == 2) a.erase(lower_bound(a.begin(),a.end(),x));
13        //查询排名O(log n)
14        else if(opt == 3) std::cout << lower_bound(a.begin(),a.end(),x) - a.begin() + 1 << "\n";
15        //查询排名对应数字O(1)
16        else if(opt == 4) std::cout << a[x - 1] << "\n";
17        //前驱O(log n)
18        else if(opt == 5) std::cout << *(lower_bound(a.begin(),a.end(),x) - 1) << "\n";
19        //后继O(log n)
20        else std::cout << *upper_bound(a.begin(),a.end(),x) << "\n";
21    }
22    return 0;
23 }

```

## 2 数论

### 2.1 欧拉筛

#### 2.1.1 线性筛质数

```

1 bitset<N> isprime;
2 vector<i64> prime;
3 void euler() {
4     isprime.set();
5     isprime[1] = 0;
6     for (i64 i = 2; i <= N; i++) {
7         if (isprime[i]) prime.push_back(i);
8         for (int j = 0; j < prime.size() && i * prime[j] < N; j++) {
9             isprime[i * prime[j]] = false;
10            if (i % prime[j] == 0) break;
11        }
12    }
13 }
```

#### 2.1.2 线性筛因数个数

```

1 vector<int> d(N); // 因数个数
2 vector<int> minp_cnt(N, 1); // 记录最小质因子的指数+1
3 void euler_d(){
4     d[1] = 1;
5     for (i64 i = 2; i < N; i++){
6         if (isprime[i]){
7             d[i] = minp_cnt[i] = 2;
8             prime.push_back(i);
9         }
10        for (int j = 0; j < prime.size() && i * prime[j] < N; j++){
11            i64 p = prime[j];
12            if (i % p == 0){
13                minp_cnt[i * p] = minp_cnt[i] + 1;
14                d[i * p] = d[i] / minp_cnt[i] * minp_cnt[i * p];
15                break;
16            }
17            else{
18                d[i * p] = d[i] * 2;
19                minp_cnt[i * p] = 2;
20            }
21        }
22    }
23 }
```

#### 2.1.3 线性筛因数和

```

1 vector<i64> sigma(N); // 因数和
2 vector<i64> minp_pow(N, 1); // 记录最小质因子的幂次和
3 void euler_sigma(){
```

```

4     sigma[1] = 1;
5     for (i64 i = 2; i < N; i++) {
6         if (isprime[i]){
7             sigma[i] = minp_pow[i] = i + 1;
8             prime.push_back(i);
9         }
10        for (int j = 0; j < prime.size() && i * prime[j] < N; j++){
11            i64 p = prime[j];
12            if (i % p == 0){
13                minp_pow[i * p] = minp_pow[i] * p + 1;
14                sigma[i * p] = sigma[i] / minp_pow[i] * minp_pow[i * p];
15                break;
16            }
17            else{
18                sigma[i * p] = sigma[i] * sigma[p];
19                minp_pow[i * p] = 1 + p;
20            }
21        }
22    }
23 }
```

### 2.2 逆元

如果一个线性同余方程  $a^x \equiv 1 \pmod{b}$ , 则  $x$  称为  $a \bmod b$  的逆元, 记作  $a^{-1}$ 。

#### 2.2.1 费马小定理

设  $p$  是素数。对于任意整数  $a$  且  $p \nmid a$ , 都成立  $a^{p-1} \equiv 1 \pmod{p}$ 。

```

1 i64 inv(i64 x){
2     return qpow(x, mod - 2);
3 }
```

#### 2.2.2 扩展欧几里得算法

其中  $x$  为  $a$  的逆元

```

1 int exgcd(int a, int b, int &x, int &y){
2     if (b == 0){
3         x = 1;
4         y = 0;
5         return a;
6     }
7     int d = exgcd(b, a % b, y, x);
8     y -= (a / b) * x;
9     return d;
10 }
```

#### 2.2.3 线性求乘法逆元

$$i^{-1} \equiv -\left\lfloor \frac{p}{i} \right\rfloor (p \bmod i)^{-1} \pmod{p}$$

```

1 inv[1] = 1;
2 for(int i = 2; i <= n; i++)
3     inv[i] = (long long)(p - p / i) * inv[p % i] % p;

```

## 2.3 扩展大步小步算法/exBSGS

以  $\mathcal{O}(\sqrt{P})$  的复杂度求解  $a^x \equiv b \pmod{P}$  的最小自然数  $x$ 。其中标准 BSGS 算法不能计算  $a$  与  $MOD$  互质的情况，而 *exbsgs* 则可以。

```

1 i64 BSGS(i64 a, i64 b, i64 p, i64 k = 1){
2     i64 A = sqrtl(p) + 1;
3     map<i64, i64> mp;
4     i64 cur = 1;
5     for(int i = 1; i <= A; i++){
6         cur = (cur * a) % p;
7         mp[(cur * b) % p] = i;
8     }
9     i64 temp = cur * k % p;
10    for(int i = 1; i <= A; i++){
11        auto it = mp.find(temp);
12        if(it != mp.end()) return i * A - it->second;
13        temp = (temp * cur) % p;
14    }
15    return -1e18;
16 }
17 i64 exBSGS(i64 a, i64 b, i64 m, i64 k = 1){ //O(sqrt(m)logm)
18     a %= m, b %= m;
19     i64 A = a, B = b, M = m;
20     if(b == 1) return 0;
21     i64 cur = 1 % m;
22     for(int i = 0; i <= A; i++){
23         if(cur == B) return i;
24         cur = cur * A % M;
25         i64 d = __gcd(a, m);
26         if(b % d) return -1e18;
27         if(d == 1) return BSGS(a, b, m, k * a % m) + i + 1;
28         k = k * a / d % m, b /= d, m /= d;
29     }
30 }
31 int main(){
32     ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
33     i64 a, b, p;
34     cin >> a >> b >> p;
35     i64 temp = exBSGS(a, b, p);
36     if(temp >= 0) cout << temp << "\n";
37     else cout << "No Solution\n";
38     return 0;
39 }

```

## 2.4 欧拉函数

若  $n = p^k$ ，其中  $p$  是质数，那么  $\varphi(n) = p^k - p^{k-1}$

由唯一分解定理，设  $n = \prod_{i=1}^s p_i^{k_i}$ ，其中  $p_i$  是质数，有  $\varphi(n) = n \times \prod_{i=1}^s \frac{p_i - 1}{p_i}$ 。

### 2.4.1 单个数的欧拉函数

1 到  $N$  中与  $N$  互质的个数称为欧拉函数，记作  $\varphi(N)$ 。求解欧拉函数的过程即为分解质因数的过程，复杂度  $\mathcal{O}(\sqrt{n})$ 。

```

1 int euler_phi(int n) {
2     int ans = n;
3     for (int i = 2; i * i <= n; i++)
4         if (n % i == 0) {
5             ans = ans / i * (i - 1);
6             while (n % i == 0) n /= i;
7         }
8     if (n > 1) ans = ans / n * (n - 1);
9     return ans;
10 }

```

### 2.4.2 线性筛欧拉函数

```

1 vector<int> phi(N);
2 void euler_phi(){
3     phi[1] = 1;
4     for(i64 i = 2; i < N; i++){
5         if(isprime[i]){
6             phi[i] = i - 1;
7             prime.push_back(i);
8         }
9         for(int j = 0; j < prime.size() && i * prime[j] < N; j++) {
10             i64 p = prime[j];
11             if(i % p == 0){
12                 phi[i * p] = phi[i] * p;
13                 break;
14             }
15             else phi[i * p] = phi[i] * (p - 1);
16         }
17     }
18 }

```

### 2.4.3 欧拉函数常见性质

- 1 -  $n$  中与  $n$  互质的数之和为  $n \cdot \frac{\varphi(n)}{2}$ ;
- 若  $a, b$  互质，则  $\varphi(a \cdot b) = \varphi(a) \cdot \varphi(b)$ 。实际上，所有满足这一条件的函数统称为积性函数；

- 若  $f$  是积性函数，且有  $n = \prod_{i=1}^m p_i^{c_i}$ ，那么  $f(n) = \prod_{i=1}^m f(p_i^{c_i})$ ;

- 若  $p$  为质数, 且满足  $p \mid n$ :
  - $n \bmod p^2 = 0$ , 那么  $\varphi(n) = \varphi(\frac{n}{p}) \cdot p$ ;
  - $n \bmod p^2 \neq 0$ , 那么  $\varphi(n) = \varphi(\frac{n}{p}) \cdot (p - 1)$ ;
- 欧拉反演:  $\sum_{d \mid n} \varphi(d) = n$ ;  
如  $n = 10$ , 则  $d = 10/5/2/1$ , 那么  $10 = \varphi(10) + \varphi(5) + \varphi(2) + \varphi(1)$ ;
- $\sum_{i=1}^n \gcd(i, n) = \sum_{d \mid n} \left\lfloor \frac{n}{d} \right\rfloor \varphi(d)$ ;
- $\varphi(n) = \sum_{d \mid n} \mu(d) \frac{n}{d}$ ;
- $\sum_{i=1}^n \varphi(n) = \sum_{d=1}^n \mu(d) \frac{\lfloor \frac{n}{d} \rfloor (\lfloor \frac{n}{d} \rfloor + 1)}{2}$ ;

## 2.5 莫比乌斯函数

$\mu$  为莫比乌斯函数, 定义为:

$$\mu(n) = \begin{cases} 1 & n = 1 \\ 0 & n \text{ 含有平方因子} \\ (-1)^k & k \text{ 为 } n \text{ 的本质不同质因子个数} \end{cases}$$

### 2.5.1 单个数的莫比乌斯函数

$\mathcal{O}(\sqrt{n})$ 。

```

1 int mu(int n) {
2     int res = 1;
3     for(int i = 2; i * i <= n; i++){
4         if(n % i == 0){
5             n /= i;
6             if (n % i == 0) return 0;
7             res = -res;
8         }
9     }
10    if (n > 1) res = -res;
11    return res;
12 }
```

### 2.5.2 线性筛莫比乌斯函数

```

1 vector<int> mu(N);
2 void euler_mu(){
3     mu[1] = 1;
4     for(i64 i = 2; i < N; i++){
5         if(isprime[i]){
6             mu[i] = -1;
7             prime.push_back(i);
8         }
9         for(int j = 0; j < prime.size() && i * prime[j] < N; j++){
10            i64 p = prime[j];
11            if(i % p == 0){
12                mu[i * p] = 0;
13                break;
14            }
15            else mu[i * p] = -mu[i];
16        }
17    }
18 }
```

### 2.5.3 莫比乌斯常见性质

- 常见积性函数之间的关系:
  - 常数函数:  $1(n) = 1$
  - 单位函数:  $\epsilon(n) = [n = 1] = \sum_{d \mid n} \mu(d) \Leftrightarrow \epsilon = \mu * 1; f * \epsilon = \epsilon * f = f$
  - 恒等函数:  $id(n) = n = \sum_{d \mid n} \varphi(d) \Leftrightarrow id = \varphi * 1$
  - 幂函数:  $id_k(n) = n^k$
  - 约数个数:  $\tau(n) = \sum_{d \mid n} 1 \Leftrightarrow \tau = 1 * 1$
  - 约数和:  $\sigma(n) = \sum_{d \mid n} d \Leftrightarrow \sigma = id * 1$
  - 约数函数:  $\sigma_k(n) = \sum_{d \mid n} d^k \Leftrightarrow \sigma_k = id_k * 1$
  - 欧拉函数:  $\varphi(n) = \sum_{i=1}^n [gcd(x, i) = 1] = \sum_{d \mid n} d \cdot \mu(\frac{n}{d}) \Leftrightarrow \varphi = id * \mu$
  - 莫比乌斯函数:  $\mu = \mu * \epsilon$
- $[gcd(a, b) == 1] = \sum_{d \mid gcd(a, b)} \mu(d)$

### 2.5.4 莫比乌斯反演

如果有  $f(n) = \sum_{d|n} g(d)$ , 那么有  $g(n) = \sum_{d|n} \mu(d)f(\frac{n}{d})$ 。

如果  $g = f * 1$ , 那么  $f = g * \mu$

一般流程: 求解  $\sum_{i=1}^n \sum_{j=1}^m f(\gcd(i, j))$

- 考虑构造出函数  $g$ , 满足下列形式:

$$f(n) = g * 1 = \sum_{d|n} g(d)$$

- 则求和式子变为:

$$\sum_{i=1}^n \sum_{j=1}^m \sum_{d|\gcd(i, j)} g(d)$$

- 考虑算术基本定理, 发现若满足  $d|\gcd(i, j)$ , 则  $d|i$  且  $d|j$  成立。考虑  $g(d)$  在何时做出贡献, 调整枚举顺序:

$$\sum_{d=1}^n g(d) \sum_{i=1}^n [d|i] \sum_{j=1}^m [d|j]$$

- $\sum_{i=1}^n [d|i]$  等价于  $1 \sim n$  中  $d$  的倍数的个数, 则上式等价于:

$$\sum_{d=1}^n g(d) \left\lfloor \frac{n}{d} \right\rfloor \left\lfloor \frac{m}{d} \right\rfloor$$

## 2.6 数论分块

$\left\lfloor \frac{n}{l} \right\rfloor = \left\lfloor \frac{n}{l+1} \right\rfloor = \dots = \left\lfloor \frac{n}{r} \right\rfloor \iff \left\lfloor \frac{n}{l} \right\rfloor \leq \frac{n}{r} < \left\lfloor \frac{n}{l} \right\rfloor + 1$ , 根据不等式左侧, 得到

$$r \leq \left\lfloor \frac{n}{\left\lfloor \frac{n}{l} \right\rfloor} \right\rfloor。$$

```

1 void solve(i64 n){
2     i64 ans = 0;
3     for(i64 i = 1, j; i <= n; i = j + 1){
4         j = n / (n / i);
5         ans += (j - i + 1) * (n / i);
6     }
7     cout << ans << "\n";
8 }

```

## 2.7 矩阵快速幂

```

1 static constexpr int mod = 1e9 + 7, N = 2;
2 using Martix = array<array<int, N>, N>;
3 Martix mul(Martix& a, Martix& b){
4     Martix res{};
5     for(int k = 0; k < N; k++){
6         for(int i = 0; i < N; i++){
7             if(a[i][k] == 0) continue;
8             for(int j = 0; j < N; j++){
9                 res[i][j] = (res[i][j] + ((long long)a[i][k] * b[k][j]) % mod) % mod;
10            }
11        }
12    }
13    return res;
14 }
15 Martix qpow(Martix& a, int b){
16     Martix temp{};
17     for(int i = 0; i < N; i++) temp[i][i] = 1;
18     while(b){
19         if(b & 1) temp = mul(temp, a);
20         a = mul(a, a);
21         b >>= 1;
22     }
23     return temp;
24 }

```

## 2.8 异或空间线性基 (贪心)

```

1 struct xorBase{
2     vector<i64> p;
3     xorBase():p(61){}
4     void insert(i64 x){
5         for(int i = 60; i >= 0; i--){
6             if(x >> i){
7                 if(p[i] ^ x ^ p[i]){
8                     else{p[i] = x; return;}
9             }
10        }
11    }
12    i64 getmx() {
13        i64 ans = 0;
14        for(int i = 60; i >= 0; i--){
15            if((ans ^ p[i]) > ans) ans ^= p[i];
16        }
17        return ans;
18    }
19 };

```

### 2.8.1 常见操作

- 求最小值: 如果是求用线性基  $p$  内的元素能异或出的最小值, 那么就是最小的  $p_i$ 。如果是求整个序列能异或出的最小值的话, 需检查有没有元素不能插入线性

基, 如果有, 那么最小值就是 0, 否则依然是最小的  $p_i$ 。

- 求  $k$  小值: 从一个序列中取任意元素进行异或, 求能异或出的所有数字中第  $k$  小的那个。首先处理这个序列的线性基  $p$ , 对于每一个  $p_i$ , 枚举  $j = i \sim 1$ , 如果  $p_i$  二进制的第  $j$  位为 1, 那么  $p_i$  异或上  $p_{j-1}$ 。将  $k$  先转成二进制, 假如第  $i$  位为 1, 则  $ans$  异或线性基中第  $i$  个元素 (注意不是直接异或  $p_{i-1}$ )。
- 询问存在性: 判断数  $x$  能否被当前线性基中元素异或得到。把  $x$  尝试插入进线性基, 假如可以插入, 说明不能异或得到, 假如插不进去, 则说明可以异或得到。

## 2.9 组合数

### 2.9.1 通项公式

```
1 i64 fac[N], invfac[N];
2 i64 calc(i64 m, i64 n){
3     return (fac[n] * ((invfac[m] * invfac[n - m]) % mod)) % mod;
4 }
5 void init(){
6     fac[0] = fac[1] = 1;
7     invfac[0] = invfac[1] = 1;
8     for(int i = 2; i < N; i++){
9         fac[i] = i;
10        invfac[i] = (mod - mod / i) * invfac[mod % i] % mod;
11    }
12    for(int i = 2; i < N; i++){
13        fac[i] = fac[i] * fac[i - 1] % mod;
14        invfac[i] = invfac[i] * invfac[i - 1] % mod;
15    }
16 }
```

### 2.9.2 卢卡斯定理

$$C_m^n \equiv C_{m \bmod p}^{n \bmod p} \cdot C_{\lfloor m/p \rfloor}^{\lfloor n/p \rfloor} \pmod{p}$$

```
1 i64 C(i64 m, i64 n, i64 p){
2     return m < n ? 0 : fact[m] * inv(fact[n], p) % p * inv(fact[m - n], p) % p;
3 }
4 i64 lucas(i64 m, i64 n, i64 p){
5     return n == 0 ? 1 % p : lucas(m / p, n / p, p) * C(m % p, n % p, p) % p;
6 }
```

## 2.10 中国剩余定理

```
1 template<typename T>
2 T exgcd(T a, T b, T &x, T &y){
3     if(b == 0){
```

```
4         x = 1;
5         y = 0;
6         return a;
7     }
8     T d = exgcd(b, a % b, y, x);
9     y = y - (a / b) * x;
10    return d;
11 }
12
13 template<typename T>
14 T inv(T a, T p){
15     T x, y;
16     exgcd(a, p, x, y);
17     return (x % p + p) % p;
18 }
19
20 int main(){
21     ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
22     int T = 1;
23     // cin >> T;
24     while(T--){
25         ll n;
26         __int128 M = 1;
27         cin >> n;
28         vector<ll> a(n + 1), m(n + 1);
29         for(int i = 1; i <= n; i++){
30             cin >> m[i] >> a[i];
31             M *= m[i];
32         }
33         __int128 sum = 0;
34         for(int i = 1; i <= n; i++){
35             __int128 tm = M / m[i];
36             sum = (sum + ((__int128)a[i] * tm) % M * inv(tm, (__int128)m[i] % M) % M;
37         }
38         cout << (ll)sum << "\n";
39     }
40     return 0;
41 }
```

## 2.11 快速傅里叶变换 (FFT)

```
1 #include<bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5 const int N = 4e6 + 10;
6 const double pi = acos(-1.0);
7
8 complex<double> f[N], g[N];
9 int R[N];
10 void change(complex<double> A[], ll n){
11     for(int i = 0; i < n; i++){
```



```

12     R[i] = R[i / 2] / 2 + (i & 1) * (n / 2);
13     for(int i = 0; i < n; i++)
14         if(i < R[i]) swap(A[i], A[R[i]]);
15 }
16
17 void FFT(complex<double> A[], ll n, int op){ //op为虚部符号, op为1时FFT, op为-1时IFFT
18     change(A, n); //位逆序变换
19     for(int m = 2; m <= n; m <= 1){ //枚举位宽
20         complex<double> w1({cos(2 * pi / m), sin(2 * pi / m) * op});
21         for(int i = 0; i < n; i += m){ //枚举块数
22             complex<double> wk({1, 0});
23             for(int j = 0; j < m / 2; j++){
24                 complex<double> x = A[i + j], y = A[i + j + m / 2] * wk;
25                 A[i + j] = x + y;
26                 A[i + j + m / 2] = x - y;
27                 wk = wk * w1;
28             }
29         }
30     }
31 }
32
33 int main(){
34     ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
35     ll n, m;
36     cin >> n >> m;
37     for(ll i = 0; i <= n; i++) cin >> f[i];
38     for(ll i = 0; i <= m; i++) cin >> g[i];
39     ll len = 1 << max((ll)ceil(log2(n + m)), 1LL); //FFT需要项数为2的整数次方倍, len为第一个大于a.size
40     FFT(f, len, 1), FFT(g, len, 1); //系数表达转点值表达
41     for(int i = 0; i <= len; i++)
42         f[i] = f[i] * g[i];
43     FFT(f, len, -1); //点值表达转系数表达
44     for(ll i = 0; i <= m + n; i++)
45         cout << (ll)(f[i].real() / len + 0.5) << " ";
46     return 0;
47 }

```

## 2.12 快速数论变换 (NTT)

```

1 static constexpr int N = 4e6 + 6, g = 3, mod = 998244353;
2 i64 ni, gi;
3 i64 qpow(i64 a, i64 b){
4     i64 ans = 1;
5     for(; b; a = a * a % mod, b >>= 1)
6         if(b & 1) ans = ans * a % mod;
7     return ans;
8 }
9 void NTT(vector<i64>& A, int n, int op){
10     vector<i64> R(n + 1, 0);
11     for(int i = 0; i < n; i++)
12         R[i] = R[i / 2] / 2 + (i & 1) * (n / 2);

```

```

13     for(int i = 0; i < n; i++){
14         if(i < R[i]) swap(A[i], A[R[i]]);
15     }
16     for(int i = 2; i <= n; i <= 1){
17         i64 g1 = qpow(op == 1 ? g : gi, (mod - 1) / i);
18         for(int j = 0; j < n; j += i){
19             i64 gk = 1;
20             for(int k = j; k < j + i / 2; k++){
21                 i64 x = A[k], y = gk * A[k + i / 2] % mod;
22                 A[k] = (x + y) % mod;
23                 A[k + i / 2] = (x - y + mod) % mod;
24                 gk = gk * g1 % mod;
25             }
26         }
27     }
28     void clac(vector<i64>& a, vector<i64>& b){
29         const int la = a.size(), lb = b.size();
30         int len = 1 << max((int)ceil(log2(la + lb)), 1);
31         vector<i64> f(3 * len + 1, 0), h(3 * len + 1, 0);
32         for(int i = 0; i < la; i++) f[i] = a[i];
33         for(int i = 0; i < lb; i++) h[i] = b[i];
34         gi = qpow(g, mod - 2), ni = qpow(len, mod - 2);
35         NTT(f, len, 1), NTT(h, len, 1);
36         for(int i = 0; i <= len; i++){
37             f[i] = (f[i] * h[i]) % mod;
38             NTT(f, len, -1);
39             for(int i = 0; i <= la + lb - 2; i++)
40                 cout << (f[i] * ni) % mod << " ";
41         }

```

## 2.13 高斯消元

```

1 static constexpr double eps = 1e-8;
2 int Gauss(std::vector<std::vector<double>>& a){
3     int n = a.size() - 1, now = 1;
4     for(int i = 1; i <= n; i++){ //枚举列
5         int r = now;
6         for(int j = i; j <= n; j++){ //找最大减小误差
7             if(fabs(a[j][i]) > fabs(a[r][i])) r = j;
8         }
9         if(fabs(a[r][i]) < eps) continue;
10        if(r != now) swap(a[now], a[r]);
11        for(int j = 1; j <= n; j++){
12            if(j == now) continue;
13            double div = a[j][i] / a[now][i];
14            for(int k = i; k <= n + 1; k++) a[j][k] -= a[now][k] * div;
15        }
16        now++;
17    }
18    if(now <= n){
19        while(now <= n){
20            if(fabs(a[now][n + 1]) > eps) return -1;

```

```

21         now++;
22     }
23     return 0;
24 }
25 for(int i = 1; i <= n; i++) a[i][n + 1] /= a[i][i];
26 return 1;
27 } // 1 有解, -1 无解, 0 无穷解

```

## 2.14 rho 算法与 MR 素性测试

```

1 std::mt19937_64 rng {std::chrono::steady_clock::now().time_since_epoch().count()};
2 i64 gcd(i64 a, i64 b);
3 i64 qpow(i64 a, i64 n, i64 p);
4 bool Mii64er_Rabin(i64 x){
5     if (x < 3) return x == 2;
6     if (x % 2 == 0) return false;
7     constexpr i64 A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022};
8     i64 d = x - 1, r = 0;
9     while (d % 2 == 0) d /= 2, ++r;
10    for (auto a : A){
11        i64 v = qpow(a, d, x);
12        if (v <= 1 || v == x - 1) continue;
13        for(int i = 0; i < r; ++i){
14            v = (__int128)v * v % x;
15            if (v == x - 1 && i != r - 1){
16                v = 1;
17                break;
18            }
19            if (v == 1) return false;
20        }
21        if (v != 1) return false;
22    }
23    return true;
24 }
25 i64 Poi64ard_Rho(i64 N){
26     if (N == 4) return 2;
27     if (Mii64er_Rabin(N)) return N;
28     while(true){
29         i64 c = rng() % (N - 1) + 1;
30         auto f = [=](i64 x) {return ((__int128)x * x + c) % N;};
31         i64 t = 0, r = 0, p = 1, q;
32         do{
33             for(int i = 0; i < 128; ++i){
34                 t = f(t), r = f(f(r));
35                 if (t == r || (q = (__int128)p * abs(t - r) % N) == 0)
36                     break;
37                 p = q;
38             }
39             i64 d = gcd(p, N);
40             if (d > 1) return d;
41         }while (t != r);
42     }

```

```

43     return N;
44 }
45 unordered_map<i64, i64> um;
46 i64 max_prime_factor(i64 x){
47     if(um.count(x)) return um[x];
48     i64 fac = Pollard_Rho(x);
49     if(fac == 1 || fac == x) um[x] = x;
50     else um[x] = max(max_prime_factor(fac), max_prime_factor(x / fac));
51     return um[x];
52 }

```

## 2.15 求解连续按位异或

以  $\mathcal{O}(1)$  复杂度计算  $0 \oplus 1 \oplus \cdots \oplus n$ 。

```

1 unsigned xor_n(unsigned n) {
2     unsigned t = n & 3;
3     if (t & 1) return t / 2u ^ 1;
4     return t / 2u ^ n;
5 }

```

```

1 i64 xor_n(i64 n) {
2     if (n % 4 == 1) return 1;
3     else if (n % 4 == 2) return n + 1;
4     else if (n % 4 == 3) return 0;
5     else return n;
6 }

```

## 2.16 其他结论

- 欧拉降幂公式 (扩展欧拉定理):

$$a^b \equiv \begin{cases} a^{b \bmod \varphi(m)}, & \gcd(a, m) = 1 \\ a^b, & \gcd(a, m) \neq 1, b < \varphi(m) \\ a^{b \bmod \varphi(m) + \varphi(m)}, & \gcd(a, m) \neq 1, b \geq \varphi(m) \end{cases}$$

- 约数个数定理: 设  $n = p_1^{a_1} \times p_2^{a_2} \times \cdots \times p_m^{a_m}$ , 其中  $p_1, p_2, \cdots, p_m$  均为互不相等的质数, 则  $n$  的因数个数为:  $d(n) = \prod_{i=1}^m (a_i + 1)$ .

- 约数和定理:  $\sigma(n) = \sum_{d|n} d = \prod_{i=1}^k \frac{p_i^{a_i+1} - 1}{p_i - 1}$

- 约数个数前缀和:  $\sum_{i=1}^n d(i) = \sum_{i=1}^n \lfloor \frac{n}{i} \rfloor$

- 约数和前缀和:  $\sum_{i=1}^n \sigma(i) = \sum_{i=1}^n \lfloor \frac{n}{i} \rfloor i$

- 勒让德定理 (Legendre 定理): 对于质数  $p$ , 函数  $v_p(n)$  为  $n$  的标准分解后  $p$  的次数, 有

$$v_p(n!) = \sum_{i=1}^{\infty} \lfloor \frac{n}{p^i} \rfloor$$

令函数  $s_p(n)$  为  $n$  在  $p$  进制下的数位和有:

$$v_p(n!) = \frac{n - s_p(n)}{p - 1}$$

$p = 2$  时有:

$$v_2(n!) = n - \text{popcount}(n)$$

kummer 定理, 二项式系数:

$$v_p(\binom{n}{m}) = \frac{s_p(m) + s_p(n - m) - s_p(n)}{p - 1}$$

- 霍尔定理 (Hall 定理): 二分图存在完美匹配的判定: 对于一个二分图  $G(V_1, V_2, E) (|V_1| \leq |V_2|)$ , 对与  $\forall v_1 \subseteq V_1, v_2 = \{v_j | (v_i, v_j) \in E, v_i \in V_1, v_j \in V_2\}$ , 其存在完美匹配的条件为  $\forall v_1 \in V_1, |v_1| \leq |v_2|$ .
- Dilworth 定理: 偏序集能划分成的最少的全序集个数等于最大反链的元素个数, 即数组的不重叠的上升子序列数量最小值 = 逆序数组的最长上升子序列长度。
- 柯尼希定理 (König 定理): 二分图最小点覆盖的点数 = 最大匹配数。
- 切比雪夫距离和曼哈顿距离: 将一个点  $(x, y)$  的坐标变为  $(x + y, x - y)$  后, 原坐标系中的曼哈顿距离 = 新坐标系中的切比雪夫距离  
将一个点  $(x, y)$  的坐标变为  $(\frac{x+y}{2}, \frac{x-y}{2})$  后, 原坐标系中的切比雪夫距离 = 新坐标系中的曼哈顿距离
- 基姆拉尔森公式: 用于计算某一天是星期几:

$$h = (d + \lfloor \frac{13(m+1)}{5} \rfloor + y + \lfloor \frac{y}{4} \rfloor + \lfloor \frac{c}{4} \rfloor + 5c) \mod 7$$

- $a - b \leq a \oplus b \leq a + b$

- 对于给定的  $X$  和序列  $[a_1, a_2, \dots, a_n]$ , 有:  $X = (X \& a_1) \text{ or } (X \& a_2) \text{ or } \dots \text{ or } (X \& a_n)$ .

- $\gcd(x, y) = x \oplus y \rightarrow \gcd(x, y) = y - x$

- $\gcd(x, y) = x + y \rightarrow \gcd(x, y) = \max(x, y)$

- 以下式子成立:  $\gcd(a, m) = \gcd(a + x, m) \Leftrightarrow \gcd(a, m) = \gcd(x, m)$ 。求解上式满足条件的  $x$  的数量即为求比  $\frac{m}{\gcd(a, m)}$  小且与其互质的数的个数, 即用欧拉函数求解  $\varphi\left(\frac{m}{\gcd(a, m)}\right)$ 。

- 令  $\begin{cases} X = a + b \\ Y = a \oplus b \end{cases}$ , 如果该式子有解, 那么存在前提条件  $\begin{cases} X \geq Y \\ X, Y \text{ 同奇偶} \end{cases}$ ; 进一步, 此时最小的  $a$  的取值为  $\frac{X - Y}{2}$

- $x + y = x|y + x \& y$ , 对于两个数字  $x$  和  $y$ , 如果将  $x$  变为  $x|y$ , 同时将  $y$  变为  $x \& y$ , 那么在本质上即将  $x$  二进制模式下的全部 1 移动到了  $y$  的对应的位置上

- 一个正整数  $x$  异或、加上另一个正整数  $y$  后奇偶性不发生变化:  $a + b \equiv a \oplus b \pmod{2}$

- 贝叶斯公式:

$$P(A_i|B) = \frac{P(A_i B)}{P(B)} = \frac{P(A_i)P(B|A_i)}{\sum_{j=1}^n P(A_j)P(B|A_j)}$$

- 全概率公式:

$$P(B) = \sum_{i=1}^n P(A_i)P(B|A_i)$$

- 全期望公式:

$$E(Y) = E(E(Y|X)) = \sum_X E(Y|X)P(X)$$

- $\forall a, b, c \in \mathbb{Z}, \lfloor \frac{a}{bc} \rfloor = \left\lfloor \frac{\lfloor \frac{a}{b} \rfloor}{c} \right\rfloor$

### 3 组合数学

#### 3.1 全排列数 (Full Permutation)

从  $n$  个不同元素中, 任取  $m(m \leq n, m$  与  $n$  均为自然数, 下同) 个元素按照一定的顺序排成一行, 叫做从  $n$  个不同元素中取出  $m$  个元素的一个排列。

- 通项公式:

$$A_n^m = \frac{n!}{(n-m)!}$$

#### 3.2 组合数 (Combinatorial Number)

从  $n$  个不同元素中每次取出  $m$  个不同元素的所有组合个数, 记作  $C_n^m$  或  $\binom{n}{m}$ ;

- 1. 通项公式:

$$C_n^m = \frac{P_n^m}{P_m} = \frac{n!}{m!(n-m)!}, C_n^0 = 1.$$

- 2. 递推式:

$$C_n^m = C_{n-1}^{m-1} + C_{n-1}^m$$

- 3. 对称性:

$$C_n^m = C_n^{n-m}$$

#### 3.3 错排列 (derangement)

错位排列是没有任何元素出现在其有序位置的排列。即, 对于  $1 \sim n$  的排列  $P$ , 如果满足  $P_i \neq i$ , 则称  $P$  是  $n$  的错位排列。

- 1. 通项公式:

$$D_n = n! - n! \sum_{k=1}^n \frac{(-1)^{k-1}}{k!} = n! \sum_{k=0}^n \frac{(-1)^k}{k!}$$

错位排列数列的前几项为 0, 1, 2, 9, 44, 265(OEIS A000166)。

- 2. 递推式:

$$D_n = (n-1)(D_{n-1} + D_{n-2})$$

或

$$D_n = nD_{n-1} + (-1)^n$$

1 1, 0, 1, 2, 9, 44, 265, 1854, 14833, 133496, 1334961

#### 3.4 第一类斯特林数 (Stirling Number)

第一类斯特林数 (斯特林轮换数)  $\left[ \begin{smallmatrix} n \\ k \end{smallmatrix} \right]$ , 也可记做  $s(n, k)$ , 表示将  $n$  个两两不同的元素, 划分为  $k$  个互不区分的非空轮换的方案数。

- 递推式

$$\left[ \begin{smallmatrix} n \\ k \end{smallmatrix} \right] = \left[ \begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \right] + (n-1) \left[ \begin{smallmatrix} n-1 \\ k \end{smallmatrix} \right]$$

$$\text{边界是 } \left[ \begin{smallmatrix} n \\ 0 \end{smallmatrix} \right] = [n=0].$$

- 结论

$$- \left[ \begin{smallmatrix} n \\ 1 \end{smallmatrix} \right] = (n-1)!$$

$$- \left[ \begin{smallmatrix} n \\ n \end{smallmatrix} \right] = 1$$

$$- \left[ \begin{smallmatrix} n \\ n-1 \end{smallmatrix} \right] = C_n^2$$

$$- \left[ \begin{smallmatrix} n \\ k \end{smallmatrix} \right] = (n-1)! \sum_{k=1}^{n-1} \frac{1}{k} = (n-1)! H_{n-1}$$

$$- n! = \sum_{k=0}^n \left[ \begin{smallmatrix} n \\ k \end{smallmatrix} \right]$$

$$- \left[ \begin{smallmatrix} p \\ k \end{smallmatrix} \right] \equiv 0 \pmod{p}$$

$$- \left[ \begin{smallmatrix} p \\ k \end{smallmatrix} \right] \equiv (p-1)! \equiv -1 \pmod{p}$$

```

1 0: 1
2 1: 0, 1
3 2: 0, 1, 1
4 3: 0, 2, 3, 1
5 4: 0, 6, 11, 6, 1
6 5: 0, 24, 50, 35, 10, 1
7 6: 0, 120, 274, 225, 85, 15, 1
8 7: 0, 720, 1764, 1624, 735, 175, 21, 1

```

```

9 8: 0,5040,13068, 13132, 6769, 1960, 322, 28, 1
10 9: 0,40320,109584,118124, 67284, 22449, 4536, 546, 36, 1
11 10: 0,362880,1026576,1172700,723680, 269325, 63273, 9450, 870, 45, 1

```

### 3.5 第二类斯特林数 (Stirling Number)

第二类斯特林数 (斯特林子集数)  $\left\{ \begin{matrix} n \\ k \end{matrix} \right\}$ , 也可记做  $S(n, k)$ , 表示将  $n$  个两两不同的元素, 划分为  $k$  个互不区分的非空子集的方案数。

- 1. 递推式

$$\left\{ \begin{matrix} n \\ k \end{matrix} \right\} = \left\{ \begin{matrix} n-1 \\ k-1 \end{matrix} \right\} + k \left\{ \begin{matrix} n-1 \\ k \end{matrix} \right\}$$

边界是  $\left\{ \begin{matrix} n \\ 0 \end{matrix} \right\} = [n=0]$ 。

- 2. 通项公式

$$\left\{ \begin{matrix} n \\ m \end{matrix} \right\} = \sum_{i=0}^m \frac{(-1)^{m-i} i^n}{i!(m-i)!}$$

```

1 0: 1
2 1: 0, 1
3 2: 0, 1, 1
4 3: 0, 1, 3, 1
5 4: 0, 1, 7, 6, 1
6 5: 0, 1, 15, 25, 10, 1
7 6: 0, 1, 31, 90, 65, 15, 1
8 7: 0, 1, 63, 301, 350, 140, 21, 1
9 8: 0, 1, 127, 966, 1701, 1050, 266, 28, 1
10 9: 0, 1, 255, 3025, 7770, 6951, 2646, 462, 36, 1
11 10: 0, 1, 511, 9330, 34105, 42525, 22827, 5880, 750, 45, 1

```

### 3.6 卡特兰数 (Catalan)

Catalan 数列  $H_n$  可以应用于以下问题:

- 1. 有  $2n$  个人排成一行进入剧场。入场费 5 元。其中只有  $n$  个人有一张 5 元钞票, 另外  $n$  人只有 10 元钞票, 剧院无其它钞票, 问有多少种方法使得只要有 10 元的人买票, 售票处就有 5 元的钞票找零?
- 2. 有一个大小为  $n \times n$  的方格图左下角为  $(0,0)$  右上角为  $(n,n)$ , 从左下角开始每次都只能向右或者向上走一单位, 不走到对角线  $y=x$  上方 (但可以触碰) 的情况下到达右上角有多少可能的路径?

- 3. 在圆上选择  $2n$  个点, 将这些点成对连接起来使得所得到的  $n$  条线段不相交的方法数?
- 4. 对角线不相交的情况下, 将一个凸多边形区域分成三角形区域的方法数?
- 5. 一个栈 (无穷大) 的进栈序列为  $1, 2, 3, \dots, n$  有多少个不同的出栈序列?
- 6.  $n$  个结点可构造多少个不同的二叉树
- 7. 由  $n$  个  $+1$  和  $n$  个  $-1$  组成的  $2n$  个数  $a_1, a_2, \dots, a_{2n}$ , 其部分和满足  $a_1 + a_2 + \dots + a_k \geq 0$  ( $k = 1, 2, 3, \dots, 2n$ ), 有多少个满足条件的数列?
- 递推式:

$$H_n = \frac{\binom{2n}{n}}{n+1} (n \geq 2, n \in \mathbf{N}_+)$$

边界是  $\left\{ \begin{matrix} n \\ 0 \end{matrix} \right\} = [n=0]$ 。

- 其他:

$$H_n = \begin{cases} \sum_{i=1}^n H_{i-1} H_{n-i} & n \geq 2, n \in \mathbf{N}_+ \\ 1 & n = 0, 1 \end{cases}$$

$$H_n = \frac{H_{n-1}(4n-2)}{n+1}$$

$$H_n = \binom{2n}{n} - \binom{2n}{n-1}$$

```

1 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900, 2674440

```

### 3.7 伯努利数 (Bernoulli)

伯努利数  $B_n$  可以应用于自然数幂和公式问题:

$$\sum_{k=1}^m k^n = \frac{1}{n+1} \sum_{k=0}^n \binom{n+1}{k} B_k m^{n+1-k}$$

- 生成函数定义:

$$\frac{x}{e^x - 1} = \sum_{n=0}^{\infty} B_n \frac{x^n}{n!}$$

- 递推关系:

$$\sum_{k=0}^m \binom{m+1}{k} B_k = 0 \quad (m \geq 1)$$

边界条件为  $B_0 = 1$

- 显式公式:

$$B_n = \sum_{k=0}^n \frac{1}{k+1} \sum_{j=0}^k (-1)^j \binom{k}{j} j^n$$

- 前几个伯努利数:

$$B_0 = 1, \quad B_1 = -\frac{1}{2}, \quad B_2 = \frac{1}{6}, \quad B_3 = 0, \\ B_4 = -\frac{1}{30}, \quad B_5 = 0, \quad B_6 = \frac{1}{42}, \quad B_7 = 0$$

### 3.8 斐波那契数列 (Fibonacci)

- 递推定义:

$$F_n = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F_{n-1} + F_{n-2} & n \geq 2 \end{cases}$$

- 通项公式 (比内公式):

$$F_n = \frac{1}{\sqrt{5}} (\phi^n - \psi^n)$$

其中  $\phi = \frac{1+\sqrt{5}}{2}, \psi = \frac{1-\sqrt{5}}{2}$

- 矩阵形式:

$$\begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n$$

- 生成函数:

$$\sum_{n=0}^{\infty} F_n x^n = \frac{x}{1-x-x^2}$$

- 直接结论:

— 卡西尼性质:  $F_{n-1} * F_{n+1} - F_n^2 = (-1)^n$  ;

$$- F_n^2 + F_{n+1}^2 = F_{2n+1} ;$$

$$- F_{n+1}^2 - F_{n-1}^2 = F_{2n} \text{ (由上一条写两遍相减得到);}$$

$$- \text{若存在序列 } a_0 = 1, a_n = a_{n-1} + a_{n-3} + a_{n-5} + \dots (n \geq 1) \text{ 则 } a_n = F_n (n \geq 1)$$

— 齐肯多夫定理: 任何正整数都可以表示成若干个不连续的斐波那契数 ( $F_2$  开始) 可以用贪心实现。

- 求和公式结论:

$$- \text{奇数项求和: } F_1 + F_3 + F_5 + \dots + F_{2n-1} = F_{2n} ;$$

$$- \text{偶数项求和: } F_2 + F_4 + F_6 + \dots + F_{2n} = F_{2n+1} - 1 ;$$

$$- \text{平方和: } F_1^2 + F_2^2 + F_3^2 + \dots + F_n^2 = F_n * F_{n+1} ;$$

$$- F_1 + 2F_2 + 3F_3 + \dots + nF_n = nF_{n+2} - F_{n+3} + 2 ;$$

$$- -F_1 + F_2 - F_3 + \dots + (-1)^n F_n = (-1)^n (F_{n+1} - F_n) + 1 ;$$

$$- F_{2n-2m-2}(F_{2n} + F_{2n+2}) = F_{2m+2} + F_{4n-2m} .$$

- 数论结论:

$$- F_a \mid F_b \Leftrightarrow a \mid b ;$$

$$- \gcd(F_a, F_b) = F_{\gcd(a,b)} ;$$

$$- \text{当 } p \text{ 为 } 5k \pm 1 \text{ 型素数时, } \begin{cases} F_{p-1} \equiv 0 \pmod{p} \\ F_p \equiv 1 \pmod{p} \\ F_{p+1} \equiv 1 \pmod{p} \end{cases} ;$$

$$- \text{当 } p \text{ 为 } 5k \pm 2 \text{ 型素数时, } \begin{cases} F_{p-1} \equiv 1 \pmod{p} \\ F_p \equiv -1 \pmod{p} \\ F_{p+1} \equiv 0 \pmod{p} \end{cases} ;$$

$$- F(n) \% m \text{ 的周期} \leq 6m \text{ ( } m = 2 \times 5^k \text{ 时取到等号);}$$

— 既是斐波那契数又是平方数的有且仅有 1, 144 .

1 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610

### 3.9 球盒模型

给定  $n$  个小球  $m$  个盒子。

- 球同、盒不同、不能空:

隔板法:  $N$  个球共  $N-1$  个空, 分成  $M$  堆即  $M-1$  个隔板, 答案为  $\binom{n-1}{m-1}$ 。

- 球同、盒不同、能空:

隔板法: 多出  $M-1$  个虚空球, 答案为  $\binom{m-1+n}{n}$ 。

- 球同、盒同、能空:

$\frac{1}{(1-x)(1-x^2)\dots(1-x^m)}$  的  $x^n$  项的系数。动态规划, 答案为

$$dp[i][j] = \begin{cases} dp[i][j-1] + dp[i-j][j] & i \geq j \\ dp[i][j-1] & i < j \\ 1 & j == 1 \parallel i \leq 1 \end{cases}$$

- 球同、盒同、不能空:

$\frac{x^m}{(1-x)(1-x^2)\dots(1-x^m)}$  的  $x^n$  项的系数。动态规划, 答案为

$$dp[n][m] = \begin{cases} dp[n-m][m] & n \geq m \\ 0 & n < m \end{cases}$$

- 球不同、盒同、不能空:

第二类斯特林数  $\text{Stirling2}(n, m)$ , 答案为

$$dp[n][m] = \begin{cases} m * dp[n-1][m] + dp[n-1][m-1] & 1 \leq m < n \\ 1 & 0 \leq n == m \\ 0 & m == 0 \& 1 \leq n \end{cases}$$

- 球不同、盒同、能空:

第二类斯特林数之和  $\sum_{i=1}^m \text{Stirling2}(n, m)$ , 答案为  $\sum_{i=0}^m dp[n][i]$ 。

- 球不同、盒不同、不能空:

第二类斯特林数乘上  $m$  的阶乘  $m! \cdot \text{Stirling2}(n, m)$ , 答案为  $dp[n][m] * m!$ 。

- 球不同、盒不同、能空: 答案为  $m^n$ 。

### 3.10 组合数结论

- 广义二项式定理:  $\frac{1}{(1-x)^n} = \sum_{k=0}^{\infty} C_{n+k-1}^{k-1} x^k$

- $k \cdot C_n^k = n \cdot C_{n-1}^{k-1}$ ;

- $C_k^n \cdot C_m^k = C_m^n \cdot C_{m-n}^{m-k}$ ;

- $C_n^k + C_n^{k+1} = C_{n+1}^{k+1}$ ;

- $\sum_{i=0}^n C_n^i = 2^n$ ;

- $\sum_{i=0}^n C_n^i \cdot i = n \cdot 2^{n-1}$

- $\sum_{k=0}^n (-1)^k \cdot C_n^k = 0$ 。

- 二项式反演:  $\begin{cases} f_n = \sum_{i=0}^n \binom{n}{i} g_i \Leftrightarrow g_n = \sum_{i=0}^n (-1)^{n-i} \binom{n}{i} f_i \\ f_k = \sum_{i=k}^n \binom{i}{k} g_i \Leftrightarrow g_k = \sum_{i=k}^n (-1)^{i-k} \binom{i}{k} f_i \end{cases}$ ;

- $\sum_{i=1}^n i \binom{n}{i} = n \cdot 2^{n-1}$ ;

- $\sum_{i=1}^n i^2 \binom{n}{i} = n \cdot (n+1) \cdot 2^{n-2}$ ;

- $\sum_{i=1}^n \frac{1}{i} \binom{n}{i} = \sum_{i=1}^n \frac{1}{i}$ ;

- $\sum_{i=0}^n \binom{n}{i}^2 = \binom{2n}{n}$ ;

- 上指标求和:

$$-\sum_{k=0}^m \binom{n+k}{k} = \binom{n+m+1}{m};$$

$$- \sum_{k=0}^n \binom{k}{m} = \binom{n+1}{m+1}。$$

- 拉格朗日恒等式:  $\sum_{i=1}^n \sum_{j=i+1}^n (a_i b_j - a_j b_i)^2 = (\sum_{i=1}^n a_i)^2 (\sum_{i=1}^n b_i)^2 - (\sum_{i=1}^n a_i b_i)^2。$
- 范德蒙德卷积公式: 在数量为  $n+m$  的堆中选  $k$  个元素, 和分别在数量为  $n$  的堆中选  $i$  个元素, 在数量为  $m$  的堆中选  $k-i$  个元素的方案数是相同的, 即  $\sum_{i=0}^k \binom{n}{i} \binom{m}{k-i} = \binom{n+m}{k}$ ; 变体:
  - $\sum_{i=0}^k C_{i+n}^i = C_{k+n+1}^k;$
  - $\sum_{i=0}^k C_n^i \cdot C_m^i = \sum_{i=0}^k C_n^i \cdot C_{m-i}^{m-i} = C_{n+m}^n。$
- 二项式定理:  $(1+x)^n = \sum_{k=0}^n \binom{n}{k} x^k$
- 朱世杰恒等式:  $\sum_{k=0}^m \binom{n+k}{k} = \binom{n+m+1}{m}$

### 3.11 多项式与生成函数结论

- 求  $B_i = \sum_{k=i}^n C_k^i A_k$ , 即  $B_i = \frac{1}{i!} \sum_{k=i}^n \frac{1}{(k-i)!} \cdot k! A_k$ , 反转后卷积。
- 遇到  $\sum_{i=0}^n [i \% k = 0] f(i)$  可以转换为  $\sum_{i=0}^n \frac{1}{k} \sum_{j=0}^{k-1} (\omega_k^i)^j f(i)$  (单位根卷积)
- 广义二项式定理  $(1+x)^\alpha = \sum_{i=0}^{\infty} \binom{\alpha}{i} x^i。$
- 普通生成函数:  $A(x) = a_0 + a_1 x + a_2 x^2 + \dots = \langle a_0, a_1, a_2, \dots \rangle;$
- $1 + x^k + x^{2k} + \dots = \frac{1}{1-x^k};$
- $x + \frac{x^2}{2} + \frac{x^3}{3} + \dots = -\ln(1-x);$

- $1 + x + x^2 + \dots + x^{m-1} = \frac{1-x^m}{1-x};$
- $1 + 2x + 3x^2 + \dots = \frac{1}{(1-x)^2}$  (借用导数,  $nx^{n-1} = (x^n)'$ );
- $C_m^0 + C_m^1 x + C_m^2 x^2 + \dots + C_m^m x^m = (1+x)^m$  (二项式定理);
- $C_m^0 + C_{m+1}^1 x + C_{m+2}^2 x^2 + \dots = \frac{1}{(1-x)^{m+1}}$  (归纳法证明);
- $\sum_{n=0}^{\infty} F_n x^n = \frac{(F_1 - F_0)x + F_0}{1 - x - x^2}$  ( $F$  为斐波那契数列, 列方程  $G(x) = xG(x) + x^2 G(x) + (F_1 - F_0)x + F_0$ );
- $\sum_{n=0}^{\infty} H_n x^n = \frac{1 - \sqrt{1-4x}}{2x}$  ( $H$  为卡特兰数);
- 前缀和  $\sum_{n=0}^{\infty} s_n x^n = \frac{1}{1-x} f(x);$
- 五边形数定理:  $\prod_{i=1}^{\infty} (1 - x^i) = \sum_{k=0}^{\infty} (-1)^k x^{\frac{1}{2}k(3k+1)}。$
- 指数生成函数:  $A(x) = a_0 + a_1 x + a_2 \frac{x^2}{2!} + a_3 \frac{x^3}{3!} + \dots = \langle a_0, a_1, a_2, a_3, \dots \rangle;$
- 普通生成函数转换为指数生成函数: 系数乘以  $n!;$
- $1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots = \exp x;$
- 长度为  $n$  的循环置换数为  $P(x) = -\ln(1-x)$ , 长度为  $n$  的置换数为  $\exp P(x) = \frac{1}{1-x}$  (注意是指数生成函数)
- $n$  个点的生成树个数是  $P(x) = \sum_{n=1}^{\infty} n^{n-2} \frac{x^n}{n!}$ ,  $n$  个点的生成森林个数是  $\exp P(x);$
- $n$  个点的无向连通图个数是  $P(x)$ ,  $n$  个点的无向图个数是  $\exp P(x) = \sum_{n=0}^{\infty} 2^{\frac{1}{2}n(n-1)} \frac{x^n}{n!};$
- 长度为  $n(n \geq 2)$  的循环置换数是  $P(x) = -\ln(1-x) - x$ , 长度为  $n$  的错排数是  $\exp P(x)。$



## 4 博弈论

### 4.1 巴什博弈

- 有  $N$  个石子, 两名玩家轮流行动, 按以下规则取石子, 每人每次可以取走  $X(1 \leq X \leq M)$  个石子, 拿到最后一颗石子的一方获胜.
- 两名玩家轮流报数, 第一个报数的人可以报  $X(1 \leq X \leq M)$ , 后报数的人需要比前者所报数大  $Y(1 \leq Y \leq M)$ , 率先报到  $N$  的人获胜.

结论:

- $N = K \cdot (M + 1)$  (其中  $K \in \mathbb{N}^+$ ), 后手必胜 (后手可以控制每一回合结束时双方恰好取走  $M + 1$  个, 重复  $K$  轮后即胜利);
- $N = K \cdot (M + 1) + R$  (其中  $K \in \mathbb{N}^+, 0 < R < M + 1$ ), 先手必胜 (先手先取走  $R$  个, 之后控制每一回合结束时双方恰好取走  $M + 1$  个, 重复  $K$  轮后即胜利).

### 4.2 扩展巴什博弈

- 有  $N$  颗石子, 两名玩家轮流行动, 按以下规则取石子: 每人每次可以取走  $X(a \leq X \leq b)$  个石子, 如果最后剩余物品的数量小于  $a$  个, 则不能再取, 拿到最后一颗石子的一方获胜.

结论:

- $N = K \cdot (a + b)$  时, 后手必胜;
- $N = K \cdot (a + b) + R_1$  (其中  $K \in \mathbb{N}^+, 0 < R_1 < a$ ) 时, 后手必胜 (这些数量不够再取一次, 先手无法逆转局面);
- $N = K \cdot (a + b) + R_2$  (其中  $K \in \mathbb{N}^+, a \leq R_2 \leq b$ ) 时, 先手必胜;
- $N = K \cdot (a + b) + R_3$  (其中  $K \in \mathbb{N}^+, b < R_3 < a + b$ ) 时, 先手必胜 (这些数量不够再取一次, 后手无法逆转局面);

### 4.3 Nim 博弈

- 有  $N$  堆石子, 给出每一堆的石子数量, 两名玩家轮流行动, 按以下规则取石子: 每人每次任选一堆, 取走正整数颗石子, 拿到最后一颗石子的一方获胜 (注: 几个特点是不能跨堆、不能不拿).

结论:

- 记初始时各堆石子的数量  $(A_1, A_2, \dots, A_n)$ , 定义尼姆和  $Sum_N = A_1 \oplus A_2 \oplus \dots \oplus A_n$ .
- 当  $Sum_N = 0$  时先手必败, 反之先手必胜.

Nim 游戏具体取法:

- 先计算出尼姆和, 再对每一堆石子计算  $A_i \oplus Sum_N$ , 记为  $X_i$ . 若得到的值  $X_i < A_i$ ,  $X_i$  即为一个可行解, 即剩下  $X_i$  颗石头, 取走  $A_i - X_i$  颗石头 (这里取小于号是因为至少要取走 1 颗石子).

### 4.4 Moore' s Nim 游戏 (Nim-K 游戏)

- 有  $N$  堆石子, 给出每一堆的石子数量, 两名玩家轮流行动, 按以下规则取石子:
- 规定每人每次任选不超过  $K$  堆, 对每堆都取走不同的正整数颗石子, 拿到最后一颗石子的一方获胜.

结论:

- 把每一堆石子的石子数用二进制表示, 定义  $One_i$  为二进制第  $i$  位上 1 的个数.
- 对于每一位,  $One_1, One_2, \dots, One_N$  均不为  $K + 1$  的倍数.

### 4.5 Anti-Nim 游戏 (反 Nim 游戏)

- 有  $N$  堆石子, 给出每一堆的石子数量, 两名玩家轮流行动, 按以下规则取石子:
- 每人每次任选一堆, 取走正整数颗石子, 拿到最后一颗石子的一方出局.

结论:

- 所有堆的石头数量均不超过 1, 且  $Sum_N = 0$  (也可看作“且有偶数堆”);
- 至少有一堆的石头数量大于 1, 且  $Sum_N \neq 0$ .

### 4.6 阶梯 - Nim 博弈

- 有  $N$  级台阶, 每一级台阶上均有一定数量的石子, 给出每一级石子的数量, 两名玩家轮流行动, 按以下规则操作石子:
- 每人每次任选一级台阶, 拿走正整数颗石子放到下一级台阶中, 已经拿到地面上的石子不能再拿, 拿到最后一颗石子的一方获胜.

结论:

- 对奇数台阶做传统 Nim 博弈, 当  $Sum_N = 0$  时先手必败, 反之先手必胜.

## 4.7 SG 游戏 (有向图游戏)

我们使用以下几条规则来定义暴力求解的过程:

- 使用数字来表示输赢情况, 0 代表局面必败, 非 0 代表存在必胜可能, 我们称这个数字为这个局面的 SG 值;
- 找到最终态, 根据题意人为定义最终态的输赢情况;
- 对于非最终态的某个节点, 其 SG 值为所有子节点的 SG 值取  $\text{mex}$ ;
- 单个游戏的输赢态即对应根节点的 SG 值是否为 0, 为 0 代表先手必败, 非 0 代表先手必胜;
- 多个游戏的总 SG 值为单个游戏 SG 值的异或和.

使用哈希表, 以  $\mathcal{O}(N + M)$  的复杂度计算.

```

1 int n, m, a[N], num[N];
2 int sg(int x){
3     if (num[x] != -1) return num[x];
4     unordered_set<int> S;
5     for (int i = 1; i <= m; ++ i)
6         if (x >= a[i]) S.insert(sg(x - a[i]));
7     for (int i = 0; ; ++ i)
8         if (S.count(i) == 0) return num[x] = i;
9 }
10 void Solve(){
11     cin >> m;
12     for(int i = 1; i <= m; ++ i) cin >> a[i];
13     cin >> n;
14     int ans = 0; memset(num, -1, sizeof num);
15     for(int i = 1; i <= n; ++ i){
16         int x; cin >> x;
17         ans ^= sg(x);
18     }
19     if (ans == 0) no;
20     else yes;
21 }

```

## 4.8 Anti-SG 游戏 (反 SG 游戏)

SG 游戏中最先不能行动的一方获胜. 以下局面先手必胜:

- 单局游戏的 SG 值均不超过 1, 且总 SG 值为 0;
- 至少有一局单局游戏的 SG 值大于 1, 且总 SG 值不为 0.

在本质上, 这与 Anti-Nim 游戏的结论一致.

## 4.9 Lasker' s-Nim 游戏 ( Multi-SG 游戏)

- 有  $N$  堆石子, 给出每一堆的石子数量, 两名玩家轮流行动, 每人每次任选以下规定的一种操作石子:
  - 任选一堆, 取走正整数颗石子;
  - 任选数量大于 2 的一堆, 分成两堆非空石子.
- 拿到最后一颗石子的一方获胜.

本题使用 SG 函数求解, SG 值定义为:

$$SG(x) = \begin{cases} x - 1 & , x \bmod 4 = 0 \\ x & , x \bmod 4 = 1 \\ x & , x \bmod 4 = 2 \\ x + 1 & , x \bmod 4 = 3 \end{cases}$$

## 4.10 Every-SG 游戏

- 给出一个有向无环图, 其中  $K$  个顶点上放置了石子, 两名玩家轮流行动, 按以下规则操作石子:
- 移动图上所有还能够移动的石子;
- 无法移动石子的一方出局.

结论:

- 定义  $step$  为某一局游戏至多需要经过的回合数.
- 以下局面先手必胜:  $step$  为奇数.

## 4.11 威佐夫博弈

- 有两堆石子, 给出每一堆的石子数量, 两名玩家轮流行动, 每人每次任选以下规定的一种操作石子:
  - 任选一堆, 取走正整数颗石子;
  - 从两队中同时取走正整数颗石子.
- 拿到最后一颗石子的一方获胜.

结论:

- $(1, 2), (3, 5), (4, 7), (6, 10), \dots$  具体而言, 每一对的第一个数为此前没出现过的最小整数, 第二个数为第一个数加上  $1, 2, 3, 4, \dots$  .
- 更一般地, 对于第  $k$  对数, 第一个数为  $First_k = \left\lfloor \frac{k*(1+\sqrt{5})}{2} \right\rfloor$ , 第二个数为  $Second_k = First_k + k$  .

```

1 const double lorry = (sqrt(5.0) + 1.0) / 2.0;
2 //const double lorry = 1.618033988749894848204586834;
3 void Solve() {
4     int n, m; cin >> n >> m;
5     if (n < m) swap(n, m);
6     double x = n - m;
7     if ((int)(lorry * x) == m) cout << "lose\n";
8     else cout << "win\n";
9 }

```

#### 4.12 斐波那契博弈

- 有一堆石子, 数量为  $N$ , 两名玩家轮流行动, 按以下规则取石子:
- 先手第 1 次可以取任意多颗, 但不能全部取完, 此后每人取的石子数不能超过上个人的两倍, 拿到最后一颗石子的一方获胜.

结论:

- 当且仅当  $N$  为斐波那契数时先手必败.

```

1 int fib[100] = {1, 2};
2 map<int, bool> mp;
3 void Force() {
4     for (int i = 2; i <= 86; ++i) fib[i] = fib[i - 1] + fib[i - 2];
5     for (int i = 0; i <= 86; ++i) mp[fib[i]] = 1;
6 }
7 void Solve() {
8     int n; cin >> n;
9     if (mp[n] == 1) cout << "lose\n";
10    else cout << "win\n";
11 }

```

#### 4.13 树上删边游戏

- 给出一棵  $N$  个节点的有根树, 两名玩家轮流行动, 按以下规则操作:
- 选择任意一棵子树并删除 (即删去任意一条边, 不与根相连的部分会同步被删去);
- 删掉最后一棵子树的一方获胜 (换句话说, 删去根节点的一方失败).

结论:

- 当根节点 SG 值非 1 时先手必胜.

相较于传统 SG 值的定义, 本题的 SG 函数值定义为:

- 叶子节点的 SG 值为 0 .
- 非叶子节点的 SG 值为其所有孩子节点 SG 值 +1 的异或和.

```

1 auto dfs = [&](auto self, int x, int fa) -> int {
2     int x = 0;
3     for (auto y : ver[x]) {
4         if (y == fa) continue;
5         x ^= self(self, y, x);
6     }
7     return x + 1;
8 };
9 cout << (dfs(dfs, 1, 0) == 1 ? "Bob\n" : "Alice\n");

```

#### 4.14 无向图删边游戏 (Fusion Principle 定理)

- 给出一张  $N$  个节点的无向联通图, 有一个点作为图的根, 两名玩家轮流行动, 按以下规则操作:
- 选择任意一条边删除, 不与根相连的部分会同步被删去;
- 删掉最后一条边的一方获胜.

结论:

- 对于奇环, 我们将其缩成一个新的点 + 一条新边;
- 对于偶环, 我们将其缩成一个新的点;
- 所有连接到原来环上的边全部与新点相连.

此时, 本模型转化为“树上删边游戏”.

## 5 计算几何

### 5.1 极角排序

```

1  #include<bits/stdc++.h>
2  using i64 = long long;
3  const double eps = 0, pi = acos(-1);
4  struct node{
5      i64 x,y,idx;
6      double dge;
7  };
8  bool cmp(node a,node b){
9      return a.dge - b.dge < eps;
10 }
11 int main(){
12     std::ios::sync_with_stdio(0),std::cin.tie(0),std::cout.tie(0);
13     i64 n;std::cin >> n;
14     std::vector<node> a(n + 1);
15     for(int i = 1;i <= n;i++){
16         i64 x,y;std::cin >> x >> y;
17         a[i] = {x,y,i,atan2(x,y)};
18         if(a[i].dge < 0) a[i].dge += 2 * pi;
19     }
20     sort(a.begin() + 1,a.end(),cmp);
21     double ans = a[1].dge - a[n].dge;
22     ans += 2 * pi;
23     i64 idx1 = a[1].idx,idx2 = a[n].idx;
24     for(int i = 1;i < n;i++){
25         double temp = a[i + 1].dge - a[i].dge;
26         if(temp - ans < eps){
27             ans = temp;
28             idx1 = a[i + 1].idx,idx2 = a[i].idx;
29         }
30     }
31     std::cout << idx1 << " " << idx2 << "\n";
32     return 0;
33 }

```

### 5.2 二维静态凸包 (Andrew 算法)

$O(n \log n)$

```

1  using Point = std::array<i64,2>;
2  using Vector = std::array<i64,2>;
3  Vector operator-(const Point& a,const Point& b){
4      return {a[0] - b[0],a[1] - b[1]};
5  }
6  i64 operator*(const Vector& a,const Vector& b){
7      return a[0] * b[1] - a[1] * b[0];
8  }
9  struct Convex{
10     //逆时针

```

```

11     std::vector<Point> points,convex;
12     void addPoint(i64 x,i64 y){
13         points.push_back({x,y});
14     }
15     void work(){
16         std::sort(points.begin(),points.end());
17         std::vector<int> stk;
18         stk.push_back(0);
19         for(int i = 1;i < points.size();i++){
20             int m = stk.size();
21             while(m >= 2 && (points[stk[m - 2]] - points[stk[m - 1]]) * (points[stk[m - 1]] - points[i]) <= 0){
22                 m--;
23                 stk.pop_back();
24             }
25             stk.push_back(i);
26         }
27         int temp = stk.size();
28         for(int i = points.size() - 1;i >= 0;i--){
29             int m = stk.size();
30             while(m > temp && (points[stk[m - 2]] - points[stk[m - 1]]) * (points[stk[m - 1]] - points[i]) <= 0){
31                 m--;
32                 stk.pop_back();
33             }
34             stk.push_back(i);
35         }
36         for(int i = 0;i < stk.size() - 1;i++){
37             if(i >= 1 && points[stk[i]] == points[stk[i - 1]]) continue;
38             convex.push_back(points[stk[i]]);
39         }
40     }
41 };

```

### 5.3 闵可夫斯基和

计算两个凸包合成的大凸包。

```

1  template<class T> vector<Point<T>> mincowski(vector<Point<T>> P1, vector<Point<T>> P2) {
2      int n = P1.size(), m = P2.size();
3      vector<Point<T>> V1(n), V2(m);
4      for (int i = 0; i < n; i++) {
5          V1[i] = P1[(i + 1) % n] - P1[i];
6      }
7      for (int i = 0; i < m; i++) {
8          V2[i] = P2[(i + 1) % m] - P2[i];
9      }
10     vector<Point<T>> ans = {P1.front() + P2.front()};
11     int t = 0, i = 0, j = 0;
12     while (i < n && j < m) {
13         Point<T> val = sign(cross(V1[i], V2[j])) > 0 ? V1[i++] : V2[j++];
14         ans.push_back(ans.back() + val);
15     }

```

```

16 while (i < n) ans.push_back(ans.back() + V1[i++]);
17 while (j < m) ans.push_back(ans.back() + V2[j++]);
18 return ans;
19 }

```

## 5.4 平面最近点对

$O(n \log n)$

```

1 struct CPOP{
2     using Point = std::array<i64,2>;
3     std::vector<Point> points;
4     i64 minDist = 9e18;
5     Point p1,p2;
6     i64 dist(Point p1,Point p2){
7         auto [x1,y1] = p1;
8         auto [x2,y2] = p2;
9         return (x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2);
10    }
11    void addPoint(i64 x,i64 y){
12        points.push_back({x,y});
13    }
14    //0 idx 左闭右开
15    void work(i64 l,i64 r){
16        if(r - l == 1) return;
17        int mid = (l + r) >> 1;
18        i64 midx = points[mid][0];
19        work(l,mid),work(mid,r);
20
21        std::inplace_merge(points.begin() + l,points.begin() + mid,points.begin() + r,
22            [&](Point a,Point b){return a[1] < b[1];});
23
24        std::vector<Point> temp;
25        for(int i = l;i < r;i++){
26            if((points[i][0] - midx) * (points[i][0] - midx) < minDist)
27                temp.push_back(points[i]);
28        }
29
30        for(int i = 0;i < temp.size();i++){
31            for(int j = i + 1;j < temp.size();j++){
32                if((temp[i][1] - temp[j][1]) * (temp[i][1] - temp[j][1]) >= minDist) break;
33                i64 d = dist(temp[i],temp[j]);
34                if(d < minDist){
35                    minDist = d;
36                    p1 = temp[i],p2 = temp[j];
37                }
38            }
39        }
40    }
41    void work(){
42        std::sort(points.begin(),points.end());
43        work(0,points.size());
44    }

```

```

45 };

```

## 5.5 扫描线

矩形面积并

```

1 struct ScanLine{
2     std::vector<i64> cover,weight,num;
3     std::vector<std::array<i64,4>> side;
4     std::map<i64,i64> mp;
5     i64 n;
6     ScanLine(){};
7     ScanLine(i64 N):n(N),cover(N << 3),weight(N << 3),num((N << 1) + 1){}
8     void push_up(i64 p,i64 l,i64 r){
9         if(cover[p]) weight[p] = num[r] - num[l];
10        else if(l + 1 == r) weight[p] = 0;
11        else weight[p] = weight[(p << 1) | 1] + weight[p << 1];
12    }
13    void update(i64 l,i64 r,i64 c,i64 s,i64 t,i64 p){
14        if(l <= s && t <= r){
15            cover[p] += c;
16            push_up(p,s,t);
17            return;
18        }
19        if(s + 1 == t) return ;
20        i64 m = s + ((t - s) >> 1);
21        if(l <= m) update(l,r,c,s,m,p << 1);
22        if(r > m) update(l,r,c,m,t,(p << 1) | 1);
23        push_up(p,s,t);
24    }
25    void addRectangle(i64 x1,i64 y1,i64 x2,i64 y2){
26        side.push_back({x1,y1,y2,1});
27        side.push_back({x2,y1,y2,-1});
28        mp[y1]++,mp[y2]++;
29    }
30    i64 run(){
31        i64 res = 0,idx = 0;
32        for(auto& i : mp){
33            i.second = ++idx;
34            num[idx] = i.first;
35        }
36        sort(side.begin(),side.end());
37        update(mp[side[0][1]],mp[side[0][2]],1,1,2 * n,1);
38        for(int i = 1;i < side.size();i++){
39            auto [x,y1,y2,tag] = side[i];
40            res += (x - side[i - 1][0]) * weight[1];
41            update(mp[y1],mp[y2],tag,1,2 * n,1);
42        }
43        return res;
44    }
45 };
46
47 int main(){

```

```

48 std::ios::sync_with_stdio(0),std::cin.tie(0),std::cout.tie(0);
49 i64 n;
50 std::cin >> n;
51 ScanLine sl(n);
52 for(int i = 1;i <= n;i++){
53     i64 x1,y1,x2,y2;
54     std::cin >> x1 >> y1 >> x2 >> y2;
55     sl.addRectangle(x1,y1,x2,y2);
56 }
57 std::cout << sl.run() << "\n";
58 return 0;
59 }

```

## 5.6 平面最远点对 (旋转卡壳)

$O(N)$ , 需要凸包

```

1 using Point = std::array<i64,2>;
2 using Vector = std::array<i64,2>;
3 Vector operator-(const Point& a,const Point& b){
4     return {a[0] - b[0],a[1] - b[1]};
5 }
6 i64 operator*(const Vector& a,const Vector& b){
7     return a[0] * b[1] - a[1] * b[0];
8 }
9 //Furthest Pair of Points  $O(N)$ 
10 i64 FPOP(std::vector<Point>& points){
11     i64 j = 0,ans = 0,m = points.size();
12     if(m == 1) return 0;
13     else if(m == 2) return dist(points[0],points[1]);
14     for(int i = 0;i < m;i++){ //需要枚举边而不是点
15         int las = i == 0 ? m - 1 : i - 1;
16         //通过叉乘算三角形面积来替代点到直线距离
17         while(std::abs((points[las] - points[j]) * (points[las] - points[i])) <=
18             std::abs((points[las] - points[(j + 1) % m]) * (points[las] - points[i]))) j = (j + 1)
19             % m;
20         ans = std::max({ans,dist(points[j],points[i]),dist(points[j],points[las])});
21     }
22     return ans;
23 }

```

## 5.7 最小矩形覆盖 (旋转卡壳)

$O(N)$ , 需要凸包

```

1 i64 MRC(std::vector<Point>& points){
2     i64 j = 1,l = 0,r = 0,ans = 1e18,m = points.size();
3     if(m <= 2) return 0;
4     for(int i = 0;i < m;i++){ //需要枚举边而不是点
5         int las = i == 0 ? m - 1 : i - 1;
6         //通过叉乘算三角形面积来替代点到直线距离
7         while(std::abs((points[las] - points[j]) * (points[las] - points[i])) <=

```

```

8         std::abs((points[las] - points[(j + 1) % m]) * (points[las] - points[i]))) j = (j + 1)
9         % m;
10        //通过点乘投影来判断长度
11        while(((points[i] - points[las]) ^ (points[r] - points[las])) <=
12            ((points[i] - points[las]) ^ (points[(r + 1) % m] - points[las]))) r = (r + 1) % m;
13        if(i == 0) l = r;
14        while(((points[las] - points[i]) ^ (points[l] - points[i])) <=
15            ((points[las] - points[i]) ^ (points[(l + 1) % m] - points[i]))) l = (l + 1) % m;
16        i64 t1 = std::abs((points[las] - points[j]) * (points[i] - points[j]));
17        i64 t2 = std::abs((points[r] - points[las]) ^ (points[i] - points[las])) +
18            std::abs((points[l] - points[i]) ^ (points[las] - points[i]));
19        i64 t3 = std::abs((points[las] - points[i]) ^ (points[las] - points[i]));
20        ans = std::min<i64>(ans,(__int128)t1 * (t2 - t3) / t3);
21    }
22    return ans;
23 }

```

## 5.8 最小圆覆盖

期望  $O(N)$ , 最坏  $O(h^3n)$

```

1 #include<bits/stdc++.h>
2 using i64 = long long;
3 std::mt19937 rng(std::chrono::steady_clock::now().time_since_epoch().count());
4 static constexpr double eps = 1e-9;
5 using Point = std::array<double,2>;
6
7 double dist(Point& a,Point& b){
8     return std::sqrt((a[0] - b[0]) * (a[0] - b[0]) + (a[1] - b[1]) * (a[1] - b[1]));
9 }
10
11 struct SEC{
12     std::vector<Point> points;
13     double r = 0;
14     Point o{};
15     void addPoint(double x,double y){
16         points.push_back({x,y});
17     }
18
19     void update(Point& x,Point& y){
20         o[0] = (x[0] + y[0]) / 2;
21         o[1] = (x[1] + y[1]) / 2;
22         r = dist(x,y) / 2;
23     }
24
25     Point geto(Point a,Point b,Point c){
26         double a1,a2,b1,b2,c1,c2;
27         Point res;
28         a1 = 2 * (b[0] - a[0]), b1 = 2 * (b[1] - a[1]),
29         c1 = b[0] * b[0] - a[0] * a[0] + b[1] * b[1] - a[1] * a[1];
30         a2 = 2 * (c[0] - a[0]), b2 = 2 * (c[1] - a[1]),
31         c2 = c[0] * c[0] - a[0] * a[0] + c[1] * c[1] - a[1] * a[1];

```

```

32     if(std::fabs(a1) <= eps){
33         res[1] = c1 / b1;
34         res[0] = (c2 - res[1] * b2) / a2;
35     }
36     else if(std::fabs(b1) <= eps){
37         res[0] = c1 / a1;
38         res[1] = (c2 - res[0] * a2) / b2;
39     }
40     else{
41         res[0] = (c2 * b1 - c1 * b2) / (a2 * b1 - a1 * b2);
42         res[1] = (c2 * a1 - c1 * a2) / (b2 * a1 - b1 * a2);
43     }
44     return res;
45 }
46
47 void work(){
48     int n = points.size();
49     for(int i = 0; i < n; i++) std::swap(points[rng() % n], points[rng() % n]);
50     o = points[0];
51     for(int i = 0; i < n; i++){
52         if(dist(o, points[i]) - r < eps) continue;
53         update(points[i], points[0]);
54         for(int j = 0; j < i; j++){
55             if(dist(o, points[j]) - r < eps) continue;
56             update(points[i], points[j]);
57             for(int k = 0; k < j; k++){
58                 if(dist(o, points[k]) - r < eps) continue;
59                 o = geto(points[i], points[j], points[k]);
60                 r = dist(o, points[i]);
61             }
62         }
63     }
64 }
65 };
66
67 int main(){
68     std::ios::sync_with_stdio(0), std::cin.tie(0), std::cout.tie(0);
69     int n;
70     std::cin >> n;
71     SEC circ;
72     for(int i = 1; i <= n; i++){
73         double x, y;
74         std::cin >> x >> y;
75         circ.addPoint(x, y);
76     }
77     circ.work();
78     std::cout << std::fixed << std::setprecision(10) << circ.r << "\n" << circ.o[0] << " " <<
        circ.o[1] << "\n";
79     return 0;
80 }

```

## 5.9 半平面交

计算多条直线左边平面部分的交集。

```

1  template<class T> vector<Point<T>> halfcut(vector<Line<T>> lines) {
2      sort(lines.begin(), lines.end(), [&](auto l1, auto l2) {
3          auto d1 = l1.b - l1.a;
4          auto d2 = l2.b - l2.a;
5          if (sign(d1) != sign(d2)) {
6              return sign(d1) == 1;
7          }
8          return cross(d1, d2) > 0;
9      });
10     deque<Line<T>> ls;
11     deque<Point<T>> ps;
12     for (auto l : lines) {
13         if (ls.empty()) {
14             ls.push_back(l);
15             continue;
16         }
17         while (!ps.empty() && !pointOnLineLeft(ps.back(), l)) {
18             ps.pop_back();
19             ls.pop_back();
20         }
21         while (!ps.empty() && !pointOnLineLeft(ps[0], l)) {
22             ps.pop_front();
23             ls.pop_front();
24         }
25         if (cross(l.b - l.a, ls.back().b - ls.back().a) == 0) {
26             if (dot(l.b - l.a, ls.back().b - ls.back().a) > 0) {
27                 if (!pointOnLineLeft(ls.back().a, l)) {
28                     assert(ls.size() == 1);
29                     ls[0] = l;
30                 }
31                 continue;
32             }
33             return {};
34         }
35         ps.push_back(lineIntersection(ls.back(), l));
36         ls.push_back(l);
37     }
38     while (!ps.empty() && !pointOnLineLeft(ps.back(), ls[0])) {
39         ps.pop_back();
40         ls.pop_back();
41     }
42     if (ls.size() <= 2) {
43         return {};
44     }
45     ps.push_back(lineIntersection(ls[0], ls.back()));
46     return vector(ps.begin(), ps.end());
47 }

```

## 6 字符串

### 6.1 字符串哈希

$\mathcal{O}(|S|)$

```

1  typedef std::pair<i64,i64> PII;
2  const int N = 1e6 + 10;
3
4  i64 pow1[N],pow2[N];
5  struct string_hash{
6      const int Base = 499,M1 = 998244853,M2 = 989244353;
7      std::string str;
8      std::vector<i64> hash1,hash2;
9      void extend(char c,int idx){
10         pow1[idx] = (pow1[idx - 1] * Base) % M1;
11         hash1[idx] = (hash1[idx - 1] * Base + c) % M1;
12         pow2[idx] = (pow2[idx - 1] * Base) % M2;
13         hash2[idx] = (hash2[idx - 1] * Base + c) % M2;
14     }
15
16     void init(int n){
17         hash1.resize(n + 1,0);
18         hash2.resize(n + 1,0);
19         for(int i = 0;i <= n + 2;i++){
20             pow1[i] = pow2[i] = 1;
21         }
22
23     void init(std::string str){
24         i64 n = str.size();
25         init(n);
26         this->str = str;
27         for(int i = 0;i < n;i++){
28             extend(str[i],i + 1);
29         }
30     }
31     string_hash(){}
32     string_hash(std::string str){init(str);}
33
34     PII getHash(i64 l,i64 r){ // 1_idx
35         i64 t1 = (hash1[r] - 1LL * hash1[l - 1] * pow1[r - l + 1]) % M1;
36         t1 = (t1 + M1) % M1;
37         i64 t2 = (hash2[r] - 1LL * hash2[l - 1] * pow2[r - l + 1]) % M2;
38         t2 = (t2 + M2) % M2;
39         return {t1,t2};
40     }
41
42     bool same(i64 l1,i64 r1,i64 l2,i64 r2){
43         PII ta = getHash(l1,r1);
44         PII tb = getHash(l2,r2);
45         return ta.first == tb.first && ta.second == tb.second;
46     }
47 };

```

### 6.2 最小表示法

最小表示法是一种用于解决字符串最小表示问题的方法。它的核心思想是找到一个字符串的所有循环同构字符串中字典序最小的那个字符串, $\mathcal{O}(|S|)$ 。

```

1  template<typename T>
2  int get_min_str(const T& str){ //返回循环同构字符串的最小字典序首位下标
3      int i = 0,j = 1,k = 0;
4      int n = str.size();
5      while(i < n && j < n && k < n){
6          int tmp = str[(i + k) % n] - str[(j + k) % n];
7          if(tmp == 0){
8              k++;
9          }
10         else{
11             if(tmp > 0) i += k + 1;
12             else j += k + 1;
13             if(i == j) j++;
14             k = 0;
15         }
16     }
17     return std::min(i,j);
18 }

```

### 6.3 KMP

在字符串中查找子串, 以最坏  $\mathcal{O}(N + M)$  的时间计算  $t$  在  $s$  中出现的全部位置。

最小周期: 字符串长度-整个字符串的 *border*;

最小循环节: 区别于周期, 当字符串长度  $n \bmod (n - \text{next}[n]) = 0$  时, 等于最小周期, 否则为  $n$ 。

```

1  i64 ne[N],m,n;
2  std::string a,b;
3  std::vector<i64> ans;
4  void getNext(std::string& str){
5      ne[0] = 0;
6      for(int i = 1,j = 0;i < str.size();i++){
7          while(j && str[i] != str[j]) j = ne[j - 1];
8          if(str[i] == str[j]) j++;
9          ne[i] = j;
10     }
11 }
12
13 void kmp(std::string& a,std::string& b){
14     getNext(b);
15     for(int i = 0,j = 0;i < a.size();i++){
16         while(j && a[i] != b[j]) j = ne[j - 1];
17         if(a[i] == b[j]) j++;
18         if(j == b.size()) ans.push_back(i - b.size() + 2);

```



```

19 }
20 }

```

## 6.4 Z 函数 (扩展 KMP)

获取字符串  $s$  和  $s[i, n-1]$  (即以  $s[i]$  开头的后缀) 的最长公共前缀 (LCP) 的长度, 总复杂度  $\mathcal{O}(N)$ 。

```

1 template<typename T>
2 std::vector<int> z_function(const T& str){
3     int n = str.size();
4     std::vector<int> z(n);
5     int j = 0;
6     for(int i = 1; i < n; i++){
7         z[i] = (j + z[j] <= i) ? 0 : std::min(j + z[j] - i, z[i - j]);
8         while(i + z[i] < n && str[z[i]] == str[i + z[i]]){
9             z[i]++;
10        }
11        if(j + z[j] < i + z[i]){
12            j = i;
13        }
14    }
15    z[0] = n;
16    return z;
17 }

```

## 6.5 Manacher

$\mathcal{O}(|S|)$

```

1 struct Manacher{
2     std::vector<int> p;
3     std::string change(std::string& str){
4         std::string temp = "!#";
5         for(int i = 0; i < str.size(); i++){
6             temp += str[i];
7             temp += '#';
8         }
9         return temp;
10    }
11
12    void get_p(std::string& str){
13        int maxright = 0, mid = 0;
14        str = change(str);
15        p.assign(str.size(), 0);
16        for(int i = 1; i < str.size(); i++){
17            if(i < maxright){
18                p[i] = std::min(p[2 * mid - i], maxright - i + 1);
19            }
20            else{
21                p[i] = 1 + (str[i] != '#');
22            }

```

```

23         while(i - p[i] >= 0 && p[i] + i < str.size() && str[i - p[i]] == str[i + p[i]]){
24             p[i]++;
25         }
26         if(i + p[i] - 1 > maxright){
27             mid = i;
28             maxright = i + p[i] - 1;
29         }
30     }
31 }
32 // 返回最大回文串长度
33 int get_max(std::string& str){
34     get_p(str);
35     int res = 0;
36     for(int i = 1; i < p.size(); i++){
37         res = std::max(res, p[i] - 1);
38     }
39     return res;
40 }
41 };

```

## 6.6 字典树

$\mathcal{O}(N \log N)$

```

1 template<int N = 26>
2 struct Trie{
3     std::vector<std::array<int, N>> son;
4     std::vector<int> cnt;
5
6     Trie(){
7         son.push_back(std::array<int, N>{});
8         cnt.push_back(0);
9     }
10
11    void insert(std::string& str){ //插入
12        int p = 0, n = str.size();
13        for(int i = 0; i < n; i++){
14            int ch = str[i] - '0';
15            if(!son[p][ch]){ //如果此处字母不存在就创建一个
16                son[p][ch] = son.size();
17                son.push_back(std::array<int, N>{});
18                cnt.push_back(0);
19            }
20            p = son[p][ch]; //指向下一个位置
21            // cnt[p]++;
22        }
23        cnt[p]++; //该处单词数+1
24    }
25
26    int find(std::string& str){ //找
27        int p = 0, n = str.size();
28        for(int i = 0; i < n; i++){
29            int ch = str[i] - '0';

```

```

30         if(!son[p][ch]){
31             return 0;
32         }
33         p = son[p][ch];
34     }
35     return cnt[p];
36 }
37 };

```

## 6.7 AC 自动机

$\mathcal{O}(\sum |s_i| + |S|)$

```

1 struct ACAM{
2     vector<int> ne,in,cnt,f;
3     vector<vector<int>> son;
4     int idx = 0;
5     ACAM(){
6         son.resize(N,vector<int>(26,0));
7         ne.resize(N,0),in.resize(N,0);
8         cnt.resize(N,0),f.resize(N,0);
9     }
10    ll insert(string str){ //插入模式串
11        int p = 0;
12        for(int i = 0;str[i];i++){
13            int a = str[i] - 'a';
14            if(!son[p][a]) son[p][a] = ++idx; //如果此处字母不存在就创建一个
15            p = son[p][a]; //指向下一个位置
16        }
17        cnt[p]++; //该处单词数+1
18        return p;
19    }
20
21    void build(){
22        queue<int> q;
23        for(int i = 0;i < 26;i++){
24            if(son[0][i]) q.push(son[0][i]);
25        }
26        while(!q.empty()){
27            int u = q.front();
28            q.pop();
29            for(int i = 0;i < 26;i++){
30                int v = son[u][i];
31                if(v){
32                    ne[v] = son[ne[u]][i]; //建立回跳边
33                    q.push(v);
34                    in[son[ne[u]][i]]++;
35                }
36                else son[u][i] = son[ne[u]][i]; //建转移边
37            }
38        }
39    }
40    void query(string str,int n){
41        for(int k = 0,i = 0;k < str.size();k++){

```

```

41            i = son[i][str[k] - 'a'];
42            f[i]++;
43            // for(int j = i;j && ~cnt[j];j = ne[j]){
44            //     ans += cnt[j];
45            //     cnt[j] = -1; //不重复计数
46            // }
47        }
48        queue<ll> q;
49        for(int i = 1;i <= idx;i++){
50            if(in[i] == 0) q.push(i);
51        }
52        while(!q.empty()){
53            ll u = q.front(),v = ne[u];
54            q.pop();
55            f[v] += f[u],in[v]--;
56            if(v != 0 && in[v] == 0) q.push(v);
57            f[u] *= cnt[u];
58        }
59    };

```

## 6.8 后缀数组

把字符串的所有后缀按字典序排序后得到的  $rk$  数组, 以  $\mathcal{O}(N \log N)$  的复杂度求解。

```

1 struct SA{
2     std::string str;
3     int n,m,tp = 128,p = 0;
4     std::vector<int> rk,sa;
5     SA(std::string _s){
6         str = " " + _s;
7         n = _s.size();
8         m = (n << 1) + 1;
9         rk.assign(m,0),sa.assign(m,0);
10        work();
11    };
12    void work(){
13        if(n == 1){
14            sa[1] = rk[1] = 1;
15            return;
16        }
17        std::vector<int> cnt(std::max(tp,n) + 1,id(n + 1));
18        for(int i = 1;i <= n;i++) cnt[rk[i] = str[i]]++;
19        for(int i = 1;i <= tp;i++) cnt[i] += cnt[i - 1];
20        for(int i = n;i >= 1;i--) sa[cnt[rk[i]]--] = i;
21
22        for(int w = 1;w < n;w <= 1,tp = p){
23            int cur = 0;
24            for(int i = n - w + 1;i <= n;i++) id[++cur] = i;
25            for(int i = 1;i <= n;i++)
26                if(sa[i] > w) id[++cur] = sa[i] - w;
27            std::fill(cnt.begin(),cnt.end(),0);
28            for(int i = 1;i <= n;i++) cnt[rk[i]]++;
29            for(int i = 1;i <= tp;i++) cnt[i] += cnt[i - 1];

```

```

30     for(int i = n; i >= 1; i--) sa[cnt[rk[id[i]]]--] = id[i];
31     std::vector<int> temp(m);
32     p = 0;
33     for(int i = 1; i <= n; i++){
34         if(rk[sa[i]] != rk[sa[i - 1]] || rk[sa[i] + w] != rk[sa[i - 1] + w]) p++;
35         temp[sa[i]] = p;
36     }
37     rk = temp;
38     if(p == n) break;
39 }
40 }
41 };

```

## 6.9 回文自动机 PAM(回文树)

本质不同的回文串个数:  $idx - 2$ ; 回文子串出现次数。

对于一个字符串  $s$ , 它的本质不同回文子串个数最多只有  $|s|$  个, 那么, 构造  $s$  的回文树的时间复杂度是  $\mathcal{O}(|s|)$ 。

```

1 struct PalindromeAutomaton {
2     constexpr static int N = 5e5 + 10;
3     int tr[N][26], fail[N], len[N];
4     int cntNodes, last;
5     int cnt[N];
6     string s;
7     PalindromeAutomaton(string s) {
8         memset(tr, 0, sizeof tr);
9         memset(fail, 0, sizeof fail);
10        len[0] = 0, fail[0] = 1;
11        len[1] = -1, fail[1] = 0;
12        cntNodes = 1;
13        last = 0;
14        this->s = s;
15    }
16    void insert(char c, int i) {
17        int u = get_fail(last, i);
18        if (!tr[u][c - 'a']) {
19            int v = ++cntNodes;
20            fail[v] = tr[get_fail(fail[u], i)][c - 'a'];
21            tr[u][c - 'a'] = v;
22            len[v] = len[u] + 2;
23            cnt[v] = cnt[fail[v]] + 1;
24        }
25        last = tr[u][c - 'a'];
26    }
27    int get_fail(int u, int i) {
28        while (i - len[u] - 1 <= -1 || s[i - len[u] - 1] != s[i]) {
29            u = fail[u];
30        }
31        return u;
32    }
33 };

```

## 6.10 后缀自动机 (SAM)

字符串  $S$  的 SAM 是一个接受  $S$  的所有后缀的最小 DFA。

对于一个长度为  $N$  的字符串, 它的空间复杂度仅为  $\mathcal{O}(N)$ 。而且, 构造 SAM 的时间复杂度也仅为  $\mathcal{O}(N)$ 。准确地说, 一个 SAM 最多有  $2N - 1$  个结点和  $3N - 4$  条转移边。最终得到的是一个 DAG。

所有状态和所有后缀链接构成根为  $t_0$  的根向树, 这个树也称为后缀链接树。

后缀链接树有如下性质:

- 祖先节点对应的字符串总是子孙节点对应的字符串的后缀。
- 每个节点处的 `endpos` 集合就是它的子树内的所有「前缀节点」 $v_i$  的下标  $i$  的集合。
- 后缀链接树的祖先节点的 `endpos` 集合总是严格包含子孙节点的 `endpos` 集合。
- 每个节点处的 `len` 的值就是它的子树内的所有「前缀节点」 $v_i$  对应前缀的最长公共后缀的长度。
- 除根节点  $t_0$  外, 每个节点对应的不同子串的数目, 就是它的 `len` 值, 减去它的父节点的 `len` 值, 即  $\text{len}(v) - \text{len}(\text{link}(v))$ 。

```

1 template<int N = 26>
2 struct SAM{
3     struct State{
4         int fa; // 后缀链接 (suffix link)
5         int len; // 该状态代表的最长子串长度
6         std::array<int, N> next; // 转移边
7         State(): fa(0), len(0) {next.fill(0);}
8     };
9
10    std::vector<State> st;
11    std::vector<int> occ; // 出现次数
12    int last = 1;
13
14    SAM(){
15        init();
16    }
17    SAM(std::string& str){
18        init();
19        for(auto ch : str){
20            extend(ch);
21        }
22    }

```

```

23
24 void init(){
25     st.resize(2);
26     occ.assign(2,0);
27     st[0].len = -1;
28     st[1].len = st[1].fa = 0;
29 }
30
31 void extend(char ch){
32     int c = ch - 'a';
33     int cur = st.size(), p = last;
34     st.push_back(State());
35     occ.push_back(1);
36     st[cur].len = st[last].len + 1;
37
38     while(p && !st[p].next[c]){
39         st[p].next[c] = cur;
40         p = st[p].fa;
41     }
42
43     if(!p){
44         st[cur].fa = 1;
45     }
46     else{
47         int q = st[p].next[c];
48         if(st[p].len + 1 == st[q].len){
49             st[cur].fa = q;
50         }
51         else{
52             int clone = st.size();
53             st.push_back(st[q]);
54             occ.push_back(0);
55             st[clone].len = st[p].len + 1;
56             while(p && st[p].next[c] == q){
57                 st[p].next[c] = clone;
58                 p = st[p].fa;
59             }
60             st[q].fa = st[cur].fa = clone;
61         }
62     }
63     last = cur;
64 }
65 };

```

### 6.10.1 后缀自动机应用

#### • 检查字符串是否出现

1. **问题：**给定文本串  $T$  和模式串  $P$ ，判断  $P$  是否为  $T$  的子串。
2. **解法：**构造  $T$  的 SAM。从初始状态  $t_0$  开始，按  $P$  的字符转移。若中途失配则  $P$  未出现；若能完整转移则出现。单次查询  $O(|P|)$ ，且可求得  $P$  在

$T$  中的最长匹配前缀。

#### • 计算不同子串个数

1. **问题：**给定字符串  $S$ ，求其不同子串的总数。
2. **解法：**构造  $S$  的 SAM。每个状态  $v$ （不含  $t_0$ ）贡献  $\text{len}(v) - \text{len}(\text{link}(v))$ ，累加所有状态即为答案。复杂度  $O(|S|)$ 。

#### • 所有不同子串的总长度

1. **问题：**给定字符串  $S$ ，求所有不同子串的长度之和。
2. **解法：**状态  $v$  贡献  $\frac{\text{len}(v)(\text{len}(v) + 1)}{2} - \frac{\text{len}(\text{link}(v))(\text{len}(\text{link}(v)) + 1)}{2}$ ，累加即得总长度。复杂度  $O(|S|)$ 。

#### • 字典序第 $k$ 大子串

1. **问题：**给定字符串  $S$ ，多组询问，每次查询  $S$  的所有子串中字典序第  $k$  大的子串。
2. **解法：**在 SAM 上统计从每个状态出发的路径总数，然后从根开始按字典序贪心查找。预处理  $O(|S|)$ ，单次查询  $O(|\text{ans}| \cdot |\Sigma|)$ 。

#### • 最小循环移位

1. **问题：**给定字符串  $S$ ，求其字典序最小的循环移位。
2. **解法：**构造  $S + S$  的 SAM，从根出发贪心走字典序最小的边，直到路径长度为  $|S|$ 。复杂度  $O(|S|)$ 。

#### • 出现次数（多模式串查询）

1. **问题：**给定文本串  $T$ ，多组询问，每次给模式串  $P$ ，求  $P$  在  $T$  中出现的次数。
2. **解法：**构造  $T$  的 SAM。在 SAM 上，每个非克隆状态初始  $occ = 1$ ，克隆状态  $occ = 0$ 。将状态按  $\text{len}$  降序排序，将  $occ$  累加到后缀链接指向的父节点。最终  $occ[v]$  即状态  $v$  代表的子串出现次数。查询时找到  $P$  对应状态，输出其  $occ$ 。预处理  $O(|T|)$ ，单次查询  $O(|P|)$ 。

#### • 第一次出现的位置

1. **问题：**给定文本串  $T$ ，多组查询，每次给模式串  $P$ ，求  $P$  在  $T$  中第一次出现的起始位置。

2. **解法：**构造  $T$  的 SAM。每个非克隆状态  $v_i$  (对应前缀  $T[1..i]$ ) 初始值为  $i$ ，克隆状态为  $\infty$ 。按  $\text{len}$  降序排序，将每个状态的初始值取  $\min$  传递给后缀链接父节点。最终  $\text{first}[v]$  为状态  $v$  代表子串的结束位置最小值。查询时找到  $P$  对应状态  $v$ ，答案  $\text{first}[v] - |P| + 1$ 。预处理  $O(|T|)$ ，单次查询  $O(|P|)$ 。

#### • 所有出现的位置

1. **问题：**给定文本串  $T$ ，多组查询，每次给模式串  $P$ ，输出  $P$  在  $T$  中的所有出现起始位置。
2. **解法：**找到  $P$  对应状态  $v$ ，在后缀链接树中遍历  $v$  的子树，输出所有非克隆状态对应的前缀位置 (减去  $|P| + 1$ )。单次查询复杂度  $O(|P| + \text{answer}(P))$ 。

#### • 最短的未出现字符串

1. **问题：**给定字符串  $S$  和字符集，求最短的未在  $S$  中出现的字符串。
2. **解法：**在  $S$  的 SAM 上 DP。设  $d_v$  为从状态  $v$  出发需添加的最小字符数。若  $v$  无任何转移，则  $d_v = 1$ ；否则  $d_v = 1 + \min_c d_{\text{next}(v,c)}$ 。答案  $d_{t_0}$ ，并可回溯构造。复杂度  $O(|S| \cdot |\Sigma|)$ 。

#### • 两个字符串的最长公共子串

1. **问题：**给定字符串  $S$  和  $T$ ，求它们的最长公共子串。
2. **解法：**构造  $S$  的 SAM。遍历  $T$ ，维护当前匹配状态  $v$  和长度  $l$ ：若存在  $c$  转移则转移并  $l + 1$ ；否则沿后缀链接回退，直到有转移或回到根。更新答案  $\text{ans} = \max(\text{ans}, l)$ 。复杂度  $O(|T|)$ 。

#### • 多个字符串的最长公共子串

1. **问题：**给定  $k$  个字符串  $S_1, \dots, S_k$ ，求它们的最长公共子串。
2. **解法：**构造新串  $T = S_1 \#_1 S_2 \#_2 \cdots S_k \#_k$ ，其中  $\#_i$  为互不相同的分隔符。构造  $T$  的 SAM，对每个状态记录它能通过哪些分隔符 (即它能到达哪些原始字符串)。通过 DP 或 DFS 计算每个状态的可达分隔符集合，若可达全部  $k$  个，则该状态对应的最长子串长度为  $\text{len}(v)$ 。取最大值。复杂度  $O(|T| + k \cdot |\Sigma|)$ 。

## 7 图论

### 7.1 Dijkstra 算法

```

1 std::vector<i64> dist(n + 1), vis(n + 1);
2 auto dijkstra = [&](int st) -> void{
3     std::fill(dist.begin() + 1, dist.end(), 1e10);
4     std::fill(vis.begin() + 1, vis.end(), 0);
5     std::priority_queue<std::array<i64, 2>, std::vector<std::array<i64, 2>>, std::greater<std::array<i64, 2>>> pq;
6     dist[st] = 0;
7     pq.push({0, st});
8     while(!pq.empty()){
9         auto [d, u] = pq.top();
10        pq.pop();
11        if(vis[u]) continue;
12        vis[u] = 1;
13        for(auto [v, w] : edge[u]){
14            if(dist[v] > d + w){
15                dist[v] = d + w;
16                pq.push({d + w, v});
17            }
18        }
19    }
20 };
21 dijkstra(s);

```

### 7.2 Floyd 算法

```

1 int n, m, k, x, y, z;
2 int dist[N][M];
3 //dist[i][j] 表示点i到点j的最短距离 ,k为中间点
4 void floyd(){
5     for(k = 1; k <= n; k++){ //枚举中间点
6         for(int i = 1; i <= n; i++){ //起始点
7             for(int j = 1; j <= n; j++){ //目的地
8                 dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
9             }
10        }
11    }
12 }

```

### 7.3 二分图最大匹配 (匈牙利算法)

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5 const int N = 5e2 + 10;
6
7 ll n, m, fri[N], vis[N];

```

```

8 vector<ll> e[N];
9
10 bool match(ll x, ll t){
11     if(vis[x] == t) return false; //已经访问过
12     vis[x] = t; //记录状态为访问过
13     for(auto i : e[x]){
14         if(fri[i] == 0 || match(fri[i], t)){ //如果暂无匹配, 或原来匹配的左侧元素可以找到新的匹配
15             fri[i] = x; //当前左侧元素成为当前右侧元素的新匹配
16             return true; //返回匹配成功
17         }
18     }
19     return false; //循环结束, 仍未找到匹配, 返回匹配失败
20 }
21
22 int main(){
23     ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
24     ll k, a, b, cnt = 0;
25     cin >> n >> m >> k;
26     while(k--){
27         cin >> a >> b;
28         e[a].push_back(b);
29     }
30     for(int i = 1; i <= n; i++){
31         if(match(i, i)) cnt++;
32     }
33     cout << cnt << "\n";
34     return 0;
35 }

```

### 7.4 差分约束

给出一组包含  $m$  个不等式, 有  $n$  个未知数的形如:

$$\begin{cases} u_1 - v_1 \leq w_1 \\ u_2 - v_2 \leq w_2 \\ \dots \\ u_m - v_m \leq w_m \end{cases}$$

的不等式组, 求任意一组满足这个不等式组的解。SPFA 解,  $O(nm)$ 。

```

1 #include<bits/stdc++.h>
2 #define all(x) (x).begin(), (x).end()
3 using namespace std;
4
5 typedef int ll;
6 typedef pair<ll, ll> PII;
7 const int N = 5e3 + 10, M = 5e3 + 10, mod = 1e9 + 7;
8
9 int head[N];
10 struct edge{

```

```

11     int next,to, w;
12 };
13 vector<edge> edges;
14
15 void add_edge(int u, int v, int w){
16     edges.push_back({head[u],v,w});
17     head[u] = edges.size() - 1;
18 }
19 bool inqueue[N];
20 int cnt[N], dis[N];
21 queue<int> Q;
22 bool SPFA(int s, int n){
23     memset(dis, 127, sizeof(dis));
24     dis[s] = 0;
25     Q.push(s);
26     while (!Q.empty()){
27         int p = Q.front();
28         if (cnt[p] > n)
29             return false;
30         Q.pop();
31         inqueue[p] = false;
32         for (int eg = head[p]; eg != -1; eg = edges[eg].next){
33             int to = edges[eg].to;
34             if (edges[eg].w + dis[p] < dis[to]){
35                 // if (edges[eg].w + dis[p] > dis[to])
36                 dis[to] = edges[eg].w + dis[p];
37                 if (!inqueue[to]){
38                     Q.push(to);
39                     inqueue[to] = true;
40                     cnt[to]++;
41                 }
42             }
43         }
44     }
45     return true;
46 }
47
48 int main(){
49     ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
50     ll n,m,x,y,w;
51     cin >> n >> m;
52     memset(head,-1,sizeof(head));
53     for (int i = 0; i < m; ++i){
54         cin >> x >> y >> w;
55         add_edge(y, x, w);
56     }
57     for (int i = 1; i <= n; ++i) add_edge(0, i, 0);
58     if (SPFA(0, n)){ //SPFA判断负环, 无解
59         for (int i = 1; i <= n; ++i)
60             cout << dis[i] << " ";
61     }
62     else cout << "NO\n";
63     return 0;

```

```

64 }

```

## 7.5 2-SAT

基于 tarjan 缩点, 时间复杂度为  $\mathcal{O}(N + M)$ 。

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 struct SCC{
6     // st: 是否在栈中 scc: 每个点所属的强连通分量编号 csc: 强连通分量的数量
7     vector<ll> dfn,low,stk,scc;
8     vector<bool> st;
9     vector<vector<ll>> e;
10    ll csc,N,cnt = 0;
11    SCC(int n){
12        dfn.resize(n + 1,0);
13        low.resize(n + 1,0);
14        st.resize(n + 1,false);
15        scc.resize(n + 1);
16        e.resize(n + 1);
17        csc = 0;
18        N = n;
19    }
20
21    void add(ll u,ll v){
22        e[u].push_back(v);
23    }
24
25    void tarjan(ll p){
26        low[p] = dfn[p] = ++cnt;
27        st[p] = true;
28        stk.push_back(p);
29        for(auto& q : e[p]){
30            if(!dfn[q]){
31                tarjan(q);
32                low[p] = min(low[q],low[p]);
33            }
34            else if(st[q]){
35                low[p] = min(low[p],dfn[q]);
36            }
37        }
38        if(low[p] == dfn[p]){
39            int top;
40            csc++;
41            do{
42                top = stk.back();
43                stk.pop_back();
44                st[top] = false;
45                scc[top] = csc;
46            }while(top != p);
47        }

```

```

48 }
49
50 void work(){
51     for(int i = 1;i <= N;i++){
52         if(!dfn[i])
53             tarjan(i);
54     }
55 };
56 //scc编号的顺序符合拓扑序 (编号越大的点拓扑序越靠前)
57
58 int main(){
59     ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
60     ll n,m;
61     cin >> n >> m;
62     SCC sc(2 * n + 1);
63     for(int i = 1;i <= m;i++){
64         ll a,ta,b,tb;
65         cin >> a >> ta >> b >> tb;
66         sc.add((2 * a + ta) ^ 1,2 * b + tb);
67         sc.add((2 * b + tb) ^ 1,2 * a + ta);
68     }
69     sc.work();
70     bool flag = true;
71     vector<ll> ans(n + 1);
72     for(int i = 1;i <= n;i++){
73         if(sc.scc[2 * i] == sc.scc[2 * i + 1]){
74             flag = false;
75             break;
76         }
77         else if(sc.scc[i * 2] > sc.scc[i * 2 + 1])
78             ans[i] = 1;
79         else ans[i] = 0;
80     }
81     if(flag){
82         cout << "POSSIBLE\n";
83         for(int i = 1;i <= n;i++){
84             cout << ans[i] << " ";
85         }
86     }
87     else cout << "IMPOSSIBLE\n";
88     return 0;
89 }

```

## 7.6 割边

```

1 //bri为割边
2 struct CutBridge{
3     vector<ll> dfn,low,fa;
4     vector<vector<ll>> edge;
5     int n,m,cnt = 0;
6     vector<PII> bri;
7

```

```

8     CutBridge(int n,int m){
9         edge.resize(n + 2),dfn.resize(n + 1,0),low.resize(n + 1,0);
10        fa.resize(n + 1,0);
11        this->n = n,this->m = m;
12    }
13
14    void add(int a,int b){
15        edge[b].push_back(a);
16        edge[a].push_back(b);
17    }
18
19    void tarjan(int p){
20        low[p] = dfn[p] = ++cnt;
21        for (auto to : edge[p]){
22            if (!dfn[to]){
23                fa[to] = p; // 记录父节点
24                tarjan(to);
25                low[p] = min(low[p], low[to]);
26                if (low[to] > dfn[p])
27                    bri.emplace_back(p, to);
28            }
29            else if (fa[p] != to) // 排除父节点
30                low[p] = min(low[p], dfn[to]);
31        }
32    }
33
34    void work(){
35        for(int i = 1;i <= n;i++){
36            if(!dfn[i])
37                tarjan(i);
38        }
39    };

```

## 7.7 割点

```

1 vector<ll> edge[N],cut;//cut 存储所有割点
2 ll dfn[N],low[N],cnt = 0;
3 void tarjan(ll p,bool root = true){
4     ll tot = 0;
5     cnt++;
6     dfn[p] = low[p] = cnt;
7     for(auto& i : edge[p]){
8         if(!dfn[i]){
9             tarjan(i,false);
10            low[p] = min(low[p],low[i]);
11            tot += (low[i] >= dfn[p]);
12        }
13        else low[p] = min(low[p],dfn[i]);
14    }
15    if(tot > root)
16        cut.push_back(p);
17 }

```



## 7.8 缩点

```

1 struct SCC{
2     // st: 是否在栈中 scc: 每个点所属的强连通分量编号 csc: 强连通分量的数量
3     vector<ll> dfn,low,stk,scc;
4     vector<bool> st;
5     vector<vector<ll>> e;
6     ll csc,N,cnt = 0;
7     SCC(int n){
8         dfn.resize(n + 1,0);
9         low.resize(n + 1,0);
10        st.resize(n + 1,false);
11        scc.resize(n + 1);
12        e.resize(n + 1);
13        csc = 0;
14        N = n;
15    }
16
17    void add(ll u,ll v){
18        e[u].push_back(v);
19    }
20
21    void tarjan(ll p){
22        low[p] = dfn[p] = ++cnt;
23        st[p] = true;
24        stk.push_back(p);
25        for(auto& q : e[p]){
26            if(!dfn[q]){
27                tarjan(q);
28                low[p] = min(low[q],low[p]);
29            }
30            else if(st[q]){
31                low[p] = min(low[p],dfn[q]);
32            }
33        }
34        if(low[p] == dfn[p]){
35            int top;
36            csc++;
37            do{
38                top = stk.back();
39                stk.pop_back();
40                st[top] = false;
41                scc[top] = csc;
42            }while(top != p);
43        }
44    }
45
46    void work(){
47        for(int i = 1;i <= N;i++){
48            if(!dfn[i])
49                tarjan(i);
50        }
51    };

```

```

52 //scc编号的顺序符合拓扑序 (编号越大的点拓扑序越靠前)
53
54 int main(){
55     ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
56     ll n,m,ans = 0;
57     cin >> n >> m;
58     SCC a(n);
59     vector<ll> w(n + 1),nw;
60     for(int i = 1;i <= n;i++) cin >> w[i];
61     for(int i = 1;i <= m;i++){
62         ll u,v;
63         cin >> u >> v;
64         a.add(u,v);
65     }
66     a.work();
67     nw.resize(a.csc + 1,0);
68     vector<vector<ll>> edge(a.csc + 1);
69     vector<ll> dp(a.csc + 1,0);
70     for(int i = 1;i <= n;i++){
71         nw[a.scc[i]] += w[i];
72         dp[a.scc[i]] += w[i];
73         ans = max(ans,dp[a.scc[i]]);
74         for(auto& j : a.e[i]){
75             if(a.scc[i] != a.scc[j]){
76                 edge[a.scc[i]].push_back(a.scc[j]);
77             }
78         }
79     }
80     for(int i = a.csc;i >= 1;i--){
81         for(auto& j : edge[i]){
82             dp[j] = max(dp[j],dp[i] + nw[j]);
83             ans = max(ans,dp[j]);
84         }
85     }
86     cout << ans << "\n";
87     return 0;
88 }

```

## 7.9 边双连通分量

```

1 struct EBCC{
2     // st: 是否在栈中 ebcc: 每个点所属的边双连通分量编号 cebcc: 边双连通分量的数量
3     std::vector<i64> dfn,low,stk,ebcc;
4     std::vector<bool> st;
5     std::vector<std::vector<std::array<i64,2>>> e;
6     i64 cebcc,N,cnt,edge_cnt;
7     EBCC(int n): dfn(n + 1),low(n + 1),st(n + 1),
8         ebcc(n + 1),e(n + 1),cebcc(0),N(n),
9         cnt(0),edge_cnt(1){}
10    void add(i64 u,i64 v){
11        e[u].push_back({v,edge_cnt << 1});
12        e[v].push_back({u,edge_cnt << 1 | 1});

```

```

13     edge_cnt++;
14 }
15 void tarjan(int p, int las){
16     low[p] = dfn[p] = ++cnt;
17     st[p] = true;
18     stk.push_back(p);
19     for(auto [q,idx]: e[p]){
20         if(idx == (las ^ 1)) continue;
21         if(!dfn[q]){
22             tarjan(q, idx);
23             low[p] = std::min(low[p], low[q]);
24         }else low[p] = std::min(low[p], dfn[q]);
25     }
26     if(low[p] == dfn[p]){
27         int top;
28         cebcc++;
29         do{
30             top = stk.back();
31             stk.pop_back();
32             st[top] = false;
33             ebcc[top] = cebcc;
34         }while(top != p);
35     }
36 }
37 void work(){
38     for(int i = 1; i <= N; i++){
39         if(!dfn[i])
40             tarjan(i, 0);
41     }
42     // 获取缩点后的树 (边双连通分量缩点后的树)
43     std::vector<std::vector<i64>> get_tree(){
44         std::vector<std::vector<i64>> tree(cebcc + 1);
45         for(int u = 1; u <= N; u++){
46             for(auto& [v,_] : e[u]){
47                 if(ebcc[u] != ebcc[v]){
48                     tree[ebcc[u]].push_back(ebcc[v]);
49                 }
50             }
51         }
52         for(int i = 1; i <= cebcc; i++){
53             std::sort(tree[i].begin(), tree[i].end());
54             tree[i].erase(std::unique(tree[i].begin(), tree[i].end()), tree[i].end());
55         }
56         return tree;
57     }
58 };

```

## 7.10 欧拉路径

### 7.10.1 有向图的欧拉路径

有向图欧拉路径存在: 恰有一个点出度比入度多 1(为起点); 恰有一个点入度比出度多 1(为终点); 恰有  $N - 2$  个点入度均等于出度。如果是欧拉回路, 则上方起点与终点的条件不存在, 全部点均要满足最后一个条件。

```

1 //有向图欧拉路径求解 (字典序最小)
2 vector<int> ans;
3 auto dfs = [&](auto self, int x) -> void {
4     while (ver[x].size()) {
5         int net = *ver[x].begin();
6         ver[x].erase(ver[x].begin());
7         self(self, net);
8     }
9     ans.push_back(x);
10 };
11 dfs(dfs, s);
12 reverse(ans.begin(), ans.end());
13 for (auto it : ans) {
14     cout << it << " ";
15 }

```

### 7.10.2 无向图的欧拉路径

无向图欧拉路径存在: 恰有两个点度数为奇数 (为起点与终点); 恰有  $N - 2$  个点度数为偶数。

```

1 //无向图欧拉路径求解
2 auto dfs = [&](auto self, int x) -> void {
3     while (ver[x].size()) {
4         int net = *ver[x].begin();
5         ver[x].erase(ver[x].find(net));
6         ver[net].erase(ver[net].find(x));
7         cout << x << " " << net << endl;
8         self(self, net);
9     }
10 };
11 dfs(dfs, s);

```

## 8 树上问题

### 8.1 树的直径

树上任意两节点之间最长的简单路径即为树的「直径」。

```
1 std::vector<int> dp(n + 1);
2 int ans = 0;
3 auto dfs = [&](auto&&self, int p, int f) -> void{
4     for(auto v : edge[p]){
5         if(v == f) continue;
6         self(self, v, p);
7         ans = std::max(ans, dp[p] + dp[v] + 1);
8         dp[p] = std::max(dp[p], dp[v] + 1);
9     }
10 };
11 dfs(dfs, 1, 0);
```

### 8.2 树的重心

- 某个点是树的重心等价于它最大子树大小不大于整棵树大小的一半。
- 树至多有两个重心。如果树有两个重心，那么它们相邻。此时树一定有偶数个节点，且可以被划分为两个大小相等的分支，每个分支各自包含一个重心。
- 树中所有点到某个点的距离和中，到重心的距离和是最小的；如果有两个重心，那么到它们的距离和一样。反过来，距离和最小的点一定是重心。
- 往树上增加或减少一个叶子，如果原节点数是奇数，那么重心可能增加一个，原重心仍是重心；如果原节点数是偶数，重心可能减少一个，另一个重心仍是重心。
- 把两棵树通过一条边相连得到一棵新的树，则新的重心在较大的一棵树一侧的连接点与原重心之间的简单路径上。如果两棵树大小一样，则重心就是两个连接点。

```
1 std::vector<int> dp(n + 1), siz(n + 1, 1);
2 std::array<int, 2> ans{};
3 auto dfs = [&](auto&&self, int p, int f) -> void{
4     for(auto v : edge[p]){
5         if(v == f) continue;
6         self(self, v, p);
7         siz[p] += siz[v];
8         dp[p] = std::max(dp[p], siz[v]);
9     }
10     dp[p] = std::max(dp[p], n - siz[p]);
11     if(ans[0] == 0) ans[0] = p;
```

```
12     else if(dp[p] < dp[ans[0]]) ans = {p, 0};
13     else if(dp[p] == dp[ans[0]]) ans[1] = p;
14 };
15 dfs(dfs, 1, 0);
```

### 8.3 最近公共祖先 (LCA)

#### 8.3.1 倍增算法

```
1 #include<bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5 const int N = 5e5 + 10, mod = 1e6 + 7;
6
7 struct edge{
8     ll next, to;
9 };
10
11 vector<edge> e;
12 ll head[N], Log2[N], fa[N][20], dep[N]; // fa的第二维大小不应小于log2(N)
13 bool vis[N];
14
15 void add(ll a, ll b){
16     e.push_back({head[a], b});
17     head[a] = e.size() - 1;
18 }
19
20 //O(nlog n)预处理
21 void dfs(ll cur, ll fath = 0){
22     if(vis[cur]) return;
23     vis[cur] = true;
24     dep[cur] = dep[fath] + 1;
25     fa[cur][0] = fath;
26     for(int i = 1; i <= Log2[dep[cur]]; ++i)
27         fa[cur][i] = fa[fa[cur][i - 1]][i - 1];
28     for(int eg = head[cur]; eg != -1; eg = e[eg].next)
29         dfs(e[eg].to, cur);
30 }
31
32 //O(log n)查询
33 ll lca(ll a, ll b){
34     if(dep[a] > dep[b]) swap(a, b);
35     while(dep[a] != dep[b])
36         b = fa[b][Log2[dep[b] - dep[a]]];
37     if(a == b) return a;
38     for(int k = Log2[dep[a]]; k >= 0; k--)
39         if(fa[a][k] != fa[b][k])
40             a = fa[a][k], b = fa[b][k];
```

```

44     return fa[a][0];
45 }
46
47 int main(){
48     ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
49     ll n,m,s,a,b;
50     cin >> n >> m >> s;
51     for(int i = 0;i <= n;i++) head[i] = -1;
52     for(int i = 0;i < n - 1;i++){
53         cin >> a >> b;
54         add(a,b);
55         add(b,a);
56     }
57     for (int i = 2; i <= n; ++i)
58         Log2[i] = Log2[i / 2] + 1; //预处理出log2(x)
59     dfs(s);
60     while(m--){
61         cin >> a >> b;
62         cout << lca(a,b) << "\n";
63     }
64     return 0;
65 }

```

### 8.3.2 重链剖分

```

1 struct HLD{
2     std::vector<std::vector<int>>> edge;
3     std::vector<int> fa,dep,son,siz,top,dfn;
4     int cnt,root;
5     HLD(){};
6     HLD(int n,int r = 1):edge(n + 1),fa(n + 1),dep(n + 1),
7         son(n + 1),siz(n + 1),top(n + 1),
8         dfn(n + 1),cnt(0),root(r){};
9     void addEdge(int u,int v){
10         edge[u].push_back(v);
11         edge[v].push_back(u);
12     }
13     void dfs1(int p){
14         dep[p] = dep[fa[p]] + 1,siz[p] = 1;
15         for(auto v : edge[p]){
16             if(v == fa[p]) continue;
17             fa[v] = p;
18             dfs1(v);
19             siz[p] += siz[v];
20             if(siz[v] > siz[son[p]]) son[p] = v;
21         }
22     }
23     void dfs2(int p,int t){
24         dfn[p] = ++cnt,top[p] = t;
25         if(son[p]) dfs2(son[p],t);
26         for(auto v : edge[p]){
27             if(v == fa[p] || v == son[p]) continue;
28             dfs2(v,v);

```

```

29     }
30 }
31 int lca(int u,int v){
32     while(top[u] != top[v]){
33         if(dep[top[u]] > dep[top[v]]) u = fa[top[u]];
34         else v = fa[top[v]];
35     }
36     if(dep[u] >= dep[v]) return v;
37     return u;
38 }
39 void work(){
40     dfs1(root);
41     dfs2(root,root);
42 }
43 };

```

## 8.4 树上差分

```

1 //点差分
2 //设将两点u,v之间路径上的所有点权增加x,则操作如下:
3 diff[u] += x,diff[v] += x,diff[lca(u,v)] -= x,diff[fa[lca(u,v)]] -= x;
4
5 //边差分
6 //将两点u,v之间路径上的所有边权增加x,以每条边两端深度较大的节点存储该边的差分数组,则操作如下:
7 diff[u] += x,diff[v] += x,diff[lca(u,v)] -= 2 * x;

```

## 8.5 树上点分治

遍历树上所有的路径，考虑包含这个点的路径和不包含这个点的路径。对于前者，我们做一趟 dfs；对于后者，我们删除该点后，对所有子树递归地处理即可。

例题：给定一棵有  $n(n \leq 10^4)$  个点的树， $m(m \leq 100)$  次询问询问树上距离为  $k$  的点对是否存在。

```

1 //Centroid Decomposition O(NlogN)
2 struct CD{
3     std::vector<std::vector<std::array<int,2>>>> edge;
4     std::vector<int> siz,del,dp,stk,dep,ask,ans;
5     int cnt = 0,root,n,m;
6     CD(){};
7     CD(int _n,int _m):edge(_n + 1),siz(_n + 1),del(_n + 1),dp(_n + 1),
8         dep(_n + 1),ask(_m + 1),ans(_m + 1),n(_n),m(_m){dp[0] = 2e9;}
9     void addEdge(int u,int v,int w){
10         edge[u].push_back({v,w});
11         edge[v].push_back({u,w});
12     }
13     void getSize(int p,int f = 0){
14         siz[p] = 1,cnt++;
15         for(auto [v,_] : edge[p]){
16             if(v == f || del[v]) continue;
17             getSize(v,p);

```

```

18     siz[p] += siz[v];
19 }
20 }
21 void getRoot(int p,int f = 0){
22     dp[p] = cnt - siz[p];
23     for(auto [v,_] : edge[p]){
24         if(v == f || del[v]) continue;
25         getRoot(v,p);
26         dp[p] = std::max(dp[p],siz[v]);
27     }
28     if(dp[p] < dp[root]) root = p;
29 }
30 void getDist(int p,int f = 0){
31     stk.push_back(dep[p]);
32     for(auto [v,w] : edge[p]){
33         if(v == f || del[v]) continue;
34         dep[v] = dep[p] + w;
35         getDist(v,p);
36     }
37 }
38 void calc(int u){
39     cnt = 0,getSize(u);
40     root = 0,getRoot(u);
41     del[root] = 1;
42     std::unordered_set<int> st;
43     st.insert(0);
44     for(auto [v,w] : edge[root]){
45         if(del[v]) continue;
46         dep[v] = w,getDist(v);
47         for(int i = 1;i <= m;i++){
48             for(auto len : stk){
49                 if(st.count(ask[i] - len))
50                     ans[i] = 1;
51             }
52         }
53         while(!stk.empty()){
54             st.insert(stk.back());
55             stk.pop_back();
56         }
57     }
58     for(auto [v,_] : edge[root]){
59         if(del[v]) continue;
60         calc(v);
61     }
62 }
63 };

```

## 8.6 树上启发式合并

```

1 const int N = 1e5 + 10,mod = 998244353;
2
3 //dsu on tree O(nlong n)

```

```

4 // son:重儿子,L[u]: 结点 u 的 DFS 序,R[u]: 结点 u 子树中结点的 DFS 序的最大值,Node[i]: DFS 序为 i 的
   结点
5 // ans: 存答案,cnt[i]: 颜色为 i 的结点个数,totColor: 目前出现过的颜色个数
6 ll n,idx,col[N],siz[N],node[N],l[N],r[N],son[N];
7 ll ans[N],cnt[N],colcnt;
8 vector<ll> edge[N];
9
10 void add(ll p){
11     if(cnt[col[p]] == 0) colcnt++;
12     cnt[col[p]]++;
13 }
14
15 void del(ll p){
16     cnt[col[p]]--;
17     if(cnt[col[p]] == 0) colcnt--;
18 }
19
20 void dfs(ll p,ll f){
21     l[p] = ++idx;
22     node[idx] = p;
23     siz[p] = 1;
24     for(auto& i : edge[p]){
25         if(i == f) continue;
26         dfs(i,p);
27         siz[p] += siz[i];
28         if(!son[p] || siz[son[p]] < siz[i]) son[p] = i;
29     }
30     r[p] = idx;
31 }
32
33 //只保留对重儿子的操作
34 void dfs1(ll p,ll f,bool keep){
35     for(auto& i : edge[p]){
36         if(i == f || i == son[p]) continue;
37         dfs1(i,p,false);
38     }
39     if(son[p]){
40         dfs1(son[p],p,true);
41     }
42     for(auto& i : edge[p]){
43         if(i == f || i == son[p]) continue;
44         // 子树结点的 DFS 序构成一段连续区间
45         for(int j = l[i];j <= r[i];j++){
46             add(node[j]);
47         }
48     }
49     add(p);
50     ans[p] = colcnt;
51     if(!keep){
52         for(int i = l[p];i <= r[p];i++){
53             del(node[i]);
54         }
55     }

```

```

56 }
57
58 int main(){
59     ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
60     ll n,m,q,u,v;
61     cin >> n;
62     for(int i = 2;i <= n;i++){
63         cin >> u >> v;
64         edge[u].push_back(v);
65         edge[v].push_back(u);
66     }
67     for(int i = 1;i <= n;i++){
68         cin >> col[i];
69     }
70     dfs(1,0);
71     dfs1(1,0,false);
72     cin >> q;
73     for(int i = 1;i <= q;i++){
74         cin >> u;
75         cout << ans[u] << "\n";
76     }
77     return 0;
78 }

```

## 8.7 Prüfer 序列

Prüfer 序列可以将一个带标号  $n$  个结点的树用  $[1, n]$  中的  $n - 2$  个整数表示。在构造完 Prüfer 序列后原树中会剩下两个结点，其中一个一定是编号最大的点  $n$ 。每个结点在序列中出现的次数是其度数减 1(没有出现的就是叶结点)。

```

1  std::vector<int> treeToPrufer(int n,std::vector<int>& fa){
2      std::vector<int> prufer(1),du(n + 1,1);
3      for(int i = 1;i < n;i++) du[fa[i]]++;
4      for(int i = 1;i < n;i++){
5          int now = i;
6          while(du[now] == 1){
7              du[now] = 0,du[fa[now]]--;
8              prufer.push_back(fa[now]);
9              if(fa[now] > i) break;
10             now = fa[now];
11         }
12     }
13     return prufer;
14 }
15
16 std::vector<int> pruferToTree(int n,std::vector<int>& prufer){
17     std::vector<int> fa(n),du(n + 1);
18     for(int i = 1;i <= n - 2;i++) du[prufer[i]]++;
19     int j = 1,now = 1;
20     for(int i = 1;i < n && j <= n - 2;i++){
21         now = i;
22         while(du[now] == 0 && j <= n - 2){

```

```

23             fa[now] = prufer[j];
24             j++;
25             du[fa[now]]--;
26             if(fa[now] > i) break;
27             now = fa[now];
28         }
29     }
30     fa[now] = n;
31     return fa;
32 }

```

相关结论：

- 无向完全图的不同生成树数 (凯莱公式):

若该完全图有  $n$  个节点, 则任意一个长度为  $n - 2$  的 Prüfer 序列都可以重构出其一棵生成树, 且各不相同。又因为 Prüfer 序列的值域在  $[1, n]$ , 故总数为  $n^{n-2}$ 。

- $n$  个点的有根树计数:

- 对每棵无根树来说, 每个点都可能是根, 故总数为  $n^{n-2} \times n = n^{n-1}$ 。

- $n$  个点, 每点度分别为  $d_i$  的无根树计数:

总排列为  $(n - 1)!$ , 即  $A_{n-2}^{n-2}$ 。

其中数字  $i$  出现了  $d_i - 1$  次, 故其重复的有  $(d_i - 1)!$  种排列, 即  $A_{d_i-2}^{d_i-2}$ 。

应当除去重复的, 故总个数为  $\frac{(n-2)!}{\prod_{i=1}^n (d_i-1)!}$

- 一个  $n$  个点  $m$  条边的带标号无向图有  $k$  个连通块。我们希望添加  $k - 1$  条边使得整个图连通, 求方案数量。

设  $s_i$  表示每个连通块的数量, 通项公式为  $n^{k-2} \cdot \prod_{i=1}^k s_i$ , 当  $k < 2$  时答案为 1。

## 9 网络流

### 9.1 dinic 算法

- 1. 在一般情况下 Dinic 算法的时间复杂度是  $O(|V|^2|E|)$ .
- 2. 在各边容量均为 1 的网络上, 时间复杂度是  $O(|E| \min(|E|^{\frac{1}{2}}, |V|^{\frac{2}{3}}))$ .
- 3. 在单位网络, 即满足除源汇外各点入度不超过 1 或出度不超过 1 且各边容量均为 1 的网络上, 时间复杂度是  $O(|E|\sqrt{|V|})$ .

```

1 struct dinic{
2     int n,s,t;
3     std::vector<int> head,cur,level;
4     struct edge{
5         i64 to,ne,w;
6     };
7     std::vector<edge> e;
8
9     dinic(int _n,int _s,int _t):
10         n(_n),s(_s),t(_t),
11         head(_n + 1,-1),cur(_n + 1,0),level(_n + 1,-1){}
12
13     void addEdge(int u,int v,i64 w){
14         e.push_back({v,head[u],w});
15         head[u] = e.size() - 1;
16         e.push_back({u,head[v],0});
17         head[v] = e.size() - 1;
18     }
19
20     bool bfs(){
21         std::queue<int> q;
22         q.push(s);
23         std::fill(level.begin(), level.end(), -1);
24         level[s] = 0;
25         while(!q.empty()){
26             int p = q.front();
27             q.pop();
28             for(int i = head[p];i != -1;i = e[i].ne){
29                 i64 v = e[i].to,w = e[i].w;
30                 if(w > 0 && level[v] == -1){
31                     level[v] = level[p] + 1;
32                     q.push(v);
33                     if(v == t) return true;
34                 }
35             }
36         }
37         return level[t] != -1;
38     }
39
40     i64 dfs(i64 p,i64 flow = 1e15){

```

```

41         if(p == t || flow == 0)
42             return flow;
43         i64 rest = flow;
44         for(int i = cur[p];i != -1 && rest;i = e[i].ne){
45             i64 v = e[i].to,w = e[i].w;
46             cur[p] = i;
47             if(w > 0 && level[v] == level[p] + 1){
48                 i64 c = dfs(v,std::min(rest,w));
49                 rest -= c;
50                 e[i].w -= c;
51                 e[i ^ 1].w += c;
52             }
53         }
54         if(flow - rest == 0) level[p] = -2;
55         return flow - rest;
56     }
57
58     i64 work(){
59         i64 res = 0;
60         while(bfs()){
61             cur = head;
62             res += dfs(s);
63         }
64         return res;
65     }
66 };
67
68 int main(){
69     std::ios::sync_with_stdio(0),std::cin.tie(0),std::cout.tie(0);
70     int n,m,s,t,u,v,w;
71     std::cin >> n >> m >> s >> t;
72     dinic f(n,s,t);
73     for(int i = 1;i <= m;i++){
74         std::cin >> u >> v >> w;
75         f.addEdge(u,v,w);
76     }
77     std::cout << f.work() << "\n";
78     return 0;
79 }

```

### 9.2 最小费用最大流

在网络  $G = (V, E)$  上, 对每条边给定一个权值  $w(u, v)$ , 称为费用 (cost), 含义是单位流量通过  $(u, v)$  所花费的代价. 对于  $G$  所有可能的最大流, 我们称其中总费用最小的一者为最小费用最大流.

```

1 struct MCMF{
2     static constexpr i64 INF = 1e18;
3     int n,s,t;
4     std::vector<int> head,cur,inq,vis;
5     std::vector<i64> dist;
6

```

```

7   MCMF(int _n,int _s,int _t):
8       n(_n),s(_s),t(_t),
9       head(_n + 1,-1),cur(_n + 1),dist(_n + 1),
10      inq(_n + 1),vis(_n + 1){}
11
12   struct edge{
13       i64 to,ne,w,c;
14   };
15   std::vector<edge> e;
16
17   void add(int u,int v,i64 w, i64 c){
18       e.push_back({v,head[u],w,c});
19       head[u] = e.size() - 1;
20   }
21
22   void addEdge(int u, int v,i64 w,i64 c){
23       add(u, v, w, c);
24       add(v, u, 0, -c);
25   }
26
27   bool SPFA(){
28       std::queue<int> q;
29       cur = head;
30       std::fill(dist.begin(),dist.end(),1e18);
31       dist[s] = 0;
32       q.push(s);
33       while (!q.empty()){
34           i64 p = q.front();
35           q.pop();
36           inq[p] = 0;
37           for (int i = head[p];i != -1;i = e[i].ne){
38               i64 v = e[i].to, w = e[i].w;
39               if (w > 0 && dist[v] > dist[p] + e[i].c){
40                   dist[v] = dist[p] + e[i].c;
41                   if (!inq[v]){
42                       q.push(v);
43                       inq[v] = 1;
44                   }
45               }
46           }
47       }
48       return dist[t] < INF;
49   }
50
51   i64 dfs(int p,i64 flow = INF){
52       if(p == t || flow == 0)
53           return flow;
54       vis[p] = 1;
55       i64 rest = flow;
56       for(int i = cur[p];i != -1 && rest;i = e[i].ne){
57           i64 v = e[i].to,w = e[i].w;
58           cur[p] = i;
59           if(w > 0 && !vis[v] && dist[v] == dist[p] + e[i].c){

```

```

60               i64 c = dfs(v,std::min(rest,w));
61               rest -= c;
62               e[i].w -= c;
63               e[i ^ 1].w += c;
64           }
65       }
66       vis[p] = 0;
67       return flow - rest;
68   }
69
70   i64 maxflow,mincost;
71   void work(){
72       maxflow = mincost = 0;
73       while (SPFA()){
74           i64 flow = dfs(s);
75           maxflow += flow;
76           mincost += dist[t] * flow;
77       }
78   }
79 };
80
81   int main(){
82       std::ios::sync_with_stdio(0),std::cin.tie(0),std::cout.tie(0);
83       int n,m,s,t,u,v,w,c;
84       std::cin >> n >> m >> s >> t;
85       MCMF f(n,s,t);
86       for(int i = 1;i <= m;i++){
87           std::cin >> u >> v >> w >> c;
88           f.addEdge(u,v,w,c);
89       }
90       f.work();
91       std::cout << f.maxflow << " " << f.mincost << "\n";
92       return 0;
93   }

```

## 9.3 网络流建图技巧

### 9.3.1 二分图最大匹配

- 1. 首先, 在图中额外添加一个源点  $s$  和汇点  $t$ .
- 2. 对于每个顶点  $u \in V_1$ , 添加一条从源点  $s$  到  $u$  的边, 边的容量设为 1.
- 3. 对于每个顶点  $v \in V_2$ , 添加一条从  $v$  到汇点  $t$  的边, 边的容量设为 1.
- 4. 对于二分图中的每一条边  $e = (u, v)$ , 其中  $u \in V_1$  且  $v \in V_2$ , 添加一条从  $u$  到  $v$  的边, 边的容量也设为 1.
- 5. 在这个流网络中, 我们的目标是找到最大流, 最大流量的大小即为对应于二分图中的最大匹配的边数.



### 9.3.2 二分图最大权匹配

- 1. 首先, 在图中额外添加一个源点  $s$  和汇点  $t$ .
- 2. 从源点向二分图的每个左部点连一条流量为 1, 费用为 0 的边.
- 3. 从二分图的每个右部点向汇点连一条流量为 1, 费用为 0 的边.
- 4. 接下来对于二分图中每一条连接左部点  $u$  和右部点  $v$ , 边权为  $w$  的边, 则连一条从  $u$  到  $v$ , 流量为 1, 费用为  $w$  的边.
- 5. 另外, 考虑到最大权匹配下, 匹配边的数量不一定与最大匹配的匹配边数量相等, 因此对于每个左部点, 还需向汇点连一条流量为 1, 费用为 0 的边.
- 6. 求这个网络的最大费用最大流即可得到答案. 此时, 该网络的最大流量一定为左部点的数量, 而最大流量下的最大费用即对应一个最大权匹配方案.

### 9.3.3 二者取其一式

**问题简介** 有  $n$  个元素, 每个元素有两种选择, 分别对应不同的收益/代价并且当某些元素同时选择某一选项时会产生额外的收益/代价目标是通过给每个元素分配合适的选择, 使得总收益最大 (或总代价最小).

- 1. 建立源点  $s$  和汇点  $t$ , 分别代表两种不同的选择类别.
- 2. 对于每个元素  $i$ , 从源点  $s$  向该节点连一条边, 容量为选择第一种选择的收益/代价.
- 3. 从该节点向汇点  $t$  连一条边, 容量为选择第二种选择的收益/代价.
- 4. 对于元素之间的附加条件 (如同选奖励或排斥惩罚), 通过在相应节点之间连边来表示, 容量为对应的收益或代价值.
- 5. 求最小割, 总收益为所有可能收益之和减去最小割的容量.

### 9.3.4 最小点覆盖

**问题简介** 要求在一个无向图中选出最少的点, 使得每条边至少有一个端点在选出的点集中. 在二分图中, 根据 König 定理, 最小点覆盖数 = 最大匹配数.

- 1. 对于二分图  $G = (U, V, E)$ , 建立源点  $s$  和汇点  $t$ .
- 2. 从源点  $s$  向  $U$  中每个节点连一条容量为 1 的边.
- 3. 从  $V$  中每个节点向汇点  $t$  连一条容量为 1 的边.

- 4. 对于每条边  $(u, v) \in E$  (其中  $u \in U, v \in V$ ), 连一条从  $u$  到  $v$  的边, 容量为  $\infty$ .
- 5. 求最大流, 最大流的值即为最小点覆盖的大小.

### 9.3.5 最大权闭合子图

**问题简介** 在一个有向图中找到一个点集, 使得该点集的所有出边都指向点集内部 (即闭合性), 并且点集的权值和最大. 该问题常见于依赖关系建模.

- 1. 建立源点  $s$  和汇点  $t$ .
- 2. 对于每个权值  $w_i > 0$  的点  $i$ , 从源点  $s$  向该点连一条容量为  $w_i$  的边.
- 3. 对于每个权值  $w_i < 0$  的点  $i$ , 从该点向汇点  $t$  连一条容量为  $-w_i$  的边.
- 4. 对于原图中的每条有向边  $(u, v)$ , 连一条从  $u$  到  $v$  的边, 容量为  $\infty$ .
- 5. 最大权闭合子图的权值 = 所有正权值之和 - 最小割的容量.

### 9.3.6 最大点权独立集问题

**问题简介** 在一个无向图中选出权值和最大的点集, 使得点集中任意两点之间没有边相连. 在一般图中该问题是 NP 难的, 但在二分图中有多项式解法.

- 1. 建立源点  $s$  和汇点  $t$ .
- 2. 将二分图  $G = (U, V, E)$  中的  $U$  部点与源点  $s$  相连, 容量为点权.
- 3. 将  $V$  部点与汇点  $t$  相连, 容量为点权.
- 4. 对于每条边  $(u, v) \in E$ , 连一条从  $u$  到  $v$  的边, 容量为  $\infty$ .
- 5. 最大点权独立集的权值 = 所有点权之和 - 最小割的容量.

### 9.3.7 最小路径覆盖问题

**问题简介** 在一个有向无环图 (DAG) 中用最少的简单路径覆盖所有顶点, 且每个顶点恰好被一条路径覆盖. 路径之间不能有公共点.

- 1. 将原图  $G$  中的每个点  $i$  拆成两个点  $i$  和  $i'$ , 分别放在二分图的左右两部.
- 2. 建立源点  $s$  和汇点  $t$ .
- 3. 从源点  $s$  向所有左部点连一条容量为 1 的边.
- 4. 从所有右部点向汇点  $t$  连一条容量为 1 的边.

- 5. 对于原图中的每条边  $(u, v)$ , 在二分图中从左部点  $u$  向右部点  $v'$  连一条容量为 1 的边.
- 6. 最小路径覆盖数 = 原图顶点数 - 最大流的值.

### 9.3.8 公平分配问题

**问题简介** 有  $m$  个任务和  $n$  个人员, 每个人员能完成某些任务且最多完成  $a_i$  个任务. 要求分配任务使得尽可能多的任务被完成, 同时满足人员的容量限制.

- 1. 建立源点  $s$  和汇点  $t$ .
- 2. 从源点  $s$  向每个人员节点  $P_i$  连一条容量为  $a_i$  的边.
- 3. 从每个任务节点  $T_j$  向汇点  $t$  连一条容量为 1 的边.
- 4. 如果人员  $P_i$  能完成任务  $T_j$ , 则从  $P_i$  向  $T_j$  连一条容量为 1 的边.
- 5. 求最大流, 其值即为能完成的最大任务数.

### 9.3.9 餐饮问题

**问题简介** 有  $n$  头牛、 $F$  种食物和  $D$  种饮料, 每头牛有喜欢的食物和饮料列表, 但只能分配一种食物和一种饮料. 每种食物和饮料数量有限, 求最多能满足多少头牛.

- 1. 建立源点  $s$  和汇点  $t$ .
- 2. 从源点  $s$  向每种食物节点连一条容量为食物数量的边.
- 3. 从每种饮料节点向汇点  $t$  连一条容量为饮料数量的边.
- 4. 将每头牛拆分成两个节点 (牛左、牛右), 中间连一条容量为 1 的边.
- 5. 如果牛喜欢某种食物, 从该食物节点向对应的牛左节点连一条容量为 1 的边.
- 6. 如果牛喜欢某种饮料, 从对应的牛右节点向该饮料节点连一条容量为 1 的边.
- 7. 求最大流, 其值即为能满足的牛的数量.

### 9.3.10 上下界可行流问题

**问题简介** 在网络流问题中, 给某些边加上流量下界限制, 即流量必须在  $[l, r]$  范围内. 判断是否存在满足上下界约束的可行流.

- 1. 建立附加源点  $s'$  和附加汇点  $t'$ .
- 2. 对于原图中每条边  $(u, v, l, r)$ : 从  $u$  向  $v$  连容量为  $r - l$  的边, 同时记录  $u$  的出流量下限和  $+l$ ,  $v$  的入流量下限和  $+l$ .
- 3. 从附加源点  $s'$  向每个节点  $u$  连边, 容量为该节点的入流量下界和.
- 4. 从每个节点  $u$  向附加汇点  $t'$  连边, 容量为该节点的出流量下界和.
- 5. 从原汇点  $t$  向原源点  $s$  连一条容量为  $\infty$  的边.
- 6. 求  $s'$  到  $t'$  的最大流, 如果满流则存在可行流.

### 9.3.11 最大密度子图问题

**问题简介** 给定无向图, 求一个子图使得子图的边数与点数的比值 (密度) 最大. 这是分数规划与网络流结合的典型问题.

- 1. 采用二分答案法, 假设当前判断的密度值为  $g$ .
- 2. 将每条边视为一个权值为 1 的点, 每个原图节点视为一个权值为  $-g$  的点.
- 3. 建立源点  $s$  和汇点  $t$ .
- 4. 对于每条边点  $e_i$ , 从  $s$  向  $e_i$  连容量为 1 的边.
- 5. 对于每个原图节点  $v_j$ , 从  $v_j$  向  $t$  连容量为  $g$  的边.
- 6. 如果边  $e_i$  关关节点  $u$  和  $v$ , 则从  $e_i$  向  $u$  和  $v$  连容量为  $\infty$  的边.
- 7. 如果最大权闭合子图的权值  $\geq 0$ , 则密度  $\geq g$ , 否则密度  $< g$ .

## 10 动态规划

### 10.1 多重背包 (单调队列优化)

```

1 int main(){
2     ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
3     ll n,V;
4     cin >> n >> V;
5     vector<ll> f(V + 1),g(V + 1);
6     for(int i = 1;i <= n;i++){
7         ll c,w,v;
8         cin >> v >> w >> c;
9         g = f;
10        vector<deque<ll>> q(w);
11        for(int j = 0;j <= V;j++){
12            if(!q[j % w].empty() && (j - q[j % w].front()) / w > c)
13                q[j % w].pop_front();
14
15            if(!q[j % w].empty()) f[j] = max(f[j],g[q[j % w].front()] + (j - q[j % w].front()) / w * v);
16            while(!q[j % w].empty() && g[j] >= g[q[j % w].back()] + (j - q[j % w].back()) / w * v)
17                q[j % w].pop_back();
18            q[j % w].push_back(j);
19        }
20    }
21    cout << *max_element(f.begin(),f.end());
22    return 0;
23 }

```

### 10.2 分组背包

有  $n$  组物品，一个容量为  $W$  的背包，每组物品有若干，同一组的物品最多选一个，第  $i$  组第  $j$  件物品的体积为  $w[i][j]$ ，价值为  $v[i][j]$ ，求解将哪些物品装入背包，可使物品总体积不超过背包容量，且使总价值最大。

考虑每组中的某件物品选不选，可以选的话，去下一组选下一个，否则在这组继续寻找可以选的物品，当这组遍历完后，去下一组寻找。

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int N = 110;
4 int n, W, s[N], w[N][N], v[N][N], dp[N];
5 int main(){
6     cin >> n >> W;
7     for (int i = 1; i <= n; i++){
8         scanf("%d", &s[i]);
9         for (int j = 1; j <= s[i]; j++)
10             scanf("%d %d", &w[i][j], &v[i][j]);
11     }
12     for (int i = 1; i <= n; i++)
13         for (int j = W; j >= 0; j--)

```

```

14         for (int k = 1; k <= s[i]; k++)
15             if (j - w[i][k] >= 0)
16                 dp[j] = max(dp[j], dp[j - w[i][k]] + v[i][k]);
17     cout << dp[W] << "\n";
18     return 0;
19 }

```

### 10.3 有依赖的背包

有  $n$  个物品和一个容量为  $W$  的背包，物品之间有依赖关系，且之间的依赖关系组成一颗树的形状，如果选择一个物品，则必须选择它的父节点，第  $i$  件物品的体积是  $w[i]$ ，价值为  $v[i]$ ，依赖的父节点的编号为  $p[i]$ ，若  $p[i]$  等于 -1，则为根节点。求将哪些物品装入背包中，使总体积不超过总容量，且总价值最大。

定义  $f[i][j]$  为以第  $i$  个节点为根，容量为  $j$  的背包的最大价值。那么结果就是  $f[root][W]$ ，为了知道根节点的最大价值，得通过其子节点来更新。所以采用递归的方式。对于每一个点，先将这个节点装入背包，然后找到剩余容量可以实现的最大价值，最后更新父节点的最大价值即可。

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int N = 110;
4 int n, W, s[N], w[N][N], v[N][N], dp[N];
5 int main(){
6     cin >> n >> W;
7     for (int i = 1; i <= n; i++){
8         scanf("%d", &s[i]);
9         for (int j = 1; j <= s[i]; j++)
10             scanf("%d %d", &w[i][j], &v[i][j]);
11     }
12     for (int i = 1; i <= n; i++)
13         for (int j = W; j >= 0; j--)
14             for (int k = 1; k <= s[i]; k++)
15                 if (j - w[i][k] >= 0)
16                     dp[j] = max(dp[j], dp[j - w[i][k]] + v[i][k]);
17     cout << dp[W] << "\n";
18     return 0;
19 }

```

### 10.4 数位 DP

```

1 /* pos 表示当前枚举到第几位,sum 表示 d 出现的次数,limit 为 1 表示枚举的数字有限制,
2 zero 为 1 表示有前导 0,d 表示要计算出出现次数的数 */
3 static constexpr int N = 15;
4 i64 dp[N][N],num[N];
5 i64 dfs(int pos, i64 sum, int limit, int zero, int d) {
6     if (pos == 0) return sum;
7     if (!limit && !zero && dp[pos][sum] != -1) return dp[pos][sum];
8     i64 ans = 0;

```

```

9   int up = (limit ? num[pos] : 9);
10  for (int i = 0; i <= up; i++) {
11      ans += dfs(pos - 1, sum + ((!zero || i) && (i == d)), limit && (i == num[pos]),
12          zero && (i == 0), d);
13  }
14  if (!limit && !zero) dp[pos][sum] = ans;
15  return ans;
16 }
17 i64 solve(i64 x, int d) {
18     std::fill(dp.begin(), dp.end(), -1);
19     int len = 0;
20     while (x) {
21         num[++len] = x % 10;
22         x /= 10;
23     }
24     return dfs(len, 0, 1, 1, d);
25 }

```

## 10.5 SOSDP

$O(N2^N)$  求解子集和问题

```

1  for(int i = 0; i < (1 << N); i++)
2      dp[i] = a[i];
3  for(int i = 0; i < N; i++)
4      for(int mask = (1 << N) - 1; mask >= 0; mask--)
5          if(mask & (1 << i))
6              dp[mask] += dp[mask ^ (1 << i)];

```

## 10.6 悬线法

```

1  int c[N][N], u[N][N], r[N][N], l[N][N];
2  int main(){
3      ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
4      int n, m, ans = 0;
5      cin >> n >> m;
6      for(int i = 1; i <= n; i++)
7          for(int j = 1; j <= m; j++)
8              cin >> c[i][j];
9
10     for(int i = 1; i <= n; i++){
11         for(int j = 1; j <= m; j++){
12             if(c[i][j]) u[i][j] = u[i - 1][j] + 1;
13             if(c[i][j]) l[i][j] = l[i][j - 1] + 1;
14         }
15     }
16
17     for(int i = 1; i <= n; i++)
18         for(int j = m; j >= 1; j--)
19             if(c[i][j]) r[i][j] = r[i][j + 1] + 1;
20
21     for(int i = 1; i <= n; i++)

```

```

22     for(int j = m; j >= 1; j--){
23
24         for(int i = 1; i <= n; i++){
25             for(int j = 1; j <= m; j++){
26                 if(c[i][j]){
27                     if(c[i - 1][j]){
28                         r[i][j] = min(r[i][j], r[i - 1][j]);
29                         l[i][j] = min(l[i][j], l[i - 1][j]);
30                     }
31                 }
32                 ans = max(ans, (r[i][j] + l[i][j] - 1) * u[i][j]); //最大矩形
33                 //ans = max(ans, min(r[i][j] + l[i][j] - 1, u[i][j]) * min(r[i][j] + l[i][j] - 1, u[i][j]
34                     )); //最大正方形
35             }
36         }
37         cout << ans << "\n";
38         return 0;
39     }

```

## 10.7 编辑距离

```

1  ll dp[2][2010];
2  /*
3  替换/跳过 删除
4  插入 当前
5  */
6  int main(){
7      ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
8      string a, b;
9      cin >> a >> b;
10     for(int i = 1; i <= b.size(); i++)
11         dp[0][i] = i;
12
13     for(int i = 1; i <= a.size(); i++){
14         dp[1][0] = i;
15         dp[0][0] = i - 1;
16         for(int j = 1; j <= b.size(); j++){
17             if(a[i - 1] == b[j - 1])
18                 dp[1][j] = dp[0][j - 1];
19             else
20                 dp[1][j] = min(min(dp[0][j], dp[0][j - 1]), dp[1][j - 1]) + 1;
21         }
22
23         for(int j = 1; j <= b.size(); j++){
24             dp[0][j] = dp[1][j];
25         }
26     }
27
28     cout << dp[1][b.size()] << "\n";
29     return 0;
30 }

```

## 10.8 最小斯坦纳树

给定一个带权无向图  $G = (V, E)$  和终端点集  $S \subseteq V$ ，求包含  $S$  中所有点的最小权连通子图。

```

1  const int N = 5e2 + 10, M = 5e4 + 10, mod = 1e6 + 7;
2  vector<PII> edge[N];
3  ll key[12], dp[N][M], n, m, k, vis[N];
4  priority_queue<PII, vector<PII>, greater<PII>> q;
5  void dijkstra(ll s){
6      for(int i = 0; i <= n; i++){
7          vis[i] = 0;
8      }
9      while(!q.empty()){
10         PII t = q.top();
11         q.pop();
12         if(vis[t.second]) continue;
13         vis[t.second] = 1;
14         for(auto i : edge[t.second]){
15             ll v = i.first, w = i.second;
16             if(dp[v][s] > dp[t.second][s] + w){
17                 dp[v][s] = dp[t.second][s] + w;
18                 q.push({dp[v][s], v});
19             }
20         }
21     }
22 }
23 int main(){
24     ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
25     cin >> n >> m >> k;
26     for(int i = 1; i <= m; i++){
27         ll u, v, w;
28         cin >> u >> v >> w;
29         edge[u].push_back({v, w});
30         edge[v].push_back({u, w});
31     }
32     for(int i = 0; i <= n; i++){
33         for(int j = 0; j <= (1LL << k); j++){
34             dp[i][j] = 1e15;
35         }
36     }
37     for(int i = 1; i <= k; i++){
38         cin >> key[i];
39         dp[key[i]][1LL << (i - 1)] = 0;
40     }
41     for(ll s = 1; s <= (1LL << k) - 1; s++){
42         for(int i = 1; i <= n; i++){
43             for(ll sub = s & (s - 1); sub > 0; sub = s & (sub - 1)){
44                 dp[i][s] = min(dp[i][s], dp[i][sub] + dp[i][sub ^ s]);
45                 if(dp[i][s] < 1e15) q.push({dp[i][s], i});
46             }
47         }
48         dijkstra(s);
49     }
50     cout << dp[key[1]][(1LL << k) - 1] << "\n";
51     return 0;
52 }

```

## 11 C++ 语法

### 11.1 int128

```

1 using i128 = __int128;
2
3 i128 to_128(string a){
4     i128 res = 0;
5     for(int i = a.size() - 1; i >= 0; i--){
6         res = res * 10 + a[i] - '0';
7     }
8     return res;
9 }
10
11 std::ostream &operator<<(std::ostream &os, i128 n) {
12     std::string s;
13     if(n == 0) s = "0";
14     while (n) {
15         s += '0' + n % 10;
16         n /= 10;
17     }
18     std::reverse(s.begin(), s.end());
19     return os << s;
20 }

```

### 11.2 随机数

```

1 std::mt19937_64 rng {std::chrono::steady_clock::now().time_since_epoch().count()};
2 a = rng(); //调用
3 shuffle(a.begin(), a.end(), rng()); //打乱数组

```

### 11.3 手工哈希

```

1 struct myhash {
2     static uint64_t hash(uint64_t x) {
3         x += 0x9e3779b97f4a7c15;
4         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
5         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
6         return x ^ (x >> 31);
7     }
8     size_t operator()(uint64_t x) const {
9         static const uint64_t SEED = chrono::steady_clock::now().time_since_epoch().count();
10        return hash(x + SEED);
11    }
12    size_t operator()(pair<uint64_t, uint64_t> x) const {
13        static const uint64_t SEED = chrono::steady_clock::now().time_since_epoch().count();
14        return hash(x.first + SEED) ^ (hash(x.second + SEED) >> 1);
15    }
16 };
17 // unordered_map<int, int, myhash>

```

### 11.4 速度测试

```

1 // #pragma GCC optimize("Ofast", "unroll-loops")
2 mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());
3 signed main() {
4     size_t n = 340000000, seed = 0;
5     for (int i = 1; i <= n; i++) {
6         seed ^= rnd();
7     }
8     return 0;
9 }

```

### 11.5 读取一行数字，个数未知

```

1 string s;
2 getline(cin, s);
3 stringstream ss;
4 ss << s;
5 while (ss >> s) {
6     auto res = stoi(s);
7     cout << res * 100 << endl;
8 }

```

### 11.6 输出流控制

```

1 #include<iomanip>
2 void Solve() {
3     cout << setw(12) << 314 << endl; //设置字段宽度,默认用空格补全。
4     cout << setw(12) << setfill('*') << 314 << endl; //设置填充字符,注意这里的x只能为char类型。
5     cout << oct << 314 << endl; //八进制输出,输入同理
6     cout << dec << 314 << endl; //十进制
7     cout << hex << 314 << endl; //十六进制
8     cout << setbase(12) << 314 << endl; //自定义进制,输入同理
9     cout << fixed << setprecision(10) << 3.14 << endl; //精度控制
10 }

```

### 11.7 查找后继

*lower* 表示  $\geq$ , *upper* 表示  $>$ 。使用前记得先进行排序。

```

1 //返回a数组[start,end)区间中第一个>=x的地址【地址!!!】
2 cout << lower_bound(a + start, a + end, x);
3 cout << lower_bound(a, a + n, x) - a; //在a数组中查找第一个>=x的元素下标
4 upper_bound(a, a + n, k) - lower_bound(a, a + n, k) //查找k在a中出现了几次

```

### 11.8 bit 库与位运算函数

对 *longlong* 操作只需要在函数后面加上 *ll* 即可 (例如 `__builtin_popcountll(x)`)

```

1 __builtin_popcount(x) // 返回x二进制下含1的数量,例如x=15=(1111)时答案为4

```

```

2  __builtin_ffs(x) // 返回x右数第一个1的位置(1-idx),1(1) 返回 1,8(1000) 返回 4,26(11010) 返回 2
3  __builtin_ctz(x) // 返回x二进制下后导0的个数,1(1) 返回 0,8(1000) 返回 3
4  bit_width(x) // 返回x二进制下的位数,9(1001) 返回 4,26(11010) 返回 5

```

## 11.9 优先队列

默认升序 (大根堆), 自定义排序需要重载 < 。

```

1  //重载运算符【注意，符号相反!!!】
2  struct Node {
3      int x; string s;
4      friend bool operator < (const Node &a, const Node &b) {
5          if (a.x != b.x) return a.x > b.x;
6          return a.s > b.s;
7      }
8  };

```

## 11.10 bitset

将数据转换为二进制, 从高位到低位排序, 以 0 为最低位。当位数相同时支持全部的位运算。

```

1  // 如果输入的是01字符串,可以直接使用">>"读入
2  bitset<10> s;
3  cin >> s;
4
5  //使用只含01的字符串构造——bitset<容器长度>B (字符串)
6  string S; cin >> S;
7  bitset<32> B (S);
8
9  //使用整数构造 (两种方式)
10 int x; cin >> x;
11 bitset<32> B1 (x);
12 bitset<32> B2 = x;
13
14 // 构造时,尖括号里的数字不能是变量
15 int x; cin >> x;
16 bitset<x> ans; // 错误构造
17
18 [] //随机访问
19 set(x) //将第x位置1,x省略时默认全部位置1
20 reset(x) //将第x位置0,x省略时默认全部位置0
21 flip(x) //将第x位取反,x省略时默认全部位取反
22 to_ullong() //重转换为ULL类型
23 to_string() //重转换为ULL类型
24 count() //返回1的个数
25 any() //判断是否至少有一个1
26 none() //判断是否全为0
27
28 _Find_fisrt() // 找到从低位到高位第一个1的位置
29 _Find_next(x) // 找到当前位置x的下一个1的位置,复杂度 O(n/w + count)
30

```

```

31 bitset<23> B1("11101001"), B2("11101000");
32 cout << (B1 ^ B2) << "\n"; //按位异或
33 cout << (B1 | B2) << "\n"; //按位或
34 cout << (B1 & B2) << "\n"; //按位与
35 cout << (B1 == B2) << "\n"; //比较是否相等
36 cout << B1 << " " << B2 << "\n"; //你可以直接使用cout输出

```

## 11.11 使用构造函数

将一些必要的声明和预处理放在构造函数, 在编译时, 无论放置在程序的哪个位置, 都会先于主函数进行。

```

1  int __FAST_IO__ = []() { // 函数名称可以随意修改
2      ios::sync_with_stdio(0), cin.tie(0);
3      cout.tie(0);
4      cout << fixed << setprecision(12);
5      freopen("out.txt", "r", stdin);
6      freopen("in.txt", "w", stdout);
7      return 0;
8  }();

```

## 11.12 其他

```

1  shuffle(ver.begin(), ver.end(), rnd); //数组打乱
2  iota(ver.begin(), ver.end(), x); //将ver区间复制成 “x,x+1,x+2,...”
3  ver.erase(unique(ver.begin(),ver.end()),ver.end()); //数组去重+删除
4  next_permutation(ver.begin(), ver.end()); //字典序全排列(prev_permutation()为逆序)
5  is_sorted(ver.begin(), ver.end()); //ver区间是否是递增的,返回bool型
6  accumulate(a + start, a + end, x); //累加,并输出累加和+x(注意longlong)的值
7
8  string to_string(T val); //数字转字符串
9  str.find(c); //字符串中查找子字符串或字符的第一个出现位置。如果找不到,则返回string::npos。
10 s.find(str, pos);//从位置pos开始查找子字符串str。
11 s.find_first_of(str);//查找字符串s中第一个出现在字符串str中的字符的位置。
12 s.find_last_of(str);//查找字符串s中最后一个出现在字符串str中的字符的位置
13
14 exp2(x); //2^x
15 log2(x); //\log_2(x)
16
17 prev(it) / next(it); //默认返回迭代器it的前/后一个迭代器
18 prev(it, 2) / next(it, 2); //可选参数可以控制返回前/后任意个迭代器

```

## 11.13 编译器设置

```

1  g++ -O2 -std=c++20 -pipe
2  -Wall -Wextra -Wconversion /* 这部分是警告相关,可能用不到 */
3  -fstack-protector
4  -Wl,--stack=268435456

```

## 12 杂项

### 12.1 动态大小 bitset

```

1 template<int N>
2 void solve(int n){
3     while(N <= n){
4         solve<std::min(N << 1,max_N)>(n);
5         return 0;
6     }
7     std::bitset<N> st;
8 }

```

### 12.2 Gosper's Hack

生成  $n$  元集合所有  $k$  元子集的算法, 只会遍历  $C_n^k$  种情况。

```

1 void GopersHack(int k, int n){
2     int cur = (1 << k) - 1;
3     int limit = (1 << n);
4     while (cur < limit && cur){
5         // do something
6         int lb = cur & -cur;
7         int r = cur + lb;
8         cur = ((r ^ cur) >> __builtin_ctz(lb) + 2) | r;
9         // 或: cur = (((r ^ cur) >> 2) / lb) | r;
10    }
11 }

```

### 12.3 绝对众数

#### 12.3.1 摩尔投票

选出 1 个候选人, 要求这个人的得票都超过总票数的一半

```

1 int m = 0, cnt = 0;
2 for(int i = 0; i < n; i++){
3     if(cnt == 0) m = nums[i];
4     if(m == nums[i]) cnt++;
5     else cnt--;
6 }
7 // 最后需要验证答案是否符合要求

```

#### 12.3.2 拓展摩尔投票

选出  $N$  个候选人, 要求这个人的得票都超过总票数的  $\frac{1}{N+1}$

```

1 int m[N], cnt[N];
2 for(auto e : nums){
3     int i = find(m, m + N, e) - m;
4     if(i != N){ // 如果当前票投给了候选人之一
5         cnt[i]++;

```

```

6         continue;
7     }
8     int j = find(cnt, cnt + N, 0) - cnt;
9     if(j != N){ // 如果当前存在一个位置"虚位以待"
10        m[j] = e;
11        cnt[j] = 1;
12        continue;
13    }
14    for(auto &c : cnt) c--;
15 }
16 // 最后需要验证答案是否符合要求

```

#### 12.3.3 区间绝对众数 (线段树上摩尔投票)

当绝对众数存在时, 可以任意交换摩尔投票的计票顺序, 而不改变选出的候选人。在合并左右两个区间时, 相当于是先对左右两边分别进行摩尔投票, 再对两边抵消后的结果进行摩尔投票。

```

1 struct Node{
2     int m, cnt;
3     Node operator+(const Node &o) const{
4         if (m == o.m) return {m, cnt + o.cnt};
5         else if (cnt > o.cnt) return {m, cnt - o.cnt};
6         return {o.m, o.cnt - cnt};
7     }
8 };
9 // 最后需要验证答案是否符合要求
10 // vector<int> pos[MAXV];
11 // pos[m]表示权值m的下标数组, 需要预处理
12 // 如果权值域过大, 需要改用map或哈希表
13 auto itr = upper_bound(pos[m].begin(), pos[m].end(), right);
14 auto itl = lower_bound(pos[m].begin(), pos[m].end(), left);
15 if (itr - itl > (right - left + 1) / 3)
16     return m;
17 else
18     return -1;

```

### 12.4 模拟退火

```

1 using point = std::array<double, 2>;
2 using pointW = std::array<double, 3>;
3 const double T0 = 10000.0; // 初始温度
4 const double q = 0.98; // 退火系数
5 const int L = 100; // 每次迭代次数
6 std::mt19937_64 rng{std::chrono::steady_clock::now().time_since_epoch().count()};
7 std::uniform_real_distribution<double> rnd(-1.0, 1.0);
8 point Point{}, bestPoint{};
9 double best = 1e18;
10 std::vector<pointW> points;
11 double dist(const point &a, const pointW &b) {
12     return sqrtl((a[0] - b[0]) * (a[0] - b[0]) + (a[1] - b[1]) * (a[1] - b[1])) * b[2];

```



```

13 }
14 double calc(const point &p) {
15     double res = 0;
16     for (int i = 0; i < (int)points.size(); i++)
17         res = std::max(res, dist(p, points[i]));
18     return res;
19 }
20 void get_new(point &p, double T) {
21     p[0] += rnd(rng) * T;
22     p[1] += rnd(rng) * T;
23 }
24 void codeFire() {
25     double T = T0, las = best;
26     while(T > 1e-6){
27         for(int i = 0; i < L; i++){
28             point temp = Point;
29             get_new(temp, T);
30             double now = calc(temp);
31             double DE = now - las;
32             if(DE < 0 || exp(-DE / T) > ((rng() % 1000000) / 1000000.0)){
33                 Point = temp;
34                 las = now;
35                 if(now < best){
36                     best = now;
37                     bestPoint = Point;
38                 }
39             }
40         }
41         T *= q;
42     }
43 }
44 int main() {
45     std::ios::sync_with_stdio(0), std::cin.tie(0), std::cout.tie(0);
46     int n;
47     std::cin >> n;
48     points.resize(n);
49     for (int i = 0; i < n; i++) {
50         double x, y, c;
51         std::cin >> x >> y >> c;
52         points[i] = {x, y, c};
53         Point[0] += x, Point[1] += y;
54     }
55     Point[0] /= n, Point[1] /= n;
56
57     best = calc(Point);
58     bestPoint = Point;
59     codeFire();
60     std::cout << std::fixed << std::setprecision(10) << best << "\n";
61     return 0;
62 }

```

## 12.5 莫队

以  $O(N\sqrt{N})$  的复杂度完成  $Q$  次询问的离线查询, 其中每个分块的大小取  $\sqrt{N} = \sqrt{10^5} = 317$ , 也可以使用  $n/\min < \text{int} > (n, \text{sqrt}(q))$ 、 $\text{ceil}((\text{double})n/(\text{int})\text{sqrt}(n))$  或者  $\text{sqrt}(n)$  划分。

```

1 signed main() {
2     int n;
3     cin >> n;
4     vector<int> w(n + 1);
5     for (int i = 1; i <= n; i++) {
6         cin >> w[i];
7     }
8
9     int q;
10    cin >> q;
11    vector<array<int, 3>> query(q + 1);
12    for (int i = 1; i <= q; i++) {
13        int l, r;
14        cin >> l >> r;
15        query[i] = {l, r, i};
16    }
17
18    int Knun = n / min<int>(n, sqrt(q)); // 计算块长
19    vector<int> K(n + 1);
20    for (int i = 1; i <= n; i++) { // 固定块长
21        K[i] = (i - 1) / Knun + 1;
22    }
23    sort(query.begin() + 1, query.end(), [&](auto x, auto y) {
24        if (K[x[0]] != K[y[0]]) return x[0] < y[0];
25        if (K[x[0]] & 1) return x[1] < y[1];
26        return x[1] > y[1];
27    });
28
29    int l = 1, r = 0, val = 0;
30    vector<int> ans(q + 1);
31    for (int i = 1; i <= q; i++) {
32        auto [ql, qr, id] = query[i];
33        auto add = [&](int x) -> void {};
34        auto del = [&](int x) -> void {};
35        while (l > ql) add(w[--l]);
36        while (r < qr) add(w[++r]);
37        while (l < ql) del(w[l++]);
38        while (r > qr) del(w[r--]);
39        ans[id] = val;
40    }
41    for (int i = 1; i <= q; i++) {
42        cout << ans[i] << endl;
43    }
44 }

```

需要注意的是, 在普通莫队中,  $K$  数组的作用是根据左边界的值进行排序, 当询问次

数很少时 ( $q \ll n$ ), 可以直接合并到 *query* 数组中。

```

1 vector<array<int, 4>> query(q);
2 for (int i = 1; i <= q; i++) {
3     int l, r;
4     cin >> l >> r;
5     query[i] = {l, r, i, (1 - 1) / Knum + 1}; // 合并
6 }
7 sort(query.begin() + 1, query.end(), [&](auto x, auto y) {
8     if (x[3] != y[3]) return x[3] < y[3];
9     if (x[3] & 1) return x[1] < y[1];
10    return x[1] > y[1];
11 });

```

## 12.6 带修莫队

以  $O(N^{\frac{5}{3}})$  的复杂度完成  $Q$  次询问的离线查询, 其中每个分块的大小取  $N^{\frac{2}{3}} = \sqrt[3]{100000^2} = 2154$  (直接取会略快), 也可以使用  $\text{pow}(n, 0.6666)$  划分。

```

1 signed main() {
2     int n, q;
3     cin >> n >> q;
4     vector<int> w(n + 1);
5     for (int i = 1; i <= n; i++) {
6         cin >> w[i];
7     }
8     vector<array<int, 4>> query = {}; // {左区间, 右区间, 累计修改次数, 下标}
9     vector<array<int, 2>> modify = {}; // {修改的值, 修改的元素下标}
10    for (int i = 1; i <= q; i++) {
11        char op;
12        cin >> op;
13        if (op == 'Q') {
14            int l, r;
15            cin >> l >> r;
16            query.push_back({l, r, (int)modify.size() - 1, (int)query.size()});
17        } else {
18            int idx, w;
19            cin >> idx >> w;
20            modify.push_back({w, idx});
21        }
22    }
23
24    int Knum = 2154; // 计算块长
25    vector<int> K(n + 1);
26    for (int i = 1; i <= n; i++) { // 固定块长
27        K[i] = (i - 1) / Knum + 1;
28    }
29    sort(query.begin() + 1, query.end(), [&](auto x, auto y) {
30        if (K[x[0]] != K[y[0]]) return x[0] < y[0];
31        if (K[x[1]] != K[y[1]]) return x[1] < y[1];
32        return x[3] < y[3];
33    });

```

```

34
35    int l = 1, r = 0, val = 0;
36    int t = 0; // 累计修改次数
37    vector<int> ans(query.size());
38    for (int i = 1; i < query.size(); i++) {
39        auto [ql, qr, qt, id] = query[i];
40        auto add = [&](int x) -> void {};
41        auto del = [&](int x) -> void {};
42        auto time = [&](int x, int l, int r) -> void {};
43        while (l > ql) add(w[--l]);
44        while (r < qr) add(w[++r]);
45        while (l < ql) del(w[l++]);
46        while (r > qr) del(w[r--]);
47        while (t < qt) time(++t, ql, qr);
48        while (t > qt) time(t--, ql, qr);
49        ans[id] = val;
50    }
51    for (int i = 1; i < ans.size(); i++) {
52        cout << ans[i] << endl;
53    }
54 }

```

## 12.7 幻方

构造题用, 其有一些性质 (保证  $N$  为奇数): 1 到  $N^2$  每个数字恰好使用一次, 且每行、每列及两条对角线上的数字之和都相同, 且为奇数。

构造方式: 将 1 写在第一行的中间, 随后不断向右上角位置填下一个数字, 直到填满。

```

1 int n;
2 cin >> n;
3 int x = 1, y = (n + 1) / 2;
4 vector ans(n + 1, vector<int>(n + 1));
5 for (int i = 1; i <= n * n; i++) {
6     ans[x][y] = i;
7     if (!ans[(x - 2 + n) % n + 1][y % n + 1]){
8         x = (x - 2 + n) % n + 1;
9         y = y % n + 1;
10    } else {
11        x = x % n + 1;
12    }
13 }

```

## 12.8 约瑟夫问题

$n$  个人编号  $0, 1, 2, \dots, n - 1$ , 每次数到  $k$  出局, 求最后剩下的人的编号。

$O(N)$ 。

```

1 int jos(int n, int k) {
2     int res = 0;
3     repeat(i, 1, n + 1) res = (res + k) % i;
4     return res; // res+1, 如果编号从1开始

```

5 }

$\mathcal{O}(K \log N)$ ，适用于  $K$  较小的情况。

```

1 int jos(int n,int k){
2     if(n==1 || k==1)return n-1;
3     if(k>n)return (jos(n-1,k)+k)%n; // 线性算法
4     int res=jos(n-n/k,k)-n%k;
5     if(res<0)res+=n; // mod n
6     else res+=res/(k-1); // 还原位置
7     return res; // res+1,如果编号从1开始
8 }

```

## 12.9 舞蹈链

理论复杂度大概在  $\mathcal{O}(c^n)$  左右，其中  $c$  为某个非常接近于 1 的常数, $n$  为矩阵中 1 的个数。

数独求解

```

1 #include<bits/stdc++.h>
2 constexpr int N = 1e6 + 10;
3 int ans[10][10], stk[N];
4
5 struct DLX {
6     static constexpr int MAXSIZE = 1e5 + 10;
7     int n, m, tot, first[MAXSIZE + 10], siz[MAXSIZE + 10];
8     int L[MAXSIZE + 10], R[MAXSIZE + 10], U[MAXSIZE + 10], D[MAXSIZE + 10];
9     int col[MAXSIZE + 10], row[MAXSIZE + 10];
10
11     void build(const int &r, const int &c) { // 进行build操作
12         n = r, m = c;
13         for (int i = 0; i <= c; ++i) {
14             L[i] = i - 1, R[i] = i + 1;
15             U[i] = D[i] = i;
16         }
17         L[0] = c, R[c] = 0, tot = c;
18         memset(first, 0, sizeof(first));
19         memset(siz, 0, sizeof(siz));
20     }
21
22     void insert(const int &r, const int &c) { // 进行insert操作
23         col[++tot] = c, row[tot] = r, ++siz[c];
24         D[tot] = D[c], U[D[c]] = tot, U[tot] = c, D[c] = tot;
25         if (!first[r]) first[r] = L[tot] = R[tot] = tot;
26         else {
27             R[tot] = R[first[r]], L[R[first[r]]] = tot;
28             L[tot] = first[r], R[first[r]] = tot;
29         }
30     }
31
32     void remove(const int &c) { // 进行remove操作
33         int i, j;

```

```

34         L[R[c]] = L[c], R[L[c]] = R[c];
35         for (i = D[c]; i != c; i = D[i])
36             for (j = R[i]; j != i; j = R[j])
37                 U[D[j]] = U[j], D[U[j]] = D[j], --siz[col[j]];
38     }
39
40     void recover(const int &c) { // 进行recover操作
41         int i, j;
42         for (i = U[c]; i != c; i = U[i])
43             for (j = L[i]; j != i; j = L[j]) U[D[j]] = D[U[j]] = j, ++siz[col[j]];
44         L[R[c]] = R[L[c]] = c;
45     }
46
47     bool dance(int dep) { // dance
48         int i, j, c = R[0];
49         if (!R[0]) {
50             for (i = 1; i < dep; ++i) {
51                 int x = (stk[i] - 1) / 9 / 9 + 1;
52                 int y = (stk[i] - 1) / 9 % 9 + 1;
53                 int v = (stk[i] - 1) % 9 + 1;
54                 ans[x][y] = v;
55             }
56             return true;
57         }
58         for (i = R[0]; i != 0; i = R[i])
59             if (siz[i] < siz[c]) c = i;
60         remove(c);
61         for (i = D[c]; i != c; i = D[i]) {
62             stk[dep] = row[i];
63             for (j = R[i]; j != i; j = R[j]) remove(col[j]);
64             if (dance(dep + 1)) return true;
65             for (j = L[i]; j != i; j = L[j]) recover(col[j]);
66         }
67         recover(c);
68         return false;
69     }
70 } solver;
71
72 int GetId(int row, int col, int num) {
73     return (row - 1) * 9 * 9 + (col - 1) * 9 + num;
74 }
75
76 void Insert(int row, int col, int num) {
77     int dx = (row - 1) / 3 + 1;
78     int dy = (col - 1) / 3 + 1;
79     int room = (dx - 1) * 3 + dy;
80     int id = GetId(row, col, num);
81     int f1 = (row - 1) * 9 + num; // task 1
82     int f2 = 81 + (col - 1) * 9 + num; // task 2
83     int f3 = 81 * 2 + (room - 1) * 9 + num; // task 3
84     int f4 = 81 * 3 + (row - 1) * 9 + col; // task 4
85     solver.insert(id, f1);
86     solver.insert(id, f2);

```

```

87 solver.insert(id, f3);
88 solver.insert(id, f4);
89 }
90 using std::cin;using std::cout;
91 int main() {
92     cin.tie(nullptr)->sync_with_stdio(false);
93     solver.build(729, 324);
94     for (int i = 1; i <= 9; ++i)
95         for (int j = 1; j <= 9; ++j) {
96             cin >> ans[i][j];
97             for (int v = 1; v <= 9; ++v) {
98                 if (ans[i][j] && ans[i][j] != v) continue;
99                 Insert(i, j, v);
100             }
101         }
102     solver.dance(1);
103     for (int i = 1; i <= 9; ++i, cout << '\n')
104         for (int j = 1; j <= 9; ++j, cout << ' ') cout << ans[i][j];
105     return 0;
106 }

```

## 12.10 对拍

### 12.10.1 Windows

```

1 int main(){
2     system("g++ -std=c++20 gen.cpp -o gen.exe");
3     system("g++ -std=c++20 brute.cpp -o brute.exe");
4     system("g++ -std=c++20 std.cpp -o std.exe");
5     for(int i = 1;i <= 1000;i++){
6         system("gen.exe > test.in");
7         system("brute.exe < test.in > brute.out");
8         system("std.exe < test.in > std.out");
9         if(system("fc std.out brute.out")){
10             cout << "Wrong Answer on test " << i << "\n";
11             system("type test.in");
12             system("type brute.out");
13             system("type std.out");
14             return 0;
15         }
16         else{
17             cout << "Accepted on test" << i << ".\n";
18         }
19     }
20     return 0;
21 }

```

### 12.10.2 Linux

```

1 int main(){
2     system("g++ -std=c++20 gen.cpp -o gen");
3     system("g++ -std=c++20 brute.cpp -o brute");

```

```

4     system("g++ -std=c++20 std.cpp -o std");
5     for(int i = 1; i <= 1000; i++){
6         system("./gen > test.in");
7         system("./brute < test.in > brute.out");
8         system("./std < test.in > std.out");
9         if(system("diff std.out brute.out")){
10             cout << "Wrong Answer on test " << i << "\n";
11             system("cat test.in");
12             system("cat brute.out");
13             system("cat std.out");
14             return 0;
15         }
16         else{
17             cout << "Accepted on test " << i << ".\n";
18         }
19     }
20     return 0;
21 }

```

## 12.11 python 高精度

```

1 t = decimal
2 decimal.getcontext().prec = 1000000
3 n = decimal.Decimal(input())
4 t = (4 + n) * (3 + n) * (2 + n) * (1 + n)
5 print(t // 24)

```

## 12.12 乱七八糟

看数据范围（多测总和），变量 shadow，多测清空，longlong，数组开小了，模数不对，MLE？，对拍记得看输出在不在变，输出格式，inf 开小，答案初值，STL 重构导致引用失效，极端情况（n=1），删除调试代码，循环内变量名重复

将二维矩阵旋转 90°:  $b[j][n - j + 1] = a[i][j]$

将二维矩阵旋转 180°:  $b[n - i + 1][n - j + 1] = a[i][j]$

将二维矩阵旋转 270°:  $b[n - j + 1][i] = a[i][j]$